

Chapter 1 : Project Overview

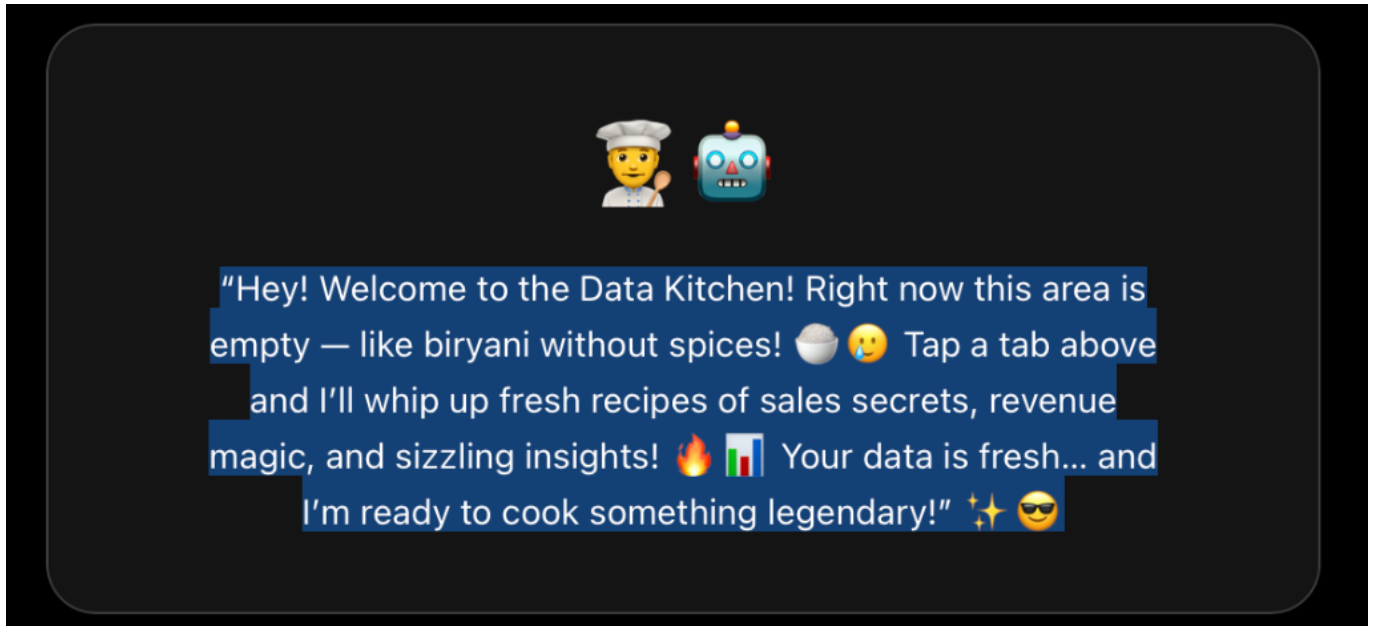
Introduction:

The Enterprise Intelligence Suite is a Reflex-based web application enabling Sales and Logistics data analysis. This manual contains the COMPLETE source code of the project, organized by module.

Reflex Dashboard Comprehensive Manual

Chapter 2 : Visual Gallery

Visual Output: Dashboard Overview



Visual Output: Key Performance Indicators

🛒 DEMAND PLANNING & REPLENISHMENT					
SKU ID	Category	Current Stock	Forecast Demand (M+1)	Reorder Qty	Status
SKU-101	Electronics	20	30	30	Partial Stock
SKU-102	Electronics	200	150	150	Critical Low
SKU-205	Home	200	0	0	Overstock
SKU-310	Fashion	50	85	85	Critical Low
SKU-404	Kitchen	50	0	0	Healthy
SKU-520	Home	200	50	50	Overdue Stock

Chapter 3 : Full Project Codebase

The following pages contain the source code for every Python module in the project.

File: `reflex_dashboard/channel_sales_card_snippet.py`

Source code module.

```
def channel_sales_card() -> rx.Component:
    return rx.card(
        rx.vstack(
            rx.heading("Channel Wise Sale", size="4", color="black", width="100%", text_align="center", margin_bottom="4"), #
            Black text on white card

            rx.flex(
                # Chart
                rx.box(
                    rx.plotly(data=State.channel_sales_chart, height="300px", config={"displayModeBar": False}),
                    width="50%",
                    min_width="300px",
                    display="flex",
                    justify_content="center",
                    align_items="center"
                ),

                # Custom Legend
                rx.vstack(
                    rx.foreach(
                        State.channel_sales_stats,
                        lambda item: rx.hstack(
                            rx.box(width="10px", height="10px", border_radius="50%", bg=item["color"]),
                            rx.text(item["channel"], color="gray.600", font_size="sm", font_weight="medium"),
                            rx.spacer(),
                            rx.text(item["value"], color="black", font_size="sm", font_weight="bold"),
                            width="100%",
                            padding_y="1",
                            border_bottom="1px dashed #E2E8F0",
                            align="center"
                        )
                    ),
                    width="50%",
                    min_width="250px",
                    padding_left="4",
                    spacing="0"
                ),
            ),

            width="100%",
            flex_wrap="wrap",
            align="center",
            justify="center"
        ),
        width="100%",
    ),
    bg="white",
    border_radius="xl",
    box_shadow="lg",
```

Reflex Dashboard Comprehensive Manual

```
padding="6",
width="100%"
)
```

File: reflex_dashboard/colors.py

Source code module.

```
EARTH_TONES = [
    "#2C3E50", "#E67E22", "#27AE60", "#C0392B", "#8E44AD",
    "#2980B9", "#D35400", "#16A085", "#F39C12", "#7F8C8D"
]

# Styling Constants - Pitch Black Mode
# Luxury / High Contrast Dark Palette
CHART_COLORS = ["#3182CE", "#2F855A", "#D69E2E", "#C05621", "#805AD5", "#2C7A7B", "#B7791F", "#2B6CB0", "#276749", "#975A16"]

SIDEBAR_BG = "#000000" # Pitch Black sidebar
CONTENT_BG = "#000000" # Pitch Black content
CARD_BG = "#000000" # Deepest Gray/Black for cards
TEXT_COLOR = "#FFFFFF" # Pure White
ACCENT_COLOR = "#D4AF37" # Metallic Gold for Luxury Accent (was Light Blue)
BORDER_COLOR = "#333333" # Subtle dark borders
```

File: reflex_dashboard/models.py

Source code module.

```
import reflex as rx
from pydantic import BaseModel

class MonthlyValue(BaseModel):
    month: str
    value: str
    is_shaded: bool = False

class StateSummary(BaseModel):
    state: str
    monthly_values: list[MonthlyValue]
    total_value: str

class ChannelSummary(BaseModel):
    channel: str
    monthly_values: list[MonthlyValue]
    total_value: str
    children: list[StateSummary]
    is_total: bool = False

class InvRetChannel(BaseModel):
    channel: str
    inv: str = "-"
    ret: str = "-"
    ret_color: str = "black"
    val: str = "" # For pct row
    color: str = ""

class InvRetRow(BaseModel):
```

Reflex Dashboard Comprehensive Manual

```
month: str
channels: list[InvRetChannel]
grand_total_inv: str = "-"
grand_total_ret: str = "-"
grand_total_val: str = "" # For pct row
is_pct: bool = False
is_total: bool = False
```

File: `reflex_dashboard/reflex_dashboard.py`

Application Entry Point: Configuration and main routing.

```
import reflex as rx
from .components.layouts import index
from .state import State

app = rx.App(
    theme=rx.theme(
        appearance="dark",
        has_background=True,
        radius="large",
        accent_color="amber", # Gold/Amber accent
        gray_color="slate",
    ),
    head_components=[
        rx.script(src="https://cdn.tailwindcss.com"),
    ],
    stylesheets=[
        "https://fonts.googleapis.com/css2?family=Inter:wght@400;700;900&display=swap",
        "https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@0,600;0,700;1,400&family=JetBrains+Mono:wght@400;700&display=swap",
        "/styles.css",
    ],
    style={
        "font_family": "Inter, sans-serif",
        "background_color": "#000000",
        "@keyframes popIn": {
            "0%": {"transform": "scale(0.6)", "opacity": "0"},
            "60%": {"transform": "scale(1.08)", "opacity": "1"},
            "100%": {"transform": "scale(1.0)", "opacity": "1"},
        },
    },
)
app.add_page(index, title="Sales Dashboard")
```

File: `reflex_dashboard/components/ai.py`

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state import State
from ..colors import ACCENT_COLOR, CARD_BG

def ai_chat_component():
    return rx.box(
        rx.vstack(
            rx.heading("Ask AI Assistant", size="4", color=ACCENT_COLOR, margin_bottom="2", width="100%",
```

Reflex Dashboard Comprehensive Manual

```
text_align="center"),
    rx.hstack(
        rx.input(
            placeholder="Ask about your sales data...",
            value=State.current_question,
            on_change=State.set_current_question,
            bg="gray.800",
            color="rgb(160, 117, 139)", # User requested gray
            font_weight="bold", # User requested bold
            width="100%",
            height="28px",
            border="1px solid #4A5568",
            text_align="center",
            font_size="sm",
        ),
        rx.button(
            "Send",
            on_click=State.ask_gemini,
            color_scheme="blue",
            height="28px",
            size="1",
            is_loading=State.is_ai_thinking
        ),
        width="50%",
        margin_x="auto",
        padding_bottom="4"
    ),
    rx.scroll_area(
        rx.vstack(
            rx.foreach(
                State.chat_history,
                lambda msg: rx.box(
                    rx.text(msg["text"], font_size="sm"),
                    bg=rx.cond(msg["role"] == "user", "blue.900", "gray.700"),
                    color=rx.cond(msg["role"] == "user", "blue.100", "white"),
                    padding="3",
                    align_self=rx.cond(msg["role"] == "user", "end", "start"),
                    max_width="80%",
                    border_radius="md",
                )
            ),
            spacing="3",
            align_items="stretch",
            width="100%"
        ),
        height="200px",
        type="always",
        scrollbars="vertical",
        style={"paddingRight": "10px"}
    ),
    padding="4",
    bg=CARD_BG,
    border_radius="xl",
    border="1px solid #4A5568",
    box_shadow="lg"
),
width="100%"
)

def ai_chef_placeholder():
    return rx.box(
```

Reflex Dashboard Comprehensive Manual

```
rx.vstack(
    rx.text("???", font_size="3rem"),
    rx.text(
        rx.text("?Hey! Welcome to the ", as_="span"),
        rx.text("Data Kitchen! ", as_="span", font_weight="bold", color="green"), # Bold Green
        rx.text("Right now this area is empty ? like biryani without spices! ?? ", as_="span"),
        rx.text("Tap a tab", as_="span", font_weight="900", font_size="2rem", color="#FF0000", font_family="Inter",
letter_spacing="1px"), # Big Bold Extreme Red
        rx.text(" above and I'll whip up fresh recipes of ", as_="span"),
        rx.text("sales secrets, revenue magic, and sizzling insights!", as_="span", font_weight="bold",
font_style="italic", color="#FF69B4", font_family="JetBrains Mono"), # Bold Italic Pink Mono
        rx.text(" ?? Your data is fresh? and I'm ready to cook something legendary!? ??", as_="span"),
        text_align="center",
        font_size="1.2rem",
        width="85%",
        line_height="1.6",
    ),
    spacing="4",
    align_items="center",
),
# ? Premium Glass Look
bg="rgba(255,255,255,0.08)",
border_radius="1.7rem",
padding="2.5rem",
backdrop_filter="blur(14px)",
box_shadow="0 8px 32px rgba(0,0,0,0.32)",
border="1.2px solid rgba(255,255,255,0.18)",
color="white",
# ? Bounce Pop Animation
animation="popIn 0.7s ease-out",
# Layout Fit
display="flex",
align_items="center",
justify_content="center",
width="100%",
max_width="700px", # Constrain width for aesthetics
margin_x="auto",
margin_y="auto",
min_height="50vh",
)
```

File: `reflex_dashboard/components/bistro.py`

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state import State

def header_html():
    return """
    <div class="text-center space-y-2 mb-4 max-w-3xl mx-auto pt-2">
        <div class="inline-flex items-center space-x-2 bg-slate-900/80 border border-slate-700 rounded-full px-3 py-1 mb-2
shadow-xl backdrop-blur-sm">
            <span class="w-1.5 h-1.5 rounded-full bg-emerald-500 animate-pulse"></span>
            <span class="text-[0.65rem] font-mono text-slate-400">KITCHEN STATUS: <span
class="text-emerald-400">OPERATIONAL</span></span></div>

        <h1 class="text-3xl md:text-5xl font-serif text-transparent bg-clip-text bg-gradient-to-r from-amber-200 via-amber-400
```

Reflex Dashboard Comprehensive Manual

```
to-amber-600 tracking-tight pb-1">
    The Analytics Bistro
</h1>

<p class="text-sm text-slate-400 font-light leading-relaxed">
    We don't just serve data. We cook it.
</p>
</div>
"""

def visual_html():
    return """
    <div class="w-full max-w-xs mb-6 relative svg-container group mx-auto">
        <svg viewBox="0 0 500 300" xmlns="http://www.w3.org/2000/svg" class="w-full h-auto drop-shadow-2xl rounded-2xl
overflow-hidden bg-slate-900/80 border border-slate-800 ring-1 ring-slate-700/50">
            <!-- Kitchen Background -->
            <rect x="0" y="0" width="500" height="300" fill="#0f172a" />
            <path d="M0,250 L500,250" stroke="#1e293b" stroke-width="2" />

            <!-- Code Elements (Floating in background) -->
            <text x="20" y="40" font-family="JetBrains Mono" font-size="10" fill="#334155" opacity="0.5">SELECT * FROM
raw_data</text>
            <text x="350" y="60" font-family="JetBrains Mono" font-size="10" fill="#334155" opacity="0.5">import
sklearn</text>
            <text x="40" y="100" font-family="JetBrains Mono" font-size="10" fill="#334155"
opacity="0.5">df.groupby('user_id')</text>

            <!-- The Waiter (Robot) -->
            <g class="animate-float">
                <!-- Body -->
                <path d="M230,220 L270,220 L265,140 L235,140 Z" fill="#1e293b" stroke="#475569" stroke-width="2"/>
                <!-- Head -->
                <rect x="230" y="100" width="40" height="35" rx="6" fill="#cbd5e1"/>
                <rect x="238" y="110" width="8" height="8" rx="2" fill="#3b82f6" class="animate-pulse"/>
                <rect x="254" y="110" width="8" height="8" rx="2" fill="#3b82f6" class="animate-pulse" style="animation-delay:
0.5s"/>

                <!-- Bowtie -->
                <path d="M240,140 L260,140 L250,145 Z" fill="#f59e0b"/>

                <!-- Arm Holding Tray -->
                <g class="robot-arm">
                    <path d="M235,180 Q200,180 180,200" stroke="#94a3b8" stroke-width="6" fill="none"/>
                    <circle cx="180" cy="200" r="5" fill="#cbd5e1"/>
                </g>

                <!-- The Platter -->
                <g transform="translate(130, 200)">
                    <ellipse cx="50" cy="0" rx="60" ry="10" fill="#e2e8f0"/>
                    <!-- Holographic Chart Projection -->
                    <path d="M20,-5 L30,-30 L40,-15 L50,-45 L60,-20 L80,-60" fill="none" stroke="#f59e0b" stroke-width="2"
stroke-dasharray="2 2">
                        <animate attributeName="stroke-dashoffset" from="100" to="0" dur="2s" repeatCount="indefinite" />
                    </path>
                    <circle cx="20" cy="-5" r="2" fill="#3b82f6"/>
                    <circle cx="80" cy="-60" r="3" fill="#10b981"/>
                    <!-- Mist -->
                    <path d="M30,-20 Q40,-30 50,-20" stroke="white" stroke-width="1" fill="none" opacity="0.3">
                        <animate attributeName="d" values="M30,-20 Q40,-30 50,-20; M30,-30 Q40,-40 50,-30" dur="2s"
repeatCount="indefinite" />
                        <animate attributeName="opacity" values="0.3;0" dur="2s" repeatCount="indefinite" />
                    </path>
                </g>
            </g>
        </svg>
    </div>
    """
```


Reflex Dashboard Comprehensive Manual

```

        </g>
      </g>
    </svg>
  </div>
  """

def menu_html():
  return """
    <h2 class="text-3xl font-serif text-slate-300 mb-8 border-b border-slate-800 pb-4 text-center">Today's Specials</h2>

    <div class="grid grid-cols-2 md:grid-cols-4 lg:grid-cols-7 gap-4 w-full px-6">

      <!-- Dish 1: Demand Roast -->
      <div class="menu-card bg-slate-900 border border-slate-800 rounded-xl p-5 cursor-pointer group
hover:border-amber-500/50 min-h-[180px] flex flex-col justify-between transition-all hover:scale-105">
        <div>
          <div class="flex justify-between items-start mb-3">
            <div class="p-2 rounded bg-blue-500/10 text-blue-400 group-hover:bg-blue-500 group-hover:text-white
transition-colors">
              <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M22 11.08V12a10 10 0 1
1-5.93-9.14"/><polyline points="22 4 12 14.01 9 11.01"/></svg>
            </div>
            <span class="text-[0.65rem] font-mono text-slate-500">HOT</span>
          </div>
          <h3 class="text-base font-serif font-bold text-slate-100 mb-1 leading-tight">Demand Roast</h3>
          <p class="text-[0.7rem] text-slate-400 mb-0 leading-snug">Predicting future stock needs.</p>
        </div>
        <div class="ingredients absolute inset-0 bg-slate-950/95 backdrop-blur-sm p-4 flex flex-col justify-center
items-start border border-amber-500/30">
          <h4 class="text-amber-400 font-bold text-xs uppercase tracking-wider mb-2">Stack:</h4>
          <ul class="text-[0.7rem] font-mono text-slate-300 space-y-1"><li>> ARIMA</li><li>> Scikit-Learn</li><li>>
History Data</li></ul>
        </div>
      </div>

      <!-- Dish 2: Cluster Curry -->
      <div class="menu-card bg-slate-900 border border-slate-800 rounded-xl p-5 cursor-pointer group
hover:border-amber-500/50 min-h-[180px] flex flex-col justify-between transition-all hover:scale-105">
        <div>
          <div class="flex justify-between items-start mb-3">
            <div class="p-2 rounded bg-purple-500/10 text-purple-400 group-hover:bg-purple-500 group-hover:text-white
transition-colors">
              <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12" r="10"/><path
d="M12 2a14.5 14.5 0 0 0 0 20 14.5 0 0 0 0 0-20"/><path d="M2 12h20"/></svg>
            </div>
            <span class="text-[0.65rem] font-mono text-slate-500">CHEF</span>
          </div>
          <h3 class="text-base font-serif font-bold text-slate-100 mb-1 leading-tight">Cluster Curry</h3>
          <p class="text-[0.7rem] text-slate-400 mb-0 leading-snug">RFM segmentation mix.</p>
        </div>
        <div class="ingredients absolute inset-0 bg-slate-950/95 backdrop-blur-sm p-4 flex flex-col justify-center
items-start border border-amber-500/30">
          <h4 class="text-amber-400 font-bold text-xs uppercase tracking-wider mb-2">Stack:</h4>
          <ul class="text-[0.7rem] font-mono text-slate-300 space-y-1"><li>> K-Means</li><li>> RFM</li><li>>
SQL</li></ul>
        </div>
      </div>

      <!-- Dish 3: Neural Nachos -->
```

Reflex Dashboard Comprehensive Manual

```
<div class="menu-card bg-slate-900 border border-slate-800 rounded-xl p-5 cursor-pointer group
hover:border-amber-500/50 min-h-[180px] flex flex-col justify-between transition-all hover:scale-105">
  <div>
    <div class="flex justify-between items-start mb-3">
      <div class="p-2 rounded bg-emerald-500/10 text-emerald-400 group-hover:bg-emerald-500
group-hover:text-white transition-colors">
        <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M12 2a3 3 0 0 1 3 3v2a3 3 0 0
1-3 3 3 3 0 0 1-3-3v5a3 3 0 0 1 3-3z"/><path d="M19 10v2a7 7 0 0 1-14 0v-2"/><line x1="12" y1="19" x2="12" y2="23"/><line
x1="8" y1="23" x2="16" y2="23"/></svg>
      </div>
      <span class="text-[0.65rem] font-mono text-slate-500">EXP</span>
    </div>
    <h3 class="text-base font-serif font-bold text-slate-100 mb-1 leading-tight">Neural Nachos</h3>
    <p class="text-[0.7rem] text-slate-400 mb-0 leading-snug">Deep fried AI layers.</p>
  </div>
  <div class="ingredients absolute inset-0 bg-slate-950/95 backdrop-blur-sm p-4 flex flex-col justify-center
items-start border border-amber-500/30">
    <h4 class="text-amber-400 font-bold text-xs uppercase tracking-wider mb-2">Stack:</h4>
    <ul class="text-[0.7rem] font-mono text-slate-300 space-y-1"><li>> PyTorch</li><li>> Bayes</li><li>> Linear
Alg</li></ul>
  </div>
</div>

<!-- Dish 4: Market Platter -->
<div class="menu-card bg-slate-900 border border-slate-800 rounded-xl p-5 cursor-pointer group
hover:border-amber-500/50 min-h-[180px] flex flex-col justify-between transition-all hover:scale-105">
  <div>
    <div class="flex justify-between items-start mb-3">
      <div class="p-2 rounded bg-pink-500/10 text-pink-400 group-hover:bg-pink-500 group-hover:text-white
transition-colors">
        <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="9" cy="21" r="1"/><circle
cx="20" cy="21" r="1"/><path d="M1 1h4l2.68 13.39a2 2 0 0 2 1.61h9.72a2 2 0 0 2-1.61L23 6H6"/></svg>
      </div>
      <span class="text-[0.65rem] font-mono text-slate-500">STD</span>
    </div>
    <h3 class="text-base font-serif font-bold text-slate-100 mb-1 leading-tight">Market Platter</h3>
    <p class="text-[0.7rem] text-slate-400 mb-0 leading-snug">Items bought together.</p>
  </div>
  <div class="ingredients absolute inset-0 bg-slate-950/95 backdrop-blur-sm p-4 flex flex-col justify-center
items-start border border-amber-500/30">
    <h4 class="text-amber-400 font-bold text-xs uppercase tracking-wider mb-2">Stack:</h4>
    <ul class="text-[0.7rem] font-mono text-slate-300 space-y-1"><li>> Apriori</li><li>> CLV</li><li>>
AOV</li></ul>
  </div>
</div>

<!-- Dish 5: Matrix Mousse -->
<div class="menu-card bg-slate-900 border border-slate-800 rounded-xl p-5 cursor-pointer group
hover:border-amber-500/50 min-h-[180px] flex flex-col justify-between transition-all hover:scale-105">
  <div>
    <div class="flex justify-between items-start mb-3">
      <div class="p-2 rounded bg-cyan-500/10 text-cyan-400 group-hover:bg-cyan-500 group-hover:text-white
transition-colors">
        <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M3 3v18h18"/><path d="M7 17l4-4
4 4-4"/></svg>
      </div>
      <span class="text-[0.65rem] font-mono text-slate-500">CORE</span>
    </div>
  </div>
```

Reflex Dashboard Comprehensive Manual

```
<h3 class="text-base font-serif font-bold text-slate-100 mb-1 leading-tight">Matrix Mousse</h3>
<p class="text-[0.7rem] text-slate-400 mb-0 leading-snug">Bedrock of data transforms.</p>
</div>
<div class="ingredients absolute inset-0 bg-slate-950/95 backdrop-blur-sm p-4 flex flex-col justify-center
items-start border border-amber-500/30">
  <h4 class="text-amber-400 font-bold text-xs uppercase tracking-wider mb-2">Stack:</h4>
  <ul class="text-[0.7rem] font-mono text-slate-300 space-y-1"><li>> Vectors</li><li>> Eigenvals</li><li>> Dot
Product</li></ul>
</div>
</div>

<!-- Dish 6: Probable Pie -->
<div class="menu-card bg-slate-900 border border-slate-800 rounded-xl p-5 cursor-pointer group
hover:border-amber-500/50 min-h-[180px] flex flex-col justify-between transition-all hover:scale-105">
  <div>
    <div class="flex justify-between items-start mb-3">
      <div class="p-2 rounded bg-orange-500/10 text-orange-400 group-hover:bg-orange-500 group-hover:text-white
transition-colors">
        <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="12" cy="12" r="10"/><path
d="M8 12a4 4 0 1 0 8 0"/></svg>
      </div>
      <span class="text-[0.65rem] font-mono text-slate-500">MATH</span>
    </div>
    <h3 class="text-base font-serif font-bold text-slate-100 mb-1 leading-tight">Probable Pie</h3>
    <p class="text-[0.7rem] text-slate-400 mb-0 leading-snug">Uncertainty quantified.</p>
  </div>
  <div class="ingredients absolute inset-0 bg-slate-950/95 backdrop-blur-sm p-4 flex flex-col justify-center
items-start border border-amber-500/30">
    <h4 class="text-amber-400 font-bold text-xs uppercase tracking-wider mb-2">Stack:</h4>
    <ul class="text-[0.7rem] font-mono text-slate-300 space-y-1"><li>> P-Values</li><li>> Bayesian</li><li>>
Distrib.</li></ul>
  </div>
</div>

<!-- Dish 7: Prophet Pudding -->
<div class="menu-card bg-slate-900 border border-slate-800 rounded-xl p-5 cursor-pointer group
hover:border-amber-500/50 min-h-[180px] flex flex-col justify-between transition-all hover:scale-105">
  <div>
    <div class="flex justify-between items-start mb-3">
      <div class="p-2 rounded bg-red-500/10 text-red-400 group-hover:bg-red-500 group-hover:text-white
transition-colors">
        <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polyline points="23 6 13.5 15.5 8.5
10.5 1 18"/><polyline points="17 6 23 6 23 12"/></svg>
      </div>
      <span class="text-[0.65rem] font-mono text-slate-500">FUTURE</span>
    </div>
    <h3 class="text-base font-serif font-bold text-slate-100 mb-1 leading-tight">Prophet Pudding</h3>
    <p class="text-[0.7rem] text-slate-400 mb-0 leading-snug">Automated trend spotting.</p>
  </div>
  <div class="ingredients absolute inset-0 bg-slate-950/95 backdrop-blur-sm p-4 flex flex-col justify-center
items-start border border-amber-500/30">
    <h4 class="text-amber-400 font-bold text-xs uppercase tracking-wider mb-2">Stack:</h4>
    <ul class="text-[0.7rem] font-mono text-slate-300 space-y-1"><li>> Prophet</li><li>> Seasonality</li><li>>
Trends</li></ul>
  </div>
</div>

</div>
"""
```

Reflex Dashboard Comprehensive Manual

```
def ticker_html():
    return """
    <!-- Tech Stack Ticker -->
    <div class="w-full bg-slate-900 border-t border-slate-800 z-20 mt-12">
      <div class="ticker-wrap">
        <div class="ticker-move">
          <div class="ticker-item">PYTHON PANDAS INSTALLED</div>
          <div class="ticker-item text-amber-500">SQL ENGINE READY</div>
          <div class="ticker-item">LINEAR ALGEBRA LOADED</div>
          <div class="ticker-item text-blue-500">NEURAL NETWORKS ACTIVE</div>
          <div class="ticker-item">EXCEL PARSERS STANDING BY</div>
          <div class="ticker-item text-purple-500">BAYESIAN PROBABILITY: HIGH</div>
          <div class="ticker-item">CLUSTERING ALGORITHMS: K-MEANS</div>
          <div class="ticker-item text-emerald-500">MARKET BASKET ANALYSIS: OPTIMIZED</div>
          <!-- Duplicate for seamless loop -->
          <div class="ticker-item">PYTHON PANDAS INSTALLED</div>
          <div class="ticker-item text-amber-500">SQL ENGINE READY</div>
          <div class="ticker-item">LINEAR ALGEBRA LOADED</div>
          <div class="ticker-item text-blue-500">NEURAL NETWORKS ACTIVE</div>
          <div class="ticker-item">EXCEL PARSERS STANDING BY</div>
          <div class="ticker-item text-purple-500">BAYESIAN PROBABILITY: HIGH</div>
          <div class="ticker-item">CLUSTERING ALGORITHMS: K-MEANS</div>
        </div>
      </div>
    </div>
    """

def bistro_component() -> rx.Component:
    return rx.box(
        # Background Grid (User provided background)
        rx.html("""<div class="absolute inset-0
bg-[linear-gradient(to_right,#1e293b_1px,transparent_1px),linear-gradient(to_bottom,#1e293b_1px,transparent_1px)]
bg-[size:4rem_4rem] [mask-image:radial-gradient(ellipse_60%_50%_at_50%_0%,#000_70%,transparent_100%)] opacity-20
pointer-events-none z-0"></div>"""),

        rx.vstack(
            rx.html(header_html()),
            rx.html(visual_html()),
            rx.html(menu_html()),

            # The Interactive Button
            rx.button(
                rx.hstack(
                    rx.text("Enter the Kitchen", font_size="3xl", font_weight="900"),
                    rx.html("""<svg xmlns="http://www.w3.org/2000/svg" width="32" height="32" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="4" stroke-linecap="round" stroke-linejoin="round"><path d="M5 12h14"/><path d="M12 5l7 7
7"/></svg>"""))
                ),
                class_name="bg-emerald-500 hover:bg-emerald-600 text-slate-900 py-6 px-16 rounded-full transition-all
shadow-xl hover:shadow-emerald-500/30 hover:scale-105 active:scale-95 flex items-center space-x-3 mt-12 cursor-pointer z-10
font-black tracking-wide border-2 border-emerald-400",
                on_click=State.dismiss_intro
            ),

            rx.html(ticker_html()),

            width="100%",
            align="center",
            spacing="0",
            position="relative",
```

Reflex Dashboard Comprehensive Manual

```
        z_index="10"
    ),

    class_name="bg-slate-950 text-slate-100 min-h-screen flex flex-col font-sans selection:bg-amber-500
selection:text-white overflow-x-hidden w-full relative"
)
```

File: `reflex_dashboard/components/charts.py`

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state import State
from ..colors import CARD_BG, TEXT_COLOR

# Channel Sales Card
def channel_sales_card() -> rx.Component:
    return rx.box(
        rx.vstack(
            rx.heading("Channel Wise Sale", size="4", color=TEXT_COLOR, width="100%", text_align="center", margin_bottom="4"),

            rx.flex(
                # Chart
                rx.box(
                    rx.plotly(data=State.channel_sales_chart, height="580px", config={"displayModeBar": False}),
                    width="55%", # Slightly increased width allocation
                    min_width="300px",
                    display="flex",
                    justify_content="center",
                    align_items="center"
                ),

                # Custom Legend
                rx.vstack(
                    rx.foreach(
                        State.channel_sales_stats,
                        lambda item: rx.hstack(
                            rx.box(width="10px", height="10px", border_radius="50%", bg=item["color"]),
                            rx.text(item["channel"], color="white", font_size="sm", font_weight="medium"),
                            rx.spacer(),
                            rx.text(item["value"], color="white", font_size="sm", font_weight="bold"),
                            rx.text(item["pct"], color="white", font_size="xs", font_weight="medium"),
                            width="100%",
                            padding_y="1",
                            border_bottom="1px dashed #4A5568",
                            align="center"
                        )
                    ),
                    width="50%",
                    min_width="250px",
                    padding_left="4",
                    spacing="0"
                ),
            ),

            width="100%",
            flex_wrap="wrap",
            align="center",
            justify="center"
        )
    )
```

Reflex Dashboard Comprehensive Manual

```
    ),
    width="100%",
),
bg="#000000",
border_radius="xl",
box_shadow="lg",
border="1px solid #4A5568",
padding="6",
width="100%",
height="100%"
)
```

File: reflex_dashboard/components/common.py

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..colors import CARD_BG, BORDER_COLOR

def table_container(title, subtitle, bg_color, content, footer=None):
    return rx.box(
        rx.flex(
            rx.box(
                rx.text(title, font_size="lg", font_weight="900", color="white", letter_spacing="0.05em", text_align="center",
width="100%"),
                rx.text(subtitle, font_size="xs", font_weight="bold", color="white", text_align="center", width="100%",
letter_spacing="0.05em"),
                width="100%",
            ),
            align="center",
            justify="center",
            padding="4",
            border_bottom=f"1px solid {BORDER_COLOR}",
            border_radius="xl xl 0 0", # Radius top only
            bg="rgba(255, 255, 255, 0.02)"
        ),
        content,
        footer if footer is not None else rx.fragment(),
        bg=CARD_BG,
        border_radius="xl",
        border=f"1px solid {BORDER_COLOR}",
        box_shadow="0 4px 20px rgba(0, 0, 0, 0.5)", # Deeper shadow
    )

def trend_badge(trend: str, text_color: str = None, font_size: str = "0.7em"):
    """Helper to render the trend badge."""
    if trend is None:
        trend = ""
    if text_color is None:
        trend_color = rx.cond(
            (trend.contains("-") | (trend.contains("?"))),
            "#D0312D", # Pitch Red
            "#00C851" # Good Green
        )
    else:
        trend_color = text_color

    return rx.el.span(
        f"{trend}",
```

Reflex Dashboard Comprehensive Manual

```
style={
    "backgroundColor": "white",
    "color": trend_color,
    "fontSize": font_size,
    "fontWeight": "bold",
    "padding": "2px 6px",
    "borderRadius": "6px",
    "marginLeft": "8px",
    "position": "relative",
    "top": "-2px", # Adjusted for alignment in various contexts
    "boxShadow": "0 2px 4px rgba(0,0,0,0.1)",
    "whiteSpace": "nowrap",
}
)

def placeholder_tab(title: str) -> rx.Component:
    return rx.center(
        rx.vstack(
            rx.icon("construction", size=48, color="#555"),
            rx.heading(title, size="6", color="white"),
            rx.text("Module under development", color="gray.500"),
            spacing="4",
            align="center"
        ),
        height="50vh",
        width="100%"
    )
```

File: reflex_dashboard/components/customer.py

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state.customer import CustomerState
from ..colors import CARD_BG, BORDER_COLOR, TEXT_COLOR
from .customer_behavior import behavioral_metrics_section

def section_header(icon: str, title: str, subtitle: str = None) -> rx.Component:
    return rx.vstack(
        rx.hstack(
            rx.icon(icon, size=18, color=TEXT_COLOR), # Slightly larger icon
            rx.text(title, font_weight="900", color=TEXT_COLOR, font_size="sm", letter_spacing="0.1em"),
            spacing="3",
            align="center"
        ),
        rx.cond(
            subtitle is not None,
            rx.text(subtitle, font_size="10px", color="white", font_weight="bold", letter_spacing="0.05em"), # Changed to
white
            rx.fragment()
        ),
        spacing="1",
        width="100%",
        padding_bottom="4",
        border_bottom=f"1px solid {BORDER_COLOR}",
        margin_bottom="4",
        align="center"
    )
```

Reflex Dashboard Comprehensive Manual

```
def rfm_radar_card() -> rx.Component:
    return rx.box(
        section_header("target", "RFM BEHAVIORAL RADAR", "ACTIVITY VS VALUE DISTRIBUTION"),
        rx.center(
            rx.plotly(data=CustomerState.rfm_radar_chart, height="660px"),
            width="100%",
            height="100%",
        ),
        rx.box(
            rx.text("INTERPRETATION: This radar chart maps the customer base across 5 key dimensions. High scores in 'Recency'
and 'Frequency' indicate active engagement, while 'Bundling' shows cross-category purchasing behavior.", font_size="xs",
color="white", text_align="center"), # Changed to white
            bg="rgba(255, 255, 255, 0.05)",
            padding="3",
            border_radius="md",
            width="100%",
            margin_top="4"
        ),
        bg=CARD_BG,
        width="100%",
        border=f"1px solid {BORDER_COLOR}",
        border_radius="xl",
        padding="6",
        height="100%",
        box_shadow="lg"
    )

def market_basket_row(item: dict) -> rx.Component:
    """Row for Market Basket Grid."""
    return rx.box(
        rx.vstack(
            rx.hstack(
                rx.text(item["pairs"], color="white", font_weight="bold", font_size="sm"),
                rx.spacer(),
                rx.badge(f"LIFT: {item['lift']}X", color_scheme="cyan", variant="solid", size="1"),
                width="100%",
                padding_bottom="2"
            ),
            rx.hstack(
                rx.text("Confidence", color="white", font_size="xs", font_weight="bold"), # Changed to white
                rx.spacer(),
                rx.text(f"{item['confidence']}%", color="white", font_size="xs", font_weight="bold"),
                rx.box(
                    rx.text("SUPPORT", font_size="9px", color="white", font_weight="bold"), # Changed to white
                    rx.text(f"{item['support']}%", font_size="sm", color="white", font_weight="900"),
                    text_align="right",
                    margin_left="4"
                ),
                width="100%",
                align="end"
            ),
            # Progress Bar for Confidence
            rx.box(
                rx.box(
                    width=f"{item['confidence']}%",
                    height="100%",
                    bg="linear-gradient(90deg, #F56565 0%, #48BB78 100%)", # Red to Green Gradient
                    border_radius="full"
                ),
                width="80%", # Bar takes up partial width
                height="6px",
```


Reflex Dashboard Comprehensive Manual

```
        bg="rgba(255,255,255,0.1)",
        border_radius="full",
        margin_top="1" # Slight spacing
    ),
    spacing="1",
    width="100%",
    padding_x="4" # Added padding for readability
),
bg="rgba(255,255,255,0.03)", # Slight lighter bg for row
border_radius="md",
padding="4",
width="100%",
border=f"1px solid {BORDER_COLOR}"
)

def market_basket_card() -> rx.Component:
    return rx.box(
        section_header("shopping-cart", "MARKET BASKET: ASSOCIATION GRID", "CROSS-SELL / BUNDLING AFFINITY"),
        rx.vstack(
            rx.foreach(
                CustomerState.market_basket_data,
                market_basket_row
            ),
            width="100%",
            spacing="3",
            overflow_y="auto",
            height="660px" # Fixed height
        ),
        rx.box(
            rx.text("INTERPRETATION: Identifies products frequently bought together. 'Lift' > 1 indicates a strong positive
association. High 'Confidence' means if Product A is bought, Product B is highly likely to be bought.", font_size="xs",
color="white", text_align="center"), # Changed to white
            bg="rgba(255, 255, 255, 0.05)",
            padding="3",
            border_radius="md",
            width="100%",
            margin_top="4"
        ),
        bg=CARD_BG,
        width="100%",
        border=f"1px solid {BORDER_COLOR}",
        border_radius="xl",
        padding="6",
        height="100%",
        box_shadow="lg"
    )

def revenue_card() -> rx.Component:
    return rx.box(
        section_header("refresh-cw", "NEW VS RETURNING REVENUE"), # Icon roughly matches cycle
        rx.center(
            rx.plotly(data=CustomerState.revenue_trends_chart, height="660px"),
            width="100%",
        ),
        rx.hstack(
            rx.text("Apr", font_size="9px", color="white"), # Changed to white
            rx.spacer(),
            rx.text("Feb (F)", font_size="9px", color="white"), # Changed to white
            width="100%",
            padding_x="2"
        ),
    ),
```

Reflex Dashboard Comprehensive Manual

```
    rx.box(
      rx.text("INTERPRETATION: Tracks revenue contribution from New vs Returning customers over time. A rising Returning
Revenue (Purple) trend indicates strong retention and customer loyalty.", font_size="xs", color="white", text_align="center"),
# Changed to white
      bg="rgba(255, 255, 255, 0.05)",
      padding="3",
      border_radius="md",
      width="100%",
      margin_top="4"
    ),
    bg=CARD_BG,
    width="100%",
    border=f"1px solid {BORDER_COLOR}",
    border_radius="xl",
    padding="6",
    height="100%",
    box_shadow="lg"
  )
)

def wallet_share_card() -> rx.Component:
  return rx.box(
    section_header("bar-chart", "CUSTOMER WALLET SHARE", "EXPANSION"),
    rx.center(
      rx.plotly(data=CustomerState.wallet_share_chart, height="660px"),
      width="100%",
    ),
    rx.hstack(
      rx.text("Apr", font_size="9px", color="white"), # Changed to white
      rx.spacer(),
      rx.text("Feb (F)", font_size="9px", color="white"), # Changed to white
      width="100%",
      padding_x="2"
    ),
    rx.box(
      rx.text("INTERPRETATION: Shows the percentage of customer spend captured by us. An upward trend suggests we are
becoming the primary vendor for our customers, displacing competitors.", font_size="xs", color="white", text_align="center"),
# Changed to white
      bg="rgba(255, 255, 255, 0.05)",
      padding="3",
      border_radius="md",
      width="100%",
      margin_top="4"
    ),
    bg=CARD_BG,
    width="100%",
    border=f"1px solid {BORDER_COLOR}",
    border_radius="xl",
    padding="6",
    height="100%",
    box_shadow="lg"
  )
)

def ltv_segments_card() -> rx.Component:
  return rx.box(
    section_header("database", "PREDICTIVE LTV SEGMENTS"),
    rx.center(
      rx.plotly(data=CustomerState.ltv_segments_chart, height="660px"),
      width="100%",
    ),
    # Custom Legend
    rx.hstack(
```

Reflex Dashboard Comprehensive Manual

```
        rx.hstack(rx.box(width="10px", height="10px", bg="#39FF14"), rx.text("Champions", font_size="9px", color="white"),
align="center", spacing="1"), # Changed to white
        rx.hstack(rx.box(width="10px", height="10px", bg="#A0AEC0"), rx.text("Loyal", font_size="9px", color="white"),
align="center", spacing="1"), # Changed to white
        rx.hstack(rx.box(width="10px", height="10px", bg="#718096"), rx.text("Potential", font_size="9px", color="white"),
align="center", spacing="1"), # Changed to white
        rx.hstack(rx.box(width="10px", height="10px", bg="#FF0000"), rx.text("Churning", font_size="9px", color="white"),
align="center", spacing="1"), # Changed to white
        spacing="3",
        width="100%",
        justify="center",
        margin_top="2"
    ),
    rx.box(
        rx.text("INTERPRETATION: Segment distribution based on predicted Lifetime Value (LTV). 'Champions' generate the
most value. 'Churning' customers require immediate re-engagement campaigns.", font_size="xs", color="white",
text_align="center"), # Changed to white
        bg="rgba(255, 255, 255, 0.05)",
        padding="3",
        border_radius="md",
        width="100%",
        margin_top="4"
    ),
    bg=CARD_BG,
    width="100%",
    border=f"1px solid {BORDER_COLOR}",
    border_radius="xl",
    padding="6",
    height="100%",
    box_shadow="lg"
)

def customer_tab_content() -> rx.Component:
    return rx.vstack(
        rfm_radar_card(),
        market_basket_card(),
        revenue_card(),
        wallet_share_card(),
        ltv_segments_card(),
        behavioral_metrics_section(),
        spacing="9",
        width="100%",
        padding_bottom="40px"
    )
```

File: reflex_dashboard/components/customer_behavior.py

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state.customer import CustomerBehaviorState
from ..colors import CARD_BG, BORDER_COLOR, TEXT_COLOR

def subsection_header(title: str, subtitle: str = "") -> rx.Component:
    return rx.vstack(
        rx.heading(title, size="4", color="white", font_weight="bold"),
        rx.cond(
            subtitle != "",
            rx.text(subtitle, size="2", color="white"), # Changed to white
        )
    )
```

Reflex Dashboard Comprehensive Manual

```
        rx.fragment()
    ),
    spacing="1",
    margin_bottom="4"
)

def aov_calculator_card() -> rx.Component:
    return rx.box(
        subsection_header("Purchasing Depth", "Key Metric: Average Order Value (AOV)"),
        rx.text("AOV = Total Revenue / Total Orders", color="white", font_size="xs", margin_bottom="4"), # Changed to white
        rx.text("Measures the spending power per transaction.", color="white", font_size="sm", margin_bottom="4"), # Changed
to white

        rx.vstack(
            rx.text("Try it:", color="white", font_weight="bold", font_size="sm"),
            rx.grid(
                rx.vstack(
                    rx.text("Rev ($)", color="white", font_size="xs"), # Changed to white
                    rx.input(
                        placeholder="0.00",
                        on_change=CustomerBehaviorState.set_aov_revenue,
                        bg="rgba(0,0,0,0.5)",
                        border_color="gray.700",
                        color="white"
                    ),
                ),
                spacing="1"
            ),
            rx.vstack(
                rx.text("Orders", color="white", font_size="xs"), # Changed to white
                rx.input(
                    placeholder="0",
                    on_change=CustomerBehaviorState.set_aov_orders,
                    bg="rgba(0,0,0,0.5)",
                    border_color="gray.700",
                    color="white"
                ),
            ),
            spacing="1"
        ),
        columns="2",
        spacing="2",
        width="100%"
    ),
    rx.button(
        "Calculate",
        on_click=CustomerBehaviorState.calculate_aov,
        width="100%",
        color_scheme="blue",
        size="2"
    ),
    rx.center(
        rx.heading(
            f"${CustomerBehaviorState.calculated_aov}",
            color="#48BB78",
            size="6",
            font_weight="900"
        ),
        ),
        bg="rgba(72, 187, 120, 0.1)",
        padding="4",
        border_radius="md",
        width="100%",
        border="1px dashed #48BB78"
```

Reflex Dashboard Comprehensive Manual

```
    ),
    spacing="4",
    width="100%"
  ),
  bg=CARD_BG,
  border=f"1px solid {BORDER_COLOR}",
  padding="6",
  border_radius="xl",
  height="100%"
)

def basket_breadth_card() -> rx.Component:
  return rx.box(
    subsection_header("Basket Breadth", "Insight: Items per Basket (IPB)"),
    rx.vstack(
      rx.hstack(
        rx.icon("shopping-bag", size=24, color="#F6E05E"),
        rx.text("Items per Basket (IPB)", color="white", font_weight="bold", font_size="md"),
        align="center",
        spacing="3"
      ),
      rx.box(
        rx.text("High IPB: Indicates a 'one-stop shop' behavior (Complete restock).", color="white", font_size="sm"),
        # Changed to white
        margin_top="2"
      ),
      rx.box(
        rx.text("Low IPB: Indicates 'specific intent' (e.g., buying just one specific SKU).", color="white",
font_size="sm"), # Changed to white
        margin_top="2"
      ),
      rx.divider(margin_y="4", border_color="gray.800"),
      rx.callout.root(
        rx.callout.icon(rx.icon("lightbulb")),
        rx.callout.text(
          "Strategy: Bundle low-IPB items with bestsellers to increase basket size.",
          color="white"
        ),
        color_scheme="yellow",
        variant="surface",
        size="1"
      ),
      spacing="2"
    ),
    bg=CARD_BG,
    border=f"1px solid {BORDER_COLOR}",
    padding="6",
    border_radius="xl",
    height="100%"
  )

def purchasing_rhythm_card() -> rx.Component:
  return rx.box(
    subsection_header("Purchasing Rhythm", "Timing: Time Between Purchases (TBP)"),
    rx.text("Crucial for FMCG. Helps define the 'nudge' window.", color="white", font_size="sm", margin_bottom="4"), #
    Changed to white

    rx.center(
      rx.vstack(
        rx.hstack(
          rx.box(rx.text("Purchase 1", font_size="xs", weight="bold", color="white"), bg="gray.700", padding="2",
```

Reflex Dashboard Comprehensive Manual

```
border_radius="md"), # Changed to white
    rx.icon("arrow-right", color="white"), # Changed to white
    rx.box(
        rx.text("7 Days (Avg)", font_size="xs", weight="bold", color="#00BFFF"),
        border="1px dashed #00BFFF", padding="2", border_radius="md"
    ),
    rx.icon("arrow-right", color="white"), # Changed to white
    rx.box(rx.text("Purchase 2", font_size="xs", weight="bold", color="white"), bg="gray.700", padding="2",
border_radius="md"), # Changed to white
    align="center",
    spacing="2"
),
    rx.text("If a user hasn't bought in 10 days, they are at risk.", color="#F56565", font_size="xs",
font_style="italic")
),
    bg="rgba(0,0,0,0.3)",
    padding="4",
    border_radius="lg",
    width="100%"
),
    bg=CARD_BG,
    border=f"1px solid {BORDER_COLOR}",
    padding="6",
    border_radius="xl",
    height="100%"
)

def loyalty_health_card() -> rx.Component:
    return rx.box(
        subsection_header("Loyalty Program Health", "Point Redemption Rate (PRR)"),
        rx.vstack(
            rx.text("PRR = Points Redeemed / Points Issued", color="white", font_size="xs"), # Changed to white
            rx.text("A low PRR suggests the loyalty program isn't 'sticky' enough.", color="white", font_size="sm"), # Changed
to white

            rx.box(height="10px"),

            rx.text("Redemption Velocity", color="white", font_weight="bold", font_size="sm"),
            rx.box(
                rx.box(width="65%", height="100%", bg="#9D00FF", border_radius="full"),
                width="100%", height="8px", bg="gray.800", border_radius="full"
            ),
            rx.hstack(
                rx.text("Earn First Point", font_size="xs", color="white"), # Changed to white
                rx.spacer(),
                rx.text("65% Speed Score", font_size="xs", color="#9D00FF", font_weight="bold"),
                rx.spacer(),
                rx.text("First Redemption", font_size="xs", color="white"), # Changed to white
                width="100%"
            ),
            rx.text("Measures how fast a user engages with rewards after joining.", color="white", font_size="xs",
margin_top="2"), # Changed to white
        ),
        bg=CARD_BG,
        border=f"1px solid {BORDER_COLOR}",
        padding="6",
        border_radius="xl",
        height="100%"
    )

def cohort_analysis_section() -> rx.Component:
```

Reflex Dashboard Comprehensive Manual

```
return rx.box(
    rx.heading("Cohort Analysis: The Gold Standard", size="5", color="white", margin_bottom="2"),
    rx.text("Cohort analysis groups users by their 'Acquisition Month' to track retention over time. This visualization
prevents seasonal spikes from hiding long-term churn issues.", color="white", margin_bottom="6"), # Changed to white

    rx.grid(
        rx.center(
            rx.plotly(data=CustomerBehaviorState.cohort_heatmap_chart, height="350px", width="100%",
width="100%",
bg="#000000",
padding="4",
border_radius="xl",
border=f"1px solid {BORDER_COLOR}")
        ),
        rx.vstack(
            rx.heading("Retention Heatmap", size="3", color="white"),
            rx.hstack(
                rx.text("Low", color="white", font_size="xs"), # Changed to white
                rx.box(width="100px", height="8px", bg="linear-gradient(90deg, #0E1117, #48BB78)"),
                rx.text("High", color="#48BB78", font_size="xs"),
                align="center",
                spacing="2"
            ),
            rx.text("Insight: Notice the drop in Month 3? This indicates a failure in the 'Early Loyalty' phase.",
color="#F56565", font_weight="bold", font_size="sm", border_left="3px solid #F56565", padding_left="3"),
            align="start",
            justify="center",
            padding="6",
            bg=CARD_BG,
            border_radius="xl",
            border=f"1px solid {BORDER_COLOR}"
        ),
        columns={"initial": "1", "lg": "2"},
        spacing="4",
        width="100%"
    )
)

def predictive_ai_section() -> rx.Component:
    return rx.box(
        rx.heading("Transitioning to AI: Churn Prediction", size="5", color="white", margin_bottom="2"),
        rx.text("Moving from reactive analysis to proactive engineering. A true AI Engineer builds predictive models to
identify who will churn before it happens using Machine Learning pipelines.", color="white", margin_bottom="6"), # Changed to
white

        rx.grid(
            # Left: Explanation & Feat Eng
            rx.vstack(
                rx.text("Feature Engineering for ML", color="white", font_weight="bold", font_size="md"),
                rx.text("Raw data must be transformed into 'features' for algorithms like XGBoost.", color="white",
font_size="sm"), # Changed to white

                rx.table.root(
                    rx.table.header(
                        rx.table.row(
                            rx.table.column_header_cell("Feature", color="white"),
                            rx.table.column_header_cell("Description", color="white"),
                        )
                    ),
                    rx.table.body(
                        rx.table.row(rx.table.cell("Recency (R)", color="white"), rx.table.cell("Days since last purchase.",
```

Reflex Dashboard Comprehensive Manual

```
color="white")), # Changed to white
    rx.table.row(rx.table.cell("Frequency (F)", color="white"), rx.table.cell("Total purchases in last 180
days.", color="white")), # Changed to white
    rx.table.row(rx.table.cell("Monetary (M)", color="white"), rx.table.cell("Total lifetime spend.",
color="white")), # Changed to white
    rx.table.row(rx.table.cell("Point Balance", color="white"), rx.table.cell("Unspent loyalty points
(retention signal).", color="white")), # Changed to white
  ),
  width="100%",
),
bg=CARD_BG,
padding="6",
border_radius="xl",
border=f"1px solid {BORDER_COLOR}",
height="100%"
),

# Middle: Pipeline Flow
rx.vstack(
  rx.text("The Predictive Pipeline", color="white", font_weight="bold", font_size="md", margin_bottom="4"),
  rx.vstack(
    rx.box(rx.text("1. Labeling", weight="bold", color="#F6E05E"), rx.text("Define 'Churn' (e.g. No purchase >
30 days)", font_size="xs", color="white"), bg="rgba(246, 224, 94, 0.1)", padding="3", border_radius="md", width="100%",
border="1px dashed #F6E05E"), # Changed to white
    rx.icon("arrow-down", color="white"), # Changed to white
    rx.box(rx.text("2. Training", weight="bold", color="#4299E1"), rx.text("Use 6-month historical data to
predict past behavior.", font_size="xs", color="white"), bg="rgba(66, 153, 225, 0.1)", padding="3", border_radius="md",
width="100%", border="1px dashed #4299E1"), # Changed to white
    rx.icon("arrow-down", color="white"), # Changed to white
    rx.box(rx.text("3. Inference", weight="bold", color="#48BB78"), rx.text("Apply model to current data for
'Probability Scores'.", font_size="xs", color="white"), bg="rgba(72, 187, 120, 0.1)", padding="3", border_radius="md",
width="100%", border="1px dashed #48BB78"), # Changed to white
    align="center",
    width="100%"
  ),
  bg=CARD_BG,
  padding="6",
  border_radius="xl",
  border=f"1px solid {BORDER_COLOR}",
  height="100%"
),

# Right: Feature Importance Chart
rx.vstack(
  rx.text("Model Output: Feature Importance", color="white", font_weight="bold", font_size="md"),
  rx.text("Which factors most strongly predict if a customer will leave?", color="white", font_size="sm"), #
Changed to white
  rx.plotly(data=CustomerBehaviorState.feature_importance_chart, height="300px", width="100%"),
  rx.button("Simulate Model Run", variant="outline", color_scheme="green", size="1", width="100%"),
  bg=CARD_BG,
  padding="6",
  border_radius="xl",
  border=f"1px solid {BORDER_COLOR}",
  height="100%"
),

columns={"initial": "1", "xl": "3"},
spacing="4",
width="100%"
)
)
```


Reflex Dashboard Comprehensive Manual

```
def strategic_recommendations_section() -> rx.Component:
    return rx.vstack(
        rx.heading("Strategic Recommendations", size="5", color="white"),
        rx.text("Data is useless without action. Here are three high-impact strategies derived from advanced analytics.",
color="white", margin_bottom="4"), # Changed to white

        rx.grid(
            # Card 1
            rx.box(
                rx.hstack(rx.icon("crosshair", color="#F6E05E", size=24), rx.heading("Micro-Segmentation", size="3",
color="white"), spacing="3", align="center", margin_bottom="3"),
                rx.text("Move beyond 'Champions'. Use K-Means Clustering to find hidden groups like 'High-Value Occasional
Buyers'."), color="white", font_size="sm", margin_bottom="3"), # Changed to white
                rx.box(rx.text("Action: Create specific campaigns for festival seasons targeting this cluster.",
font_weight="bold", color="white", font_size="xs"), bg="rgba(246, 224, 94, 0.1)", padding="3", border_radius="md"),
                bg=CARD_BG, padding="6", border_radius="xl", border=f"1px solid {BORDER_COLOR}"
            ),
            # Card 2
            rx.box(
                rx.hstack(rx.icon("zap", color="#4299E1", size=24), rx.heading("Next Best Action (NBA)", size="3",
color="white"), spacing="3", align="center", margin_bottom="3"),
                rx.text("Don't send generic coupons. Use Market Basket Analysis to predict the exact SKU a customer wants
next.", color="white", font_size="sm", margin_bottom="3"), # Changed to white
                rx.box(rx.text("Action: If they bought 'Product A', recommend 'Product B' (80% affinity).",
font_weight="bold", color="white", font_size="xs"), bg="rgba(66, 153, 225, 0.1)", padding="3", border_radius="md"),
                bg=CARD_BG, padding="6", border_radius="xl", border=f"1px solid {BORDER_COLOR}"
            ),
            # Card 3
            rx.box(
                rx.hstack(rx.icon("wallet", color="#48BB78", size=24), rx.heading("Wallet Share Analysis", size="3",
color="white"), spacing="3", align="center", margin_bottom="3"),
                rx.text("Estimate your share of the customer's total category budget compared to competitors.", color="white",
font_size="sm", margin_bottom="3"), # Changed to white
                rx.box(rx.text("Action: Identify high-spending users with low wallet share and target aggressively.",
font_weight="bold", color="white", font_size="xs"), bg="rgba(72, 187, 120, 0.1)", padding="3", border_radius="md"),
                bg=CARD_BG, padding="6", border_radius="xl", border=f"1px solid {BORDER_COLOR}"
            ),
            columns={"initial": "1", "lg": "3"},
            spacing="4",
            width="100%"
        ),
        width="100%",
        spacing="4"
    )

def behavioral_metrics_section() -> rx.Component:
    """The main container for the new Behavioral Metrics section."""
    return rx.vstack(
        rx.divider(margin_y="8", border_color="gray.800"),
        rx.heading("Beyond the Dashboard: Behavioral Metrics", size="7", color="white", margin_bottom="6"),
        rx.text("Current reporting often focuses on 'what' happened. To understand the 'how' and 'why', we must analyze the
depth and rhythm of transactions.", color="white", font_size="lg", margin_bottom="8"), # Changed to white

        # Row 1: Interactive Cards
        rx.grid(
            aov_calculator_card(),
            basket_breadth_card(),
            purchasing_rhythm_card(),
            loyalty_health_card(),
            columns={"initial": "1", "md": "2", "xl": "4"},

```

Reflex Dashboard Comprehensive Manual

```
        spacing="4",
        width="100%"
    ),

    rx.box(height="40px"),

    # Row 2: Cohort Analysis
    cohort_analysis_section(),

    rx.box(height="40px"),

    # Row 3: Predictive AI
    predictive_ai_section(),

    rx.box(height="40px"),

    # Row 4: Strategic Recs
    strategic_recommendations_section(),

    rx.box(height="60px"), # Bottom padding

    width="100%",
    spacing="6"
)
```

File: `reflex_dashboard/components/empty_state.py`

State Management Module: Handles data logic and events.

```
import reflex as rx
from ..state import State

def no_data_view() -> rx.Component:
    """The empty state view shown when no data is loaded."""
    return rx.center(
        rx.vstack(
            rx.heading("Upload Your Data", size="8", color="#1A202C", margin_bottom="8px"),
            rx.text(
                "Follow the steps to upload Sales and Courier data.",
                color="#718096",
                font_size="16px",
                margin_bottom="32px",
            ),
        ),

        rx.tabs.root(
            rx.tabs.list(
                rx.tabs.trigger(
                    rx.hstack(
                        rx.icon("file-spreadsheet", size=24),
                        rx.text("1. Sales Data", font_size="20px", weight="bold"),
                        spacing="3",
                        align="center",
                    ),
                    value="sales",
                    color="#1A202C",
                    padding_x="32px",
                    padding_y="16px",
                ),
                rx.tabs.trigger(
```

Reflex Dashboard Comprehensive Manual

```
rx.hstack(  
  rx.icon("truck", size=24),  
  rx.text("2. Courier Data", font_size="20px", weight="bold"),  
  spacing="3",  
  align="center",  
)  
),  
value="courier",  
color="#1A202C",  
padding_x="32px",  
padding_y="16px",  
)  
),  
rx.tabs.trigger(  
  rx.hstack(  
    rx.icon("bar-chart-2", size=24),  
    rx.text("3. Analyze", font_size="20px", weight="bold"),  
    spacing="3",  
    align="center",  
  ),  
  value="analyze",  
  color="#1A202C",  
  padding_x="32px",  
  padding_y="16px",  
)  
)  
),  
justify="center", # Center the tabs  
spacing="8", # Substantial space between tabs  
margin_bottom="32px",  
)  
  
# TAB 1: SALES DATA  
rx.tabs.content(  
  rx.vstack(  
    rx.box(  
      rx.vstack(  
        rx.icon("file-spreadsheet", size=32, color="#5B45FF"),  
        rx.text("Upload Sales Data Excel File", weight="bold", size="4", color="#1A202C"),  
        rx.text("Required for dashboard visualization", size="2", color="#718096"),  
        rx.cond(  
          State.sales_data_uploaded,  
          rx.badge("? File Uploaded Successfully", color_scheme="green", variant="solid", size="3"),  
          rx.fragment()  
        ),  
      ),  
      rx.upload(  
        rx.button(  
          "Select Sales File",  
          size="3",  
          variant="solid", # Solid for better visibility on white  
          color_scheme="indigo",  
          width="100%",  
        ),  
        id="sales_tab_upload",  
        accept={  
          "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet": [".xlsx"],  
          "application/vnd.ms-excel": [".xls"]  
        },  
        max_files=1,  
        on_drop=State.handle_sales_upload,  
        border="1px dashed #CBD5E0",  
        padding="32px",  
        border_radius="lg",  
        width="100%",  
      ),  
    ),  
  ),  
)
```

Reflex Dashboard Comprehensive Manual

```
        spacing="4",
        align="center",
    ),
    padding="32px",
    bg="#F7FAFC",
    border_radius="xl",
    width="100%",
    border="1px solid #E2E8F0", # Add border for definition
),
),
value="sales",
padding="24px",
),

# TAB 2: COURIER DATA
rx.tabs.content(
    rx.vstack(
        rx.box(
            rx.vstack(
                rx.icon("truck", size=32, color="#3182CE"),
                rx.text("Upload Courier Data Excel File", weight="bold", size="4", color="#1A202C"),
                rx.text("Required for courier performance metrics", size="2", color="#718096"),
                rx.cond(
                    State.courier_data_uploaded,
                    rx.badge("? File Uploaded Successfully", color_scheme="green", variant="solid", size="3"),
                    rx.fragment()
                ),
            ),
            rx.upload(
                rx.button(
                    "Select Courier File",
                    size="3",
                    variant="solid", # Solid
                    color_scheme="blue",
                    width="100%",
                ),
                id="courier_tab_upload",
                accept={
                    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet": [".xlsx"],
                    "application/vnd.ms-excel": [".xls"]
                },
                max_files=1,
                on_drop=State.handle_courier_upload,
                border="1px dashed #CBD5E0",
                padding="32px",
                border_radius="lg",
                width="100%",
            ),
            spacing="4",
            align="center",
        ),
        padding="32px",
        bg="#F7FAFC",
        border_radius="xl",
        width="100%",
        border="1px solid #E2E8F0",
    ),
),
value="courier",
padding="24px",
),
```

Reflex Dashboard Comprehensive Manual

```
# TAB 3: ANALYZE
rx.tabs.content(
  rx.vstack(
    rx.heading("Ready to Analyze?", size="6", color="#1A202C"),
    rx.box(
      rx.vstack(
        rx.hstack(
          rx.text("Sales Data Status:", weight="bold", color="#1A202C"),
          rx.cond(
            State.sales_data_uploaded,
            rx.badge("Ready", color_scheme="green", variant="solid"),
            rx.badge("Missing", color_scheme="red", variant="solid"),
          ),
          justify="between",
          width="100%",
        ),
        rx.hstack(
          rx.text("Courier Data Status:", weight="bold", color="#1A202C"),
          rx.cond(
            State.courier_data_uploaded,
            rx.badge("Ready", color_scheme="green", variant="solid"),
            rx.badge("Optional", color_scheme="gray", variant="solid"),
          ),
          justify="between",
          width="100%",
        ),
      ),
      width="100%",
      spacing="4",
    ),
    padding="24px",
    bg="#F7FAFC",
    border_radius="lg",
    width="100%",
    border="1px solid #E2E8F0",
  ),
  rx.button(
    "Launch Dashboard ?",
    on_click=State.start_analysis,
    size="4",
    width="100%",
    # disabled=~State.sales_data_uploaded, # Removed disabling to ensure visibility
    color_scheme="purple",
    variant="solid",
    opacity=rx.cond(State.sales_data_uploaded, "1", "0.5"), # Visual cue instead via opacity but
    cursor=rx.cond(State.sales_data_uploaded, "pointer", "not-allowed"),
  ),
  spacing="6",
  align="center",
  width="100%",
),
value="analyze",
padding="24px",
),

defaultValue="sales",
width="100%",
),

# Error message
rx.cond(
```

Reflex Dashboard Comprehensive Manual

```
        State.deployment_status != "",
        rx.center( # Wrap in center
            rx.callout.root(
                rx.callout.text(State.deployment_status),
                color_scheme="red",
                role="alert",
                margin_top="16px",
            ),
            width="100%", # Center needs width
        ),
        align="center",
        width="100%",
        max_width="1000px", # Increased from 600px to accommodate large tabs
    ),
    width="100%",
    height="100vh",
    background_color="white", # Explicit white background, so we force dark text everywhere above
)
```

File: reflex_dashboard/components/filters.py

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state import State
from ..colors import ACCENT_COLOR, BORDER_COLOR

def date_picker_modal() -> rx.Component:
    return rx.dialog.root(
        rx.dialog.content(
            rx.flex(
                # Left Panel (Header Display - Teal)
                rx.center(
                    rx.vstack(
                        rx.text(State.picker_display_weekday, color="white", font_size="sm", opacity=0.8,
text_align="center", width="100%"),
                        rx.text(State.picker_display_day, color="white", font_size="6xl", font_weight="bold",
line_height="1", text_align="center", width="100%"),
                        rx.text(State.picker_display_month, color="white", font_size="2xl", font_weight="bold",
text_align="center", width="100%"),
                        rx.text(State.picker_display_year, color="white", font_size="2xl", opacity=0.7, text_align="center",
width="100%"),

                        spacing="1",
                        align="center",
                        justify="center",
                        height="100%",
                        width="100%"
                    ),
                    width="160px",
                    bg="#009688", # Teal
                    padding="4",
                    height="100%",
                ),

                # Right Panel (Calendar - White)
                rx.vstack(
                    # Header Month Year
```

Reflex Dashboard Comprehensive Manual

```
rx.hstack(
  rx.icon("chevron-left", on_click=State.picker_prev_month, cursor="pointer", color="black", size=18),
  rx.text(State.picker_month_year_title, color="black", font_size="sm", font_weight="bold"),
  rx.icon("chevron-right", on_click=State.picker_next_month, cursor="pointer", color="black", size=18),
  width="100%",
  padding_x="4",
  padding_top="4",
  align="center",
  justify="between"
),

# Weekday Headers
rx.grid(
  *[rx.center(rx.text(day, color="#009688", font_size="xs", font_weight="bold")) for day in ["S", "M",
"T", "W", "T", "F", "S"]],
  columns="7",
  width="100%",
  padding_x="4",
  margin_top="2"
),

# Days Grid
rx.grid(
  rx.foreach(
    State.picker_calendar_grid,
    lambda item: rx.center(
      rx.text(
        item["day"],
        font_size="sm",
        color=rx.cond(item["is_selected"], "white", rx.cond(item["is_current_month"], "black",
"gray.300")),
        font_weight=rx.cond(item["is_selected"], "bold", "normal")
      ),
      bg=rx.cond(item["is_selected"], "#009688", "transparent"), # Teal selected
      border_radius="full",
      width="30px",
      height="30px",
      cursor="pointer",
      _hover={"bg": rx.cond(item["is_selected"], "#009688", "gray.100")},
      on_click=lambda: State.picker_select_date(item["date_str"])
    )
  ),
  columns="7",
  width="100%",
  padding_x="4",
  row_gap="1"
),

rx.spacer(),

# Footer
rx.hstack(
  rx.text("Clear", color="#E53E3E", font_size="xs", font_weight="bold", cursor="pointer"),
  rx.spacer(),
  rx.text("CANCEL", on_click=State.close_picker, color="#009688", font_size="xs", font_weight="bold",
cursor="pointer"),
  rx.text("OK", on_click=State.picker_confirm, color="#009688", font_size="xs", font_weight="bold",
cursor="pointer"),
  width="100%",
  justify="end",
  padding="4",
```

Reflex Dashboard Comprehensive Manual

```
        spacing="4",
        align="center"
    ),

    bg="white", # White background for calendar
    width="280px",
    height="100%"
),

    height="350px",
    width="fit-content",
    overflow="hidden",
    border_radius="4px", # Slightly rounded
),
bg="transparent",
padding="0",
overflow="hidden",
max_width="none",
box_shadow="xl"
),
open=State.show_picker,
)
```

```
def filter_popover(label: str, icon: str, items: list, selected_items: list, toggle_all_fn, toggle_item_fn, color_scheme: str = "gray") -> rx.Component:
    """Helper to create a filter popover."""
    return rx.popover.root(
        rx.popover.trigger(
            rx.button(
                rx.hstack(
                    rx.icon(icon, size=18), # Slightly larger icon
                    rx.text(label, font_size="15px", font_weight="bold"),
                    rx.icon("chevron-down", size=16, opacity=0.7),
                    spacing="2",
                    align="center"
                ),
                variant="surface", # Surface variant for pop
                color_scheme=color_scheme, # Use specific color
                size="3", # Larger button
                high_contrast=True,
                cursor="pointer",
                _hover={"opacity": 0.9, "transform": "scale(1.02)"},
                transition="all 0.2s"
            )
        ),
        rx.popover.content(
            rx.vstack(
                rx.checkbox(rx.text("Select All", color="white"), on_change=toggle_all_fn, color_scheme="gray", size="1"),
                rx.scroll_area(
                    rx.vstack(
                        rx.foreach(
                            items,
                            lambda item: rx.checkbox(
                                rx.text(item, color="white"),
                                checked=selected_items.contains(item),
                                on_change=lambda checked: toggle_item_fn(item, checked),
                                color_scheme="gray",
                                size="1"
                            )
                        )
                    ),
                    spacing="2",
```


Reflex Dashboard Comprehensive Manual

```
        align_items="start"
    ),
    type="always",
    scrollbars="vertical",
    style={"height": 150},
),
spacing="2",
width="100%"
),
bg="#1A202C",
border=f"1px solid {BORDER_COLOR}",
width="200px"
),
)

def horizontal_filter_bar() -> rx.Component:
    """The modern horizontal filter bar."""
    return rx.box(
        rx.flex(
            rx.spacer(), # Spacer on Left

            # 1. Date Range
            rx.hstack(
                rx.hstack(
                    rx.icon("calendar-days", size=16, color=ACCENT_COLOR),
                    rx.text("PERIOD:", font_weight="900", color="white", font_size="xs"),
                    spacing="2",
                    align="center",
                    margin_right="2"
                ),

                # From
                rx.box(
                    rx.hstack(
                        rx.text(rx.cond(State.start_date, State.start_date, "From"), color="white", font_size="sm",
font_weight="medium"),
                        rx.icon("chevron-down", size=12, color="gray.500"),
                        spacing="2", align="center"
                    ),
                    padding="2",
                    border=f"1px solid {BORDER_COLOR}",
                    border_radius="md",
                    cursor="pointer",
                    on_click=lambda: State.open_picker("start"),
                    _hover={"bg": "rgba(255,255,255,0.05)"}
                ),
                rx.text("-", color="gray.500"),
                # To
                rx.box(
                    rx.hstack(
                        rx.text(rx.cond(State.end_date, State.end_date, "To"), color="white", font_size="sm",
font_weight="medium"),
                        rx.icon("chevron-down", size=12, color="gray.500"),
                        spacing="2", align="center"
                    ),
                    padding="2",
                    border=f"1px solid {BORDER_COLOR}",
                    border_radius="md",
                    cursor="pointer",
                    on_click=lambda: State.open_picker("end"),
                    _hover={"bg": "rgba(255,255,255,0.05)"}
                )
            )
        )
    )
```

Reflex Dashboard Comprehensive Manual

```
        ),
        spacing="3",
        align="center",
        border_right=f"1px solid {BORDER_COLOR}",
        padding_right="6",
        margin_right="4"
    ),

    # 2. Filters Group
    rx.hstack(
        filter_popover("Region", "map", State.states, State.selected_states, State.toggle_all_states,
        State.toggle_state, "blue"),
        filter_popover("Brand", "tag", State.brands, State.selected_brands, State.toggle_all_brands,
        State.toggle_brand, "purple"),
        filter_popover("Channel", "share-2", State.channels, State.selected_channels, State.toggle_all_channels,
        State.toggle_channel, "pink"),
        filter_popover("Supply", "truck", State.supply_types, State.selected_supply_types,
        State.toggle_all_supply, State.toggle_supply, "orange"),
        spacing="4"
    ),

    rx.spacer(), # Spacer on Right

    # 3. Clear Filters Action (Moved to Right - BIG)
    rx.button(
        "Reset",
        icon="rotate-ccw",
        variant="solid",
        color_scheme="red",
        size="4", # Maximum standard size
        padding_x="8", # Extra padding
        padding_y="6",
        font_size="xl", # Larger font
        margin_left="4",
        on_click=State.reset_filters
    ),

    width="100%",
    max_width="1900px",
    margin_x="auto",
    align="center",
    justify="center",
    padding_x="6",
    padding_y="4"
),

width="100%",
bg="#111111",
border_bottom=f"1px solid {BORDER_COLOR}"
)
```

File: `reflex_dashboard/components/intro.py`

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state import State
from .bistro import bistro_component
```

Reflex Dashboard Comprehensive Manual

```
def intro_tab_content() -> rx.Component:
    return rx.vstack(
        rx.vstack(
            rx.heading("Welcome to Enterprise Intelligence", size="9", color="white", font_weight="900", text_align="center",
letter_spacing="0.05em"),
            rx.text("Where raw data meets actionable insights (and some humor!)", color="gray.400", size="5"),

            rx.hstack(
                rx.image(src="/ai_cartoon.jpg", height="180px", border_radius="xl", border="2px solid #333"),
                rx.image(src="/ml_comic.png", height="180px", border_radius="xl", border="2px solid #333"),
                spacing="4",
                margin_y="4",
                wrap="wrap",
                justify="center"
            ),
        ),

        rx.vstack(
            rx.text('"Without data, you\'re just another person with an opinion." ? W. Edwards Deming', color="#F56565",
font_weight="bold", font_size="sm", text_align="center", font_family="Lettwestich"),
            rx.text('"We trust in God. All others must bring data." ? W. Edwards Deming', color="#63B3ED",
font_weight="bold", font_size="sm", text_align="center", font_family="Europa grotesk sh"),
            rx.text('"Better decisions begin with better questions."', color="#F6E05E", font_weight="bold",
font_size="sm", text_align="center", font_family="Letterstich plain"),
            rx.text('"Gut feelings are strong, still stronger are data."', color="#68D391", font_weight="bold",
font_size="sm", text_align="center", font_family="FF ifentification fivec"),
            spacing="2",
            align="center",
            bg="rgba(0,0,0,0.4)",
            padding="4",
            border_radius="xl",
            width="100%",
            max_width="800px",
            border="1px solid rgba(255,255,255,0.2)",
            backdrop_filter="blur(10px)"
        ),

        bistro_component(),

        spacing="2",
        align="center"
    ),
    height="100vh",
    width="100%",
    spacing="2",
    padding_x="4",
    padding_y="4",
    align="center",
    justify="start",
    overflow="hidden", # Prevent scrolling
)
```

File: [reflex_dashboard/components/kpi.py](#)

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state import State
from ..colors import CARD_BG, BORDER_COLOR, TEXT_COLOR
```

Reflex Dashboard Comprehensive Manual

```
from .common import trend_badge

def kpi_card(title: str, value: str, trend: str, icon: str, color_scheme: str, bg_color: str = CARD_BG, align: str = "start",
value_size: str = "6", title_size: str = "2", bottom_left: str = None, bottom_right: str = None, bottom_left_trend: str =
None, bottom_right_trend: str = None, bottom_left_icon: str = None, bottom_right_icon: str = None, trend_inline: bool = False,
subtitle: str = None):
    """
    A modern KPI card with:
    - Icon on the left (or top-right)
    - Value prominent
    - Title subtle
    - Subtitle option
    - Trend indicator
    - Optional bottom corners
    - Optional inline trend
    """
    # Color mapping for trends
    trend_color = rx.cond(
        (trend.contains("-") | (trend.contains("?"))),
        "#D0312D", # Pitch Red
        "#68D391" # Lighter green for dark mode
    )

    return rx.box(
        rx.vstack(
            rx.hstack(
                rx.vstack(
                    rx.heading(title, size=title_size, font_weight="900", color="white", letter_spacing="0.05em",
text_transform="uppercase"), # Uppercase title
                    rx.cond(
                        subtitle is not None,
                        rx.text(subtitle, font_size="xs", color="white", font_weight="bold"),
                        rx.fragment()
                    ),
                ),
                rx.heading(
                    value,
                    rx.cond(
                        trend_inline,
                        rx.box(trend_badge(trend, font_size="0.5em"), display="inline-block", style={"verticalAlign":
"middle", "marginTop": "-10px", "marginLeft": "8px"}),
                        rx.fragment()
                    ),
                    size=value_size,
                    font_weight="900",
                    color="white",
                    letter_spacing="-0.02em" # Tight spacing for numbers
                ),
                align_items=align,
                spacing="1",
                width="100%",
            ),
            rx.cond(
                align == "start",
                rx.box(
                    rx.spacer(),
                    rx.icon(icon, size=28, color=color_scheme, stroke_width=1.5), # Thinner stroke for elegance
                ),
                rx.fragment()
            ),
        ),
        width="100%",
        align_items="center" if align == "center" else "start",
```

Reflex Dashboard Comprehensive Manual

```

        justify_content="center" if align == "center" else "start",
    ),

    rx.cond(
        not trend_inline,
        rx.hstack(
            rx.text(trend, color=trend_color, font_size="sm", font_weight="bold", text_align=align, width="100%"),
            width="100%",
            justify_content=align,
        ),
        rx.fragment()
    ),

    # Bottom Corner Content (for Total Sales)
    rx.cond(
        bottom_left is not None,
        rx.flex(
            rx.hstack(
                rx.icon(bottom_left_icon, size=16, color="gray.400") if bottom_left_icon is not None else
rx.fragment(),

                rx.text(bottom_left, font_size="sm", color="white", font_weight="bold"),
                trend_badge(bottom_left_trend, font_size="0.75em") if bottom_left_trend is not None else
rx.fragment(),

                align="center",
                spacing="2"
            ),
            rx.spacer(),
            rx.hstack(
                rx.icon(bottom_right_icon, size=16, color="gray.400") if bottom_right_icon is not None else
rx.fragment(),

                rx.text(bottom_right, font_size="sm", color="white", font_weight="bold"),
                trend_badge(bottom_right_trend, font_size="0.75em") if bottom_right_trend is not None else
rx.fragment(),

                align="center",
                spacing="2"
            ),
            width="100%",
            justify="between",
            padding_top="4",
            border_top=f"1px solid {BORDER_COLOR}",
            margin_top="2"
        ),
        rx.fragment()
    ),

    spacing="3",
    align_items=align,
),
padding="6",
bg=bg_color,
border=f"1px solid {BORDER_COLOR}",
border_radius="xl",
width="100%",
box_shadow="0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06)",
transition="all 0.2s",
_hover={
    "transform": "translateY(-2px)",
    "box_shadow": "0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -2px rgba(0, 0, 0, 0.05)",
    "border_color": color_scheme
},
)

```

Reflex Dashboard Comprehensive Manual

```
def card_avg_monthly_sale():
    return rx.box(
        rx.vstack(
            rx.hstack(
                rx.text("AVG. MONTHLY SALE", color="white", font_weight="bold", font_size="xs", letter_spacing="0.1em"),
                rx.icon("chart-bar", color="#FF9966", size=20),
                width="100%",
                align="center",
                justify="center",
                spacing="2"
            ),
            rx.hstack(
                rx.heading(State.average_monthly_sale, color="white", size="8", font_weight="900", letter_spacing="-0.02em"),
                rx.badge("? 14.9%", color_scheme="green", variant="surface", size="1"),
                align="baseline",
                spacing="3",
                justify="center",
                width="100%"
            ),
            rx.spacer(),
            rx.hstack(
                rx.hstack(
                    rx.text("B2B", color="#FF9966", font_size="xs", font_weight="900"),
                    rx.text(State.avg_monthly_b2b, color="white", font_size="xs", font_weight="bold"),
                    align="center",
                    spacing="1"
                ),
                rx.spacer(),
                rx.hstack(
                    rx.text("B2C", color="#63b3ed", font_size="xs", font_weight="900"),
                    rx.text(State.avg_monthly_b2c, color="white", font_size="xs", font_weight="bold"),
                    align="center",
                    spacing="1"
                ),
                width="100%",
                align="center",
                padding_top="4",
                border_top=f"1px solid {BORDER_COLOR}"
            ),
            height="100%",
            justify="between",
            align_items="center",
            spacing="1"
        ),
        bg=CARD_BG,
        border=f"1px solid {BORDER_COLOR}",
        border_left="4px solid #FF9966", # Orange Accent
        border_radius="xl",
        padding="24px",
        width="100%",
        height="180px",
        box_shadow="lg"
    )

def card_daily_velocity():
    return rx.box(
        rx.vstack(
            rx.hstack(
                rx.text("DAILY SALES VELOCITY", color="white", font_weight="bold", font_size="xs", letter_spacing="0.1em"),
                rx.icon("zap", color="#63b3ed", size=20),
```

Reflex Dashboard Comprehensive Manual

```
        width="100%",
        align="center",
        justify="center",
        spacing="2"
    ),
    rx.heading(State.daily_sales_velocity_data["total"], color="white", size="8", font_weight="900",
letter_spacing="-0.02em"),
    rx.spacer(),
    rx.hstack(
        rx.hstack(
            rx.text("B2B", color="#FF9966", font_size="xs", font_weight="900"),
            rx.text(State.daily_sales_velocity_data["b2b"], color="white", font_size="xs", font_weight="bold"),
            align="center",
            spacing="1"
        ),
        rx.spacer(),
        rx.hstack(
            rx.text("B2C", color="#63b3ed", font_size="xs", font_weight="900"),
            rx.text(State.daily_sales_velocity_data["b2c"], color="white", font_size="xs", font_weight="bold"),
            align="center",
            spacing="1"
        ),
        width="100%",
        align="center",
        padding_top="4",
        border_top=f"1px solid {BORDER_COLOR}"
    ),
    height="100%",
    justify="between",
    align_items="center",
    spacing="1"
),
bg=CARD_BG,
border=f"1px solid {BORDER_COLOR}",
border_left="4px solid #63b3ed", # Blue Accent
border_radius="xl",
padding="24px",
width="100%",
height="180px",
box_shadow="lg"
)

def card_returns_invoices():
    return rx.box(
        rx.vstack(
            rx.hstack(
                # Left: Returns (CENTERED)
                rx.box(
                    rx.vstack(
                        rx.hstack(rx.icon("rotate-ccw", color="red", size=14), rx.text("RETURNS", color="white",
font_weight="bold", font_size="xs", letter_spacing="0.05em"), spacing="2", align="center"),
                        rx.heading(State.return_metrics["count"], color="white", size="6", font_weight="900"),
                        rx.hstack(
                            rx.text(State.return_metrics["rate_pct"], color="red", font_size="xs", font_weight="bold"),
                            rx.text(State.return_metrics["val_cr"], color="white", font_size="xs"),
                            spacing="2"
                        ),
                    ),
                    align_items="center", # Center align internal content
                    spacing="1",
                    width="100%"
                ),
            ),
        ),
    ),
```

Reflex Dashboard Comprehensive Manual

```
        flex="1",
        display="flex",
        justify_content="center",
        width="100%"
    ),
    # Right: Invoices (RIGHT ALIGNED BLOCK, EXTREME RIGHT)
    rx.box(
        rx.vstack(
            rx.hstack(
                rx.icon("file-text", color="white", size=14), rx.text("INVOICES", color="white",
font_weight="bold", font_size="xs", letter_spacing="0.05em"), spacing="2", align="center"),
            rx.heading(State.invoice_stats["count"], color="white", size="6", font_weight="900"),
            rx.hstack(
                rx.text("Avg", color="white", font_size="xs"),
                rx.text(State.invoice_stats["daily_avg"], color="white", font_size="xs", font_weight="bold"),
                spacing="2"
            ),
            align_items="end", # Right align internal content
            spacing="1",
            width="100%"
        ),
        flex="1",
        display="flex",
        justify_content="end", # Push to right
        width="100%"
    ),
    width="100%",
    align_items="center",
    justify="between" # Ensure they are spread out
),
# Removed B2C RETURN RATE section as requested
spacing="3",
width="100%",
height="100%", # Fill height
justify="center", # Center vertically
),
bg=CARD_BG,
border=f"1px solid {BORDER_COLOR}",
border_left="4px solid #e53935", # Red Accent
border_radius="xl",
padding="24px", # Explicit padding
width="100%",
height="180px", # Consistent height
box_shadow="lg"
)

def card_pareto():
    return rx.box(
        rx.vstack(
            rx.center(
                rx.vstack(
                    rx.text("PARETO (80% SALE)", color="white", font_weight="bold", font_size="xs",
letter_spacing="0.1em"),
                    rx.heading(
                        State.pareto_stats["count_80"],
                        color="white",
                        size="8",
                        font_weight="900",
                        letter_spacing="-0.02em"
                    ),
                    spacing="1",
                    align_items="center",
```


Reflex Dashboard Comprehensive Manual

```
        width="100%"
    ),
    width="100%"
),
rx.box(
    rx.vstack(
        rx.foreach(
            State.pareto_top_products,
            lambda item: rx.hstack(
                rx.badge(item["rank"], variant="solid", color_scheme="yellow", border_radius="full",
size="1"),

                rx.text(item["name"], color="white", font_size="xs", no_of_lines=1, width="40%"),
                rx.spacer(),
                rx.text(item["value_cr"], color="white", font_size="xs", font_weight="bold"),
                rx.spacer(),
                rx.text(item["pct"], color="white", font_size="xs", font_weight="bold"),
                width="100%",
                padding_y="1",
                border_bottom=f"1px solid {BORDER_COLOR}"
            )
        ),
        width="100%",
        spacing="1"
    ),
    width="100%",
    bg="rgba(255,255,255,0.02)",
    border_radius="md",
    padding="4"
),
rx.box( # Wrapper for spacing
    rx.text(
        rx.text.span("Products driving 80% of sales ", font_weight="bold", color="white"),
        rx.text.span(State.pareto_stats["pct_catalog"], font_weight="bold", color="white"),
        font_size="xs",
        text_align="center"
    ),
    padding_top="4", # Added padding to push it down
    width="100%",
    display="flex",
    justify_content="center"
),
    spacing="1", # Reduced spacing since we use manual padding
    align_items="center",
    width="100%"
),
bg=CARD_BG,
border=f"1px solid {BORDER_COLOR}",
border_left=f"4px solid {rx.color('yellow', 9)}",
# Gradient background override
background=f"linear-gradient(135deg, {CARD_BG} 0%, rgba(80, 40, 100, 0.3) 100%)",
border_radius="xl",
padding="24px",
width="100%",
box_shadow="lg"
)
```

File: `reflex_dashboard/components/layouts.py`

UI Component Module: specific aspect of the dashboard interface.

Reflex Dashboard Comprehensive Manual

```
import reflex as rx
from ..state import State
from ..colors import CARD_BG, TEXT_COLOR, CONTENT_BG, BORDER_COLOR
from .common import placeholder_tab
from .empty_state import no_data_view
from .filters import date_picker_modal, horizontal_filter_bar
from .ai import ai_chat_component, ai_chef_placeholder
from .overview import overview_tab_content
from .customer import customer_tab_content
from .intro import intro_tab_content
from .predictive import predictive_tab_content

def index() -> rx.Component:
    return rx.box(
        rx.cond(
            # CONDITIONAL: Show No Data View if dashboard is not started
            ~State.show_dashboard,
            no_data_view(),
            # ELSE: Dashboard is active
            rx.cond(
                State.show_intro,
                rx.box(
                    intro_tab_content(),
                    height="100vh",
                    width="100%",
                    bg="black"
                ),
                # ELSE: Show New Enterprise Layout
                rx.box(
                    date_picker_modal(),
                    # 1. Main Header
                    rx.box(
                        rx.center( # Force centering using rx.center wrapper
                            rx.vstack(
                                rx.hstack(
                                    rx.icon("database", size=20, color="white"),
                                    rx.heading("ENTERPRISE INTELLIGENCE SUITE", size="4", color="white", font_weight="900",
letter_spacing="0.05em"),
                                    align="center",
                                    spacing="3"
                                ),
                                rx.badge("FULL-SPECTRUM STRATEGIC DASHBOARD V10.0", color_scheme="gray", variant="soft",
size="1"),
                                align="center",
                                spacing="1",
                            ),
                            width="100%"
                        ),
                        rx.box(
                            rx.hstack(
                                rx.button("Code", variant="outline", size="1", color_scheme="gray"),
                                rx.button("Preview", variant="solid", size="1", color_scheme="blue"),
                                rx.button("Share", variant="solid", size="1", color_scheme="blue"),
                                spacing="2"
                            ),
                            position="absolute",
                            right="4",
                            top="50%",
                            transform="translateY(-50%)",
                        ),
                        width="100%",
```

Reflex Dashboard Comprehensive Manual

```
padding="4",
bg="#000000",
border_bottom=f"1px solid {BORDER_COLOR}",
position="relative",
),

# 2. Tabs Navigation (Keyboard Key Style - Centered - 3 Per Row)
rx.tabs.root(
    rx.tabs.list(
        # Single Row - 6 Tabs
        rx.tabs.trigger(
            "BUSINESS SUMMARY",
            value="overview",
            width="100%", height="60px",
            bg="rgba(56, 161, 105, 0.15)", # Green tint
            border="1px solid #38A169",
            color="#38A169",
            _active={"bg": "#38A169", "color": "white", "transform": "translateY(2px)", "box_shadow": "none"},

            style={"border_radius": "8px", "box_shadow": "0px 4px 0px #22543d", "transition": "all 0.1s",
"font_weight": "900", "font_size": "15px"},
            cursor="pointer"
        ),
        rx.tabs.trigger(
            "PREDICTIVE ENGINE",
            value="predictive",
            width="100%", height="60px",
            bg="rgba(128, 90, 213, 0.15)", # Purple tint
            border="1px solid #805AD5",
            color="#805AD5",
            _active={"bg": "#805AD5", "color": "white", "transform": "translateY(2px)", "box_shadow": "none"},

            style={"border_radius": "8px", "box_shadow": "0px 4px 0px #44337a", "transition": "all 0.1s",
"font_weight": "900", "font_size": "15px"},
            cursor="pointer"
        ),
        rx.tabs.trigger(
            "STRATEGY LAB",
            value="strategy",
            width="100%", height="60px",
            bg="rgba(49, 130, 206, 0.15)", # Blue tint
            border="1px solid #3182CE",
            color="#3182CE",
            _active={"bg": "#3182CE", "color": "white", "transform": "translateY(2px)", "box_shadow": "none"},

            style={"border_radius": "8px", "box_shadow": "0px 4px 0px #2a4365", "transition": "all 0.1s",
"font_weight": "900", "font_size": "15px"},
            cursor="pointer"
        ),
        rx.tabs.trigger(
            "FINANCIAL AUDIT",
            value="finance",
            width="100%", height="60px",
            bg="rgba(213, 63, 140, 0.15)", # Magenta tint
            border="1px solid #D53F8C",
            color="#D53F8C",
            _active={"bg": "#D53F8C", "color": "white", "transform": "translateY(2px)", "box_shadow": "none"},

            style={"border_radius": "8px", "box_shadow": "0px 4px 0px #702459", "transition": "all 0.1s",
"font_weight": "900", "font_size": "15px"},
            cursor="pointer"
```

Reflex Dashboard Comprehensive Manual

```
),
rx.tabs.trigger(
  "OPERATIONS LAB",
  value="operations",
  width="100%", height="60px",
  bg="rgba(246, 135, 179, 0.15)", # Pinkish tint
  border="1px solid #F687B3",
  color="#F687B3",
  _active={"bg": "#F687B3", "color": "white", "transform": "translateY(2px)", "box_shadow": "none"},

  style={"border_radius": "8px", "box_shadow": "0px 4px 0px #97266d", "transition": "all 0.1s",
"font_weight": "900", "font_size": "15px"},
  cursor="pointer"
),
rx.tabs.trigger(
  "CUSTOMER SCIENCE",
  value="customer",
  width="100%", height="60px",
  bg="rgba(214, 158, 46, 0.15)", # Yellowish tint
  border="1px solid #D69E2E",
  color="#D69E2E",
  _active={"bg": "#D69E2E", "color": "white", "transform": "translateY(2px)", "box_shadow": "none"},

  style={"border_radius": "8px", "box_shadow": "0px 4px 0px #744210", "transition": "all 0.1s",
"font_weight": "900", "font_size": "15px"},
  cursor="pointer"
),

# Apply Grid Styles directly to the list container
display="grid",
grid_template_columns="repeat(6, 1fr)",
gap="16px",
padding_y="24px",
bg="transparent",
width="100%",
max_width="1600px",
margin_x="auto"
),

# 3. Filter Bar (Sticky)
rx.box(
  horizontal_filter_bar(),
  position="sticky",
  top="0",
  z_index="50",
  width="100%"
),

# 4. AI Assistant
rx.box(
  ai_chat_component(),
  padding="4",
  bg="#111111",
  border_bottom=f"1px solid {BORDER_COLOR}"
),

# 5. Tab Content Areas
rx.box(
  # Empty State Hint
  rx.cond(State.current_tab == "", ai_chef_placeholder()),
```

Reflex Dashboard Comprehensive Manual

```
rx.tabs.content(  
    overview_tab_content(),  
    value="overview",  
    padding="6",  
),  
rx.tabs.content(predictive_tab_content(), value="predictive"),  
rx.tabs.content(placeholder_tab("Strategy Lab"), value="strategy"),  
rx.tabs.content(placeholder_tab("Financial Audit"), value="finance"),  
rx.tabs.content(placeholder_tab("Operations Lab"), value="operations"),  
rx.tabs.content(  
    customer_tab_content(),  
    value="customer",  
    padding="6"  
),  
  
bg=CONTENT_BG,  
min_height="calc(100vh - 150px)", # Adjust for headers  
width="100%",  
max_width="1900px", # Limit width for readability  
margin_x="auto"  
),  
  
    value=State.current_tab,  
    on_change=State.set_current_tab,  
    width="100%",  
),  
),  
),  
),  
width="100%",  
min_height="100vh",  
on_mount=State.load_data,  
bg=CONTENT_BG  
)
```

File: reflex_dashboard/components/logistics.py

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx  
from ..state import State  
from ..colors import CARD_BG, TEXT_COLOR, BORDER_COLOR  
  
def courier_performance_card():  
    return rx.box(  
        rx.vstack(  
            rx.center(  
                rx.hstack(  
                    rx.center(  
                        rx.icon("truck", color="#3182CE", size=20),  
                        bg="rgba(49, 130, 206, 0.1)", # Dark Blue tint  
                        padding="2",  
                        border_radius="md",  
                        border="1px solid #3182CE"  
                    ),  
                    rx.text("Courier Performance(Only for Rajnigandha.com)", font_weight="900", color=TEXT_COLOR,
```

Reflex Dashboard Comprehensive Manual

```
font_size="md", letter_spacing="0.05em"),
    align="center",
    spacing="3",
  ),
  width="100%",
  margin_bottom="4"
),
rx.grid(
  # Card 1: Avg Delivery Cost
  rx.box(
    rx.vstack(
      rx.hstack(
        rx.icon("indian-rupee", size=14, color="white"), # Changed to white
        rx.text("AVG DELIVERY COST", font_size="xs", font_weight="bold", color="white",
letter_spacing="0.1em"), # Changed to white
        spacing="2",
        align="center"
      ),
      rx.heading(State.courier_metrics["avg_cost"], size="6", color="white", font_weight="900"),
      rx.text("per shipment", font_size="xs", color="white", font_weight="bold"), # Max visibility
      spacing="1",
      align_items="center"
    ),
    bg="rgba(255,255,255,0.02)",
    padding="4",
    border_radius="lg",
    border=f"1px solid {BORDER_COLOR}",
    width="100%"
  ),
  # Card 2: Avg Delivery Time
  rx.box(
    rx.vstack(
      rx.hstack(
        rx.icon("clock", size=14, color="white"),
        rx.text("AVERAGE DELIVERY TIME", font_size="xs", font_weight="bold", color="white",
letter_spacing="0.1em"),
        spacing="2",
        align="center"
      ),
      rx.heading(State.courier_metrics["avg_time"], size="6", color="white", font_weight="900"),
      rx.text("pickup to delivered", font_size="xs", color="white", font_weight="bold"), # Max visibility
      spacing="1",
      align_items="center"
    ),
    bg="rgba(255,255,255,0.02)",
    padding="4",
    border_radius="lg",
    border=f"1px solid {BORDER_COLOR}",
    width="100%"
  ),
  # Card 3: Successful Delivery Rate (Green)
  rx.box(
    rx.vstack(
      rx.hstack(
        rx.icon("circle_check", size=14, color="#48BB78"),
        rx.text("SUCCESSFUL DELIVERY RATE", font_size="xs", font_weight="bold", color="#48BB78",
letter_spacing="0.1em"),
        spacing="2",
        align="center"
      ),
      rx.heading(State.courier_metrics["success_rate"], size="6", color="white", font_weight="900"),
```

Reflex Dashboard Comprehensive Manual

```
        rx.text("delivery rate", font_size="xs", color="white", font_weight="bold"), # Max visibility
        spacing="1",
        align_items="center"
    ),
    bg="rgba(72, 187, 120, 0.05)", # Dark Green tint
    padding="4",
    border_radius="lg",
    border="1px solid #2F855A",
    width="100%"
),
# Card 4: Return Rate (Red)
rx.box(
    rx.vstack(
        rx.hstack(
            rx.icon("rotate_cw", size=14, color="#F56565"),
            rx.text("RETURN RATE", font_size="xs", font_weight="bold", color="#F56565",
letter_spacing="0.1em"),
            spacing="2",
            align="center"
        ),
        rx.heading(State.courier_metrics["return_rate"], size="6", color="white", font_weight="900"),
        rx.text("of total orders", font_size="xs", color="white", font_weight="bold"), # Max visibility
        spacing="1",
        align_items="center"
    ),
    bg="rgba(245, 101, 101, 0.05)", # Dark Red tint
    padding="4",
    border_radius="lg",
    border="1px solid #9B2C2C",
    width="100%"
),
columns={"initial": "1", "sm": "1", "lg": "2"},
spacing="4",
width="100%"
),
width="100%"
),
bg=CARD_BG,
padding="24px",
border_radius="xl",
box_shadow="lg",
width="100%",
border=f"1px solid {BORDER_COLOR}"
)
```

File: `reflex_dashboard/components/overview.py`

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state import State
from ..colors import CARD_BG, TEXT_COLOR, BORDER_COLOR
from .common import table_container
from .kpi import kpi_card, card_avg_monthly_sale, card_daily_velocity, card_returns_invoices, card_pareto
from .logistics import courier_performance_card
from .charts import channel_sales_card
from .tables import monthly_summary_table, monthly_sales_return_table, invoice_vs_return_table

def overview_tab_content() -> rx.Component:
```

Reflex Dashboard Comprehensive Manual

```
""The main dashboard content.""
return rx.vstack(
  # KPI Section
  rx.vstack(
    kpi_card(
      title="TOTAL SALE",
      subtitle="(Apr'25 - Nov'25)",
      value=State.total_sales,
      trend=State.total_sales_trend,
      icon="dollar-sign",
      color_scheme="#48BB78",
      align="center",
      value_size="9",
      title_size="8",
      bottom_left=State.total_b2b_sales,
      bottom_right=State.total_b2c_sales,
      bottom_left_trend=State.b2b_trend,
      bottom_right_trend=State.b2c_trend,
      bottom_left_icon="briefcase",
      bottom_right_icon="shopping-cart",
      trend_inline=True,
    ),
    rx.grid(
      card_avg_monthly_sale(),
      card_daily_velocity(),
      card_returns_invoices(),
      card_pareto(),
      columns={"initial": "1", "sm": "1", "lg": "2", "xl": "2"}, # 2x2 Layout
      spacing="4",
      width="100%"
    ),
  ),

  # Courier Performance Card
  courier_performance_card(),

  spacing="4",
  width="100%",
),

rx.box(height="10px"),

# Monthly Summary Table
rx.box(
  monthly_summary_table(),
  width="100%"
),

rx.box(height="20px"),

# Monthly Sales Return Table
rx.box(
  monthly_sales_return_table(),
  width="100%"
),

rx.box(height="20px"),

# Invoice vs Returns Table
rx.box(
  invoice_vs_return_table(),
  width="100%"
)
```


Reflex Dashboard Comprehensive Manual

```
),

rx.separator(margin_y="6", color_scheme="gray"),

# Charts Row 1
rx.grid(
  rx.box(
    rx.vstack(
      rx.heading("Monthly Sales Trend (Value in Crore)", size="4", color=TEXT_COLOR, width="100%",
text_align="center"),
      rx.plotly(data=State.monthly_sales_chart, height="400px"),
      width="100%",
      align="center",
    ),
    bg="#000000",
    box_shadow="lg",
    border="1px solid #4A5568",
    border_radius="xl",
    padding="4"
  ),

  # Added Channel Wise Sale (Donut Chart)
  channel_sales_card(),

  rx.box(
    rx.vstack(
      rx.heading("Sales by State Top -10 Distribution (Value in Crore)", size="4", color=TEXT_COLOR,
width="100%", text_align="center"),
      rx.plotly(data=State.state_sales_chart, height="580px"),
      width="100%",
      align="center",
    ),
    bg="#000000",
    box_shadow="lg",
    border="1px solid #4A5568",
    border_radius="xl",
    padding="4"
  ),
  columns="1",
  spacing="4",
  width="100%",
),

# Charts Row 2
rx.grid(
  rx.box(
    rx.vstack(
      rx.heading("Sales by Product - Top 5 (Value in Crore)", size="4", color=TEXT_COLOR, width="100%",
text_align="center"),
      rx.plotly(data=State.product_sales_chart, height="580px"),
      width="100%",
      align="center",
    ),
    bg="#000000",
    box_shadow="lg",
    border="1px solid #4A5568",
    border_radius="xl",
    padding="4"
  ),
  rx.box(
```

Reflex Dashboard Comprehensive Manual

```
        rx.vstack(  
            rx.heading("Sales by Supply Type (Value in Crore)", size="4", color=TEXT_COLOR, width="100%",  
text_align="center"),  
            rx.plotly(data=State.supply_sales_chart, height="580px"),  
            width="100%",  
            align="center",  
        ),  
        bg="#000000",  
        box_shadow="lg",  
        border="1px solid #4A5568",  
        border_radius="xl",  
        padding="4"  
    ),  
    columns="1",  
    spacing="4",  
    width="100%",  
),  
  
rx.separator(margin_y="6", color_scheme="gray"),  
  
# --- AI & Advanced Analytics Section ---  
  
# Top Products & B2C AOV Grid  
rx.grid(  
    # Top Products Table  
    table_container(  
        "Product Sales Distribution", "Top 10 items & others contribution", "transparent",  
        rx.table.root(  
            rx.table.header(  
                rx.table.row(  
                    rx.table.column_header_cell("Rank", text_align="center", color="white", font_weight="bold"),  
                    rx.table.column_header_cell("Product Name", text_align="center", color="white",  
font_weight="bold"), # Center align  
                    rx.table.column_header_cell("Sales", text_align="center", color="white", font_weight="bold"), #  
shortened for space  
                    rx.table.column_header_cell("Contribution (in %)", text_align="center", color="white",  
font_weight="bold"), # Renamed  
                ),  
                bg="rgba(255,255,255,0.05)"  
            ),  
            rx.table.body(  
                rx.foreach(  
                    State.product_data,  
                    lambda row: rx.table.row(  
                        rx.table.cell(  
                            rx.center(  
                                rx.text(row["rank"], font_weight="bold", color="white", font_size="xs"),  
                                bg=row["rank_bg"],  
                                width="24px",  
                                height="24px",  
                                border_radius="full",  
                                margin_x="auto" # Center alignment fix  
                            ),  
                            text_align="center"  
                        ),  
                        rx.table.cell(rx.text(row["label"], font_size="xs", weight="medium", color="white",  
text_shadow="0px 1px 2px black"), text_align="center"), # Center align  
                        rx.table.cell(rx.text(row["formatted_value"], font_family="mono", color="white",  
font_size="xs", font_weight="bold", text_shadow="0px 1px 2px black"), text_align="center"),  
                        rx.table.cell(  
                            rx.badge(row["formatted_pct"], color_scheme="blue", variant="solid"),
```

Reflex Dashboard Comprehensive Manual

```
                text_align="center"
            ),
            bg=row["row_bg"] # Banded lines
        )
    )
    ),
    # footer removed
),

columns={"initial": "1", "sm": "1", "lg": "1"}, # Changed to single column for top products
spacing="4",
width="100%",
),

rx.box(height="20px"),

# B2C AOV Table in new row
rx.box(
    table_container(
        "B2C Average Order Value", "AOV by Channel (B2C Only)", "transparent",
        rx.table.root(
            rx.table.header(
                rx.table.row(
                    rx.table.column_header_cell("Channel", text_align="center", color="white", font_weight="bold"),
                    rx.table.column_header_cell("Sales", text_align="center", color="white", font_weight="bold"),
                    rx.table.column_header_cell("Orders", text_align="center", color="white", font_weight="bold"),
                    rx.table.column_header_cell("AOV", text_align="center", color="white", font_weight="bold"),
                ),
                bg="rgba(255,255,255,0.05)"
            ),
            rx.table.body(
                rx.foreach(
                    State.b2c_data,
                    lambda row: rx.table.row(
                        rx.table.cell(
                            rx.text(
                                row["channel"],
                                color=row["channel_color"],
                                font_weight="bold",
                                text_shadow="0px 1px 2px black"
                            ),
                            text_align="center"
                        ),
                        rx.table.cell(rx.text(row["formatted_sales"], font_family="mono", font_size="xs",
color="white"), text_align="center"),
                        rx.table.cell(rx.text(row["orders"], font_family="mono", font_size="xs", color="white"),
text_align="center"),
                        rx.table.cell(rx.badge(row["formatted_aov"], color_scheme="green", variant="solid"),
text_align="center"),
                    )
                )
            )
        ),
        width="100%",
    ),
    rx.box(height="20px"),
```

Reflex Dashboard Comprehensive Manual

```
# State Wise Table
rx.box(
    table_container(
        "State-wise Top Selling B2C Product", "Highest revenue product by state", "transparent",
        rx.table.root(
            rx.table.header(
                rx.table.row(
                    rx.table.column_header_cell("State", color="white", font_weight="bold"),
                    rx.table.column_header_cell("Top Product", color="white", font_weight="bold"),
                    rx.table.column_header_cell("Sales", text_align="right", color="white", font_weight="bold"),
                    rx.table.column_header_cell("% contribution to Sales in the State", text_align="center",
color="white", font_weight="bold"),
                    rx.table.column_header_cell("Total Sale in the State", text_align="center", color="white",
font_weight="bold"),
                ),
                bg="rgba(255,255,255,0.05)"
            ),
            rx.table.body(
                rx.foreach(
                    State.state_data,
                    lambda row: rx.table.row(
                        rx.table.cell(rx.text(row["state"], font_weight="bold", font_size="xs", color="white")),
                        rx.table.cell(rx.text(row["product"], font_size="xs", color="white")),
                        rx.table.cell(rx.badge(row["formatted_sales"], color_scheme="blue", variant="solid"),
text_align="right"),
                        rx.table.cell(rx.badge(row["formatted_pct"], color_scheme="green", variant="outline"),
text_align="center"),
                        rx.table.cell(rx.badge(row["formatted_total_sales"], color_scheme="purple", variant="soft"),
text_align="center"),
                    )
                )
            )
        ),
        width="100%",
    ),
    width="100%",
    spacing="4",
    padding_bottom="40px",
)
```

Visual Output: Overview Tab Layout

🛒 DEMAND PLANNING & REPLENISHMENT					
SKU ID	Category	Current Stock	Forecast Demand (M+1)	Reorder Qty	Status
SKU-101	Electronics	50	30	30	Good Stock
SKU-102	Electronics	100	150	150	Critical Low
SKU-205	Home	750	0	0	Overstock
SKU-310	Fashion	100	85	85	Critical Low
SKU-404	Electronics	25	0	0	Healthy
SKU-520	Home	750	50	50	Overstock

File: `reflex_dashboard/components/predictive.py`

UI Component Module: specific aspect of the dashboard interface.

Reflex Dashboard Comprehensive Manual

```
import reflex as rx
from ..state.predictive import PredictiveState
from ..colors import CARD_BG, TEXT_COLOR, BORDER_COLOR

def chart_container(title: str, chart: rx.Component, height: str = "400px") -> rx.Component:
    return rx.box(
        rx.vstack(
            rx.hstack(
                rx.icon("activity", size=20, color="white"), # Generic icon, can be customized
                rx.heading(title.upper(), size="4", color=TEXT_COLOR, font_weight="900", letter_spacing="0.05em"),
                align="center",
                justify="center", # Center align the header content
                spacing="2",
                width="100%",
                padding_bottom="4"
            ),
            rx.box(
                chart,
                width="100%",
                height="100%",
            ),
            width="100%",
            height="100%",
        ),
        bg=CARD_BG,
        padding="6",
        border_radius="xl",
        box_shadow="lg",
        border=f"1px solid {BORDER_COLOR}",
        width="100%",
        height=height
    )

def predictive_tab_content() -> rx.Component:
    return rx.vstack(
        # Row 1: Revenue Growth & Regression
        rx.grid(
            chart_container(
                "Revenue Growth Trajectory",
                rx.plotly(data=PredictiveState.revenue_growth_chart, height="300px"),
                height="420px"
            ),
            chart_container(
                "Regression Analysis",
                rx.plotly(data=PredictiveState.regression_chart, height="300px"),
                height="420px"
            ),
            columns={"initial": "1", "lg": "2"},
            spacing="6",
            width="100%",
        ),
        # Row 2: Survival & Propensity
        rx.grid(
            chart_container(
                "Survival Probability",
                rx.plotly(data=PredictiveState.survival_chart, height="300px"),
                height="420px"
            ),
            chart_container(
                "Propensity Matrix",
```

Reflex Dashboard Comprehensive Manual

```
        rx.plotly(data=PredictiveState.propensity_chart, height="300px"),
        height="420px"
    ),
    columns={"initial": "1", "lg": "2"},
    spacing="6",
    width="100%",
),

# Section Header
rx.center(
    rx.heading("DECISION MATRICES", size="6", color="white", font_weight="900", letter_spacing="0.05em"),
    padding_y="4",
    width="100%"
),

# Row 3: Decision Matrices
rx.grid(
    chart_container(
        "BCG Matrix (Share vs Growth)",
        rx.plotly(data=PredictiveState.bcg_matrix_chart, height="300px"),
        height="420px"
    ),
    chart_container(
        "Toxic SKU Diagnostic",
        rx.plotly(data=PredictiveState.toxic_sku_chart, height="300px"),
        height="420px"
    ),
    columns={"initial": "1", "lg": "2"},
    spacing="6",
    width="100%",
),

# Section Header Phase 3
rx.center(
    rx.heading("ADVANCED FORECASTING & ALERTS", size="6", color="white", font_weight="900", letter_spacing="0.05em"),
    padding_y="4",
    width="100%"
),

# Row 4: SKU Forecast & Seasonality
rx.grid(
    chart_container(
        "SKU-Level Revenue Forecast",
        rx.plotly(data=PredictiveState.sku_forecast_chart, height="300px"),
        height="420px"
    ),
    chart_container(
        "Seasonality & Promo Uplift",
        rx.plotly(data=PredictiveState.seasonality_chart, height="300px"),
        height="420px"
    ),
    columns={"initial": "1", "lg": "2"},
    spacing="6",
    width="100%",
),

# Row 5: Demand Planning Table
rx.box(
    rx.vstack(
        rx.hstack(
            rx.icon("shopping-cart", size=20, color="white"),
```

Reflex Dashboard Comprehensive Manual

```
        rx.heading("DEMAND PLANNING & REPLENISHMENT", size="4", color=TEXT_COLOR, font_weight="900",
letter_spacing="0.05em"),
        align="center",
        justify="center",
        spacing="2",
        width="100%",
        padding_bottom="4"
    ),
    rx.table.root(
        rx.table.header(
            rx.table.row(
                rx.table.column_header_cell("SKU ID", color="white", font_weight="bold"),
                rx.table.column_header_cell("Category", color="white", font_weight="bold"),
                rx.table.column_header_cell("Current Stock", text_align="center", color="white",
font_weight="bold"),
                rx.table.column_header_cell("Forecast Demand (M+1)", text_align="center", color="white",
font_weight="bold"),
                rx.table.column_header_cell("Reorder Qty", text_align="center", color="white",
font_weight="bold"),
                rx.table.column_header_cell("Status", text_align="center", color="white", font_weight="bold"),
            ),
            bg="rgba(255,255,255,0.05)"
        ),
        rx.table.body(
            rx.foreach(
                PredictiveState.demand_planning_display,
                lambda row: rx.table.row(
                    rx.table.cell(rx.text(row["SKU_ID"], font_weight="bold", color="white")),
                    rx.table.cell(rx.text(row["Category"], color="white")), # Changed to white
                    rx.table.cell(rx.text(row["Current Stock"], font_weight="bold", color="white"),
text_align="center"), # Changed
                    rx.table.cell(rx.text(row["Forecasted Demand (M+1)"], font_weight="bold", color="white"),
text_align="center"),
                    rx.table.cell(rx.text(row["Reorder Qty"], font_weight="bold", color="white"),
text_align="center"),
                    rx.table.cell(
                        rx.badge(row["Status"], color_scheme=row["status_color"], variant="solid"),
                        text_align="center"
                    ),
                    bg="rgba(255,255,255,0.01)"
                )
            )
        ),
        width="100%"
    ),
    width="100%",
    align="center" # Centering the content (table)
),
bg=CARD_BG,
padding="6",
border_radius="xl",
box_shadow="lg",
border=f"1px solid {BORDER_COLOR}",
width="100%",
),

# Row 6: Risk Alerts
rx.box(
    rx.vstack(
        rx.hstack(
            rx.icon("alert-triangle", size=20, color="#EF4444"),
```

Reflex Dashboard Comprehensive Manual

```
        rx.heading("OUT-OF-STOCK RISK ALERTS", size="4", color="#EF4444", font_weight="900",
letter_spacing="0.05em"),
        align="center",
        justify="center",
        spacing="2",
        width="100%",
        padding_bottom="4"
    ),
    rx.grid(
        rx.foreach(
            PredictiveState.risk_alerts,
            lambda item: rx.box(
                rx.vstack(
                    rx.hstack(
                        rx.badge(item["Risk Level"], color_scheme="red", variant="solid"),
                        rx.text(item["SKU_ID"], font_weight="bold", color="white"),
                        justify="between",
                        width="100%"
                    ),
                    rx.text(item["Alert Message"], color="white", font_weight="bold", font_size="sm"), # Changed
to bold white
                    rx.text(f"Days Cover: {item['Days Cover']} days", color="white", font_weight="bold",
font_size="xs"), # Changed to bold white
                    align="start",
                    spacing="2"
                ),
                bg="rgba(239, 68, 68, 0.1)", # Red tint
                padding="4",
                border_radius="lg",
                border="1px solid #7F1D1D"
            )
        ),
        columns={"initial": "1", "md": "2", "lg": "3"},
        spacing="4",
        width="100%"
    ),
    width="100%"
),
bg=CARD_BG,
padding="6",
border_radius="xl",
box_shadow="lg",
border=f"1px solid {BORDER_COLOR}",
width="100%",
),

width="100%",
spacing="6",
padding_bottom="40px",
on_mount=PredictiveState.load_predictive_data
)
```

File: reflex_dashboard/components/tables.py

UI Component Module: specific aspect of the dashboard interface.

```
import reflex as rx
from ..state import State
from ..colors import CARD_BG, BORDER_COLOR
```


Reflex Dashboard Comprehensive Manual

```
MONTH_COL_WIDTH = "110px"

def monthly_summary_table() -> rx.Component:
    return rx.box(
        # 1. Main Title Bar
        rx.flex(
            rx.box(
                rx.text(f"Monthly Summary of Gross Sale Value (Channel wise) {State.table_date_range_title}", font_size="md",
font_weight="900", color="white", letter_spacing="0.05em", text_align="center", width="100%"),
                rx.box(height="12px"), # Gap between title and subtitle
                rx.text("Breakdown by Channel and Top 5 States", font_size="xs", font_weight="bold", color="white",
letter_spacing="0.05em", text_align="center", width="100%"),
                rx.box(height="12px"), # Extra spacing
                width="100%",
            ),
            align="center",
            justify="center",
            padding="4",
            bg=CARD_BG,
            border_bottom=f"1px solid {BORDER_COLOR}",
            border_top_left_radius="xl",
            border_top_right_radius="xl",
        ),

        # 2. Table Container
        rx.box(
            # HEADER ROW
            rx.flex(
                rx.box(
                    rx.center(
                        rx.text("CHANNEL / REGION", font_weight="bold", color="white", font_size="10px",
letter_spacing="0.1em"),
                        height="100%",
                        width="100%"
                    ),
                    width="150px", min_width="150px", max_width="150px", padding_left="0", padding_y="0", border_right=f"1px
solid {BORDER_COLOR}"
                ),
                rx.foreach(
                    State.summary_table_columns,
                    lambda col: rx.flex(
                        rx.text(col, font_weight="bold", color="white", font_size="10px", text_align="center", width="100%",
letter_spacing="0.1em"),
                        width="100%", # Flex width
                        min_width=MONTH_COL_WIDTH,
                        flex="1",
                        padding_y="3",
                        border_right=f"1px solid {BORDER_COLOR}",
                        justify="center",
                        align="center",
                        overflow="hidden"
                    )
                ),
            ),

            rx.box(rx.text("TOTAL", font_weight="bold", color="white", font_size="10px", text_align="center",
letter_spacing="0.1em"), width="140px", min_width="140px", max_width="140px", padding_y="3", overflow="hidden", bg="#064E3B"),
            bg="#064E3B",
            width="100%", # Stretched
            min_width="max-content", # Prevent squishing
            align="center",
```

Reflex Dashboard Comprehensive Manual

```
border_bottom=f"1px solid {BORDER_COLOR}",
),

# BODY ROWS
rx.vstack(
  rx.foreach(
    State.monthly_summary_data,
    lambda row: rx.box(
      # Condition: Grand Total Footer vs Data Rows
      rx.cond(
        row.is_total,
        # FOOTER ROW (Grand Total)
        rx.flex(
          rx.box(rx.text(row.channel, font_weight="900", color="white", font_size="sm",
letter_spacing="0.05em", text_transform="capitalize"), width="150px", min_width="150px", max_width="150px", padding_left="4",
padding_y="2", border_right=f"1px solid {BORDER_COLOR}", overflow="hidden"),
          rx.foreach(
            row.monthly_values,
            lambda m, i: rx.flex(
              rx.text(m.value, color="white", font_size="10px", font_weight="medium",
text_align="center", width="100%"),
              width="100%",
              min_width=MONTH_COL_WIDTH,
              flex="1",
              padding_y="2",
              border_right=f"1px solid {BORDER_COLOR}",
              bg=rx.cond(i % 2 == 0, "rgba(255,255,255,0.03)", "rgba(255,255,255,0.01)"),
              justify="center",
              align="center",
              overflow="hidden"
            )
          ),
          rx.box(rx.text(row.total_value, font_weight="900", color="#22c55e", font_size="sm",
text_align="center"), width="140px", min_width="140px", max_width="140px", padding_y="2", overflow="hidden", bg="#000000"), #
Black for total column

          bg="#064E3B", # Dark Green for Total ROW
          width="100%",
          min_width="max-content",
          align="center",
          border_top=f"1px solid {BORDER_COLOR}",
        ),
      # DATA ROW (Channel + Accordion)
      rx.accordion.root(
        rx.accordion.item(
          rx.accordion.trigger(
            rx.flex(
              rx.box(
                rx.hstack(
                  rx.icon("chevron-down", size=16, color="white"),
                  rx.text(row.channel, font_weight="bold", color="white", font_size="10px",
white_space="nowrap", text_overflow="ellipsis", overflow="hidden", text_transform="capitalize"),
                  spacing="2",
                  align="center"
                ),
                width="150px",
                min_width="150px",
                max_width="150px",
                padding_left="2",
                padding_y="2",
                border_right=f"1px solid {BORDER_COLOR}",
                overflow="hidden"
              )
            )
          )
        )
      )
    )
  )
)
```

Reflex Dashboard Comprehensive Manual

```
),
    rx.foreach(
        row.monthly_values,
        lambda m, i: rx.flex(
            rx.text(m.value, color="white", font_size="10px", text_align="center",
width="100%", white_space="nowrap", text_overflow="ellipsis", overflow="hidden"),
            width="100%",
            min_width=MONTH_COL_WIDTH,
            flex="1",
            padding_y="2",
            border_right=f"1px solid {BORDER_COLOR}",
            bg=rx.cond(i % 2 == 0, "transparent", "rgba(255,255,255,0.02)"),
            justify="center",
            align="center",
            overflow="hidden"
        )
    ),
    rx.box(rx.text(row.total_value, font_weight="bold", color="white",
font_size="10px", text_align="center"), width="140px", min_width="140px", max_width="140px", padding_y="2", overflow="hidden",
bg="#000000"),

    width="100%",
    min_width="max-content",
    align="center",
    bg="transparent",
    _hover={"bg": "rgba(255,255,255,0.03)"}
),
padding="0",
_hover={"bg": "transparent"},
),
rx.accordion.content(
    rx.vstack(
        rx.foreach(
            row.children,
            lambda child: rx.flex(
                rx.box(
                    rx.text(child.state, color="gray.400", font_size="10px",
padding_left="8"),

                    width="150px",
                    min_width="150px",
                    max_width="150px",
                    padding_y="2",
                    border_right=f"1px solid {BORDER_COLOR}",
                    overflow="hidden"
                ),
                rx.foreach(
                    child.monthly_values,
                    lambda m, i: rx.flex(
                        rx.text(m.value, color="gray.400", font_size="10px",
text_align="center", width="100%", white_space="nowrap", text_overflow="ellipsis", overflow="hidden"),
                        width="100%",
                        min_width=MONTH_COL_WIDTH,
                        flex="1",
                        padding_y="2",
                        border_right=f"1px solid {BORDER_COLOR}",
                        bg=rx.cond(i % 2 == 0, "rgba(0,0,0,0.2)", "rgba(0,0,0,0.3)"),
                        justify="center",
                        align="center",
                        overflow="hidden"
                    )
                ),
            ),
            rx.box(rx.text(child.total_value, color="gray.400", font_size="10px",
```

Reflex Dashboard Comprehensive Manual

```
text_align="center"), width="140px", min_width="140px", max_width="140px", padding_y="2", overflow="hidden", bg="#000000"),
    bg="rgba(0,0,0,0.4)",
    width="100%",
    min_width="max-content",
    align="center",
    border_top=f"1px dotted {BORDER_COLOR}",
    )
    ),
    width="100%",
    min_width="max-content",
    spacing="0",
    ),
    padding="0",
    ),
    border_width="0",
    padding="0",
    ),
    type="multiple",
    collapsible=True,
    width="100%",
    min_width="max-content",
    variant="ghost",
    )
    ),
    width="100%",
    border_bottom=f"1px solid {BORDER_COLOR}",
    bg="transparent"
    )
    ),
    width="100%",
    spacing="0"
    ),
    width="100%",
    overflow="auto",
    ),
    bg="#000000",
    border=f"1px solid {BORDER_COLOR}",
    border_radius="xl",
    width="100%",
    box_shadow="lg",
    )
)
```

```
def monthly_sales_return_table() -> rx.Component:
    return rx.box(
        # 1. Main Title Bar
        rx.flex(
            rx.box(
                rx.text(f"Monthly Sales Return (Channel wise) {State.table_date_range_title}", font_size="md",
font_weight="900", color="white", letter_spacing="0.05em", text_align="center", width="100%"),
                width="100%",
            ),
            align="center",
            justify="center",
            padding="4",
            bg=CARD_BG,
            border_bottom=f"1px solid {BORDER_COLOR}",
            border_top_left_radius="xl",
            border_top_right_radius="xl",
        ),
        rx.box(
```

Reflex Dashboard Comprehensive Manual

```
# HEADER ROW
rx.flex(
  rx.box(
    rx.center(
      rx.text("CHANNEL", font_weight="bold", color="white", font_size="10px", letter_spacing="0.1em"),
      height="100%",
      width="100%"
    ),
    width="150px", min_width="150px", max_width="150px", padding_left="0", padding_y="0", border_right=f"1px
solid {BORDER_COLOR}"
  ),
  rx.foreach(
    State.summary_table_columns,
    lambda col: rx.flex(
      rx.text(col, font_weight="bold", color="white", font_size="10px", text_align="center", width="100%",
letter_spacing="0.1em"),
      width="100%",
      min_width=MONTH_COL_WIDTH,
      flex="1",
      padding_y="3",
      border_right=f"1px solid {BORDER_COLOR}",
      justify="center",
      align="center",
      overflow="hidden"
    )
  ),
  rx.box(rx.text("TOTAL", font_weight="bold", color="white", font_size="10px", text_align="center",
letter_spacing="0.1em"), width="140px", min_width="140px", max_width="140px", padding_y="3", overflow="hidden", bg="#064E3B",
bg="#064E3B",
width="100%",
min_width="max-content",
align="center",
border_bottom=f"1px solid {BORDER_COLOR}",
),

# BODY ROWS
rx.vstack(
  rx.foreach(
    State.monthly_sales_return_data,
    lambda row: rx.box(
      rx.flex(
        rx.box(rx.text(row.channel, font_weight="bold", color="white", font_size="10px",
text_transform="capitalize"), width="150px", min_width="150px", max_width="150px", padding_left="4", padding_y="2",
border_right=f"1px solid {BORDER_COLOR}", overflow="hidden"),
        rx.foreach(
          row.monthly_values,
          lambda m, i: rx.flex(
            rx.text(m.value, color="white", font_size="10px", text_align="center", width="100%",
white_space="nowrap", text_overflow="ellipsis", overflow="hidden"),
            width="100%",
            min_width=MONTH_COL_WIDTH,
            flex="1",
            padding_y="2",
            border_right=f"1px solid {BORDER_COLOR}",
            bg=rx.cond(
              row.is_total,
              rx.cond(i % 2 == 0, "rgba(255,255,255,0.05)", "rgba(255,255,255,0.02)"),
              rx.cond(i % 2 == 0, "transparent", "rgba(255,255,255,0.04)")
            ),
            justify="center",
```

Reflex Dashboard Comprehensive Manual

```
                align="center",
                overflow="hidden"
            )
        ),
        rx.box(rx.text(row.total_value, font_weight="bold", color=rx.cond(row.is_total, "#22c55e",
"white"), font_size=rx.cond(row.is_total, "sm", "10px"), text_align="center"), width="140px", min_width="140px",
max_width="140px", padding_y="2", overflow="hidden", bg=rx.cond(row.is_total, "#000000", "#000000")),
            bg=rx.cond(row.is_total, "#064E3B", "transparent"),
            width="100%", # Added explicitly to inner flex
            align="center",
            border_top=rx.cond(row.is_total, f"1px solid {BORDER_COLOR}", "none"),
            border_bottom=f"1px solid {BORDER_COLOR}"
        ),
        width="100%"
    )
),
width="100%",
spacing="0"
),
width="100%",
overflow="auto",
),
bg="#000000",
width="100%",
border_radius="xl",
border=f"1px solid {BORDER_COLOR}",
box_shadow="lg",
)

def invoice_vs_return_table() -> rx.Component:
    COL_WIDTH = "68px" # Smaller cols for Inv/Ret
    MONTH_COL_WIDTH_INV = "150px"

    return rx.box(
        # 1. Main Title Bar
        rx.flex(
            rx.box(
                rx.hstack(
                    rx.icon("file-text", size=20, color="white"),
                    rx.text(f"Invoice vs Sales Return {State.table_date_range_title}", font_size="md", font_weight="900",
color="white", letter_spacing="0.05em", text_align="center"),
                    justify="center",
                    align="center",
                    spacing="2"
                ),
                width="100%",
            ),
            align="center",
            justify="center",
            padding="4",
            bg=CARD_BG,
            border_bottom=f"1px solid {BORDER_COLOR}",
            border_top_left_radius="xl",
            border_top_right_radius="xl",
        ),

        rx.box(
            # COMPLEX HEADER
            rx.flex(
                # Column 1: Month (Merged Vertically)
                rx.box(
```

Reflex Dashboard Comprehensive Manual

```
rx.center(  
  rx.text("MONTH", font_weight="bold", color="white", font_size="10px", letter_spacing="0.1em"),  
  height="100%",  
  width="100%"  
),  
width=MONTH_COL_WIDTH_INV,  
min_width=MONTH_COL_WIDTH_INV,  
border_right=f"1px solid {BORDER_COLOR}",  
height="auto", # Fill height of parent flex  
flex_shrink=0  
),  
  
# Columns: Channels (Iterated as VStacks)  
rx.foreach(  
  State.invoice_return_columns,  
  lambda col: rx.vstack(  
    # Row A: Title  
    rx.flex(  
      rx.text(col, font_weight="bold", color="white", font_size="10px", text_align="center",  
width="100%", letter_spacing="0.1em", text_transform="capitalize"),  
      width="100%",  
      padding_y="3",  
      justify="center",  
      align="center",  
      overflow="hidden",  
      border_bottom=f"1px solid {BORDER_COLOR}",  
      flex="1"  
    ),  
    # Row B: Inv / Ret  
    rx.flex(  
      rx.box(rx.text("Inv", color="white", font_size="10px", font_weight="bold", text_align="center"),  
width="50%", border_right=f"1px solid {BORDER_COLOR}", padding_y="2"),  
      rx.box(rx.text("Ret", color="#FA5252", font_size="10px", font_weight="bold",  
text_align="center"), width="50%", padding_y="2"),  
      width="100%",  
      justify="center",  
      align="center",  
      overflow="hidden",  
      spacing="0"  
    ),  
    width="100%",  
    min_width="136px",  
    flex="1",  
    border_right=f"1px solid {BORDER_COLOR}",  
    spacing="0"  
  )  
),  
# Grand Total Column Header  
rx.vstack(  
  rx.flex(  
    rx.text("Grand Total", font_weight="bold", color="white", font_size="10px", text_align="center",  
width="100%", letter_spacing="0.1em"),  
    width="100%",  
    min_width="136px",  
    padding_y="3",  
    border_right=f"1px solid {BORDER_COLOR}",  
    justify="center",  
    align="center",  
    overflow="hidden",  
    flex="1"  
  ),  
),
```

Reflex Dashboard Comprehensive Manual

```
        rx.flex(
            rx.box(rx.text("Inv", color="white", font_size="10px", font_weight="bold", text_align="center"),
width="50%", border_right=f"1px solid {BORDER_COLOR}", padding_y="2"),
            rx.box(rx.text("Ret", color="#FA5252", font_size="10px", font_weight="bold", text_align="center"),
width="50%", border_right=f"1px solid {BORDER_COLOR}", padding_y="2"),
            width="100%",
            min_width="136px",
            flex="1",
            justify="center",
            align="center",
            overflow="hidden",
            spacing="0",
            border_top=f"1px solid {BORDER_COLOR}"
        ),
        bg="transparent",
        width="100%",
        min_width="136px",
        flex="1",
        spacing="0"
    ),
    bg="#064E3B",
    width="100%",
    min_width="max-content",
    align="stretch"
),

# BODY
rx.vstack(
    rx.foreach(
        State.invoice_vs_return_data,
        lambda row, i: rx.box(
            rx.flex(
                rx.box(rx.text(row.month, font_weight="bold", color="white", font_size="10px"),
width=MONTH_COL_WIDTH_INV, min_width=MONTH_COL_WIDTH_INV, padding_left="4", padding_y="3", border_right=f"1px solid
{BORDER_COLOR}", overflow="hidden"),

                # Check if it has 'channels' list (Normal Rows)
                rx.cond(
                    row.is_pct,
                    # PCT ROW
                    rx.foreach(
                        row.channels,
                        lambda chan, j: rx.flex(
                            rx.text(chan.val, color=chan.color, font_size="sm", font_weight="bold",
text_align="center", width="100%"),
                            width="100%",
                            min_width="136px",
                            flex="1",
                            padding_y="2",
                            bg=rx.cond(
                                row.is_total,
                                rx.cond(j % 2 == 0, "rgba(255,255,255,0.05)", "rgba(255,255,255,0.02)"),
                                rx.cond(j % 2 == 0, "transparent", "rgba(255,255,255,0.04)")
                            ),
                            border_right=f"1px solid {BORDER_COLOR}",
                            justify="center",
                            align="center"
                        )
                    ),
                    # DATA ROW

```


Reflex Dashboard Comprehensive Manual

```
        rx.foreach(
            row.channels,
            lambda chan, j: rx.flex(
                rx.box(rx.text(chan.inv, color=rx.cond(chan.channel=="Grand Total", "#22c55e",
"white"), font_size="10px", text_align="center"), width="50%", border_right=f"1px solid {BORDER_COLOR}", padding_y="2"),
                rx.box(rx.text(chan.ret, color="#FA5252", font_size="10px", text_align="center"),
width="50%", border_right=f"1px solid {BORDER_COLOR}", padding_y="2"),
                width="100%",
                min_width="136px",
                flex="1",
                justify="center",
                align="center",
                overflow="hidden",
                spacing="0",
                bg=rx.cond(
                    row.is_total,
                    rx.cond(j % 2 == 0, "rgba(255,255,255,0.05)", "rgba(255,255,255,0.02)"),
                    rx.cond(j % 2 == 0, "transparent", "rgba(255,255,255,0.04)")
                )
            )
        ),
        bg=rx.cond(
            row.is_total,
            "#064E3B",
            rx.cond(i % 2 == 0, "transparent", "rgba(255,255,255,0.04)")
        ), # Alternating colors
        width="100%",
        border_bottom=f"1px solid {BORDER_COLOR}",
        align="center"
    ),
    width="100%"
)
),
width="100%",
spacing="0"
),
width="100%",
overflow="auto",
),
bg="#000000",
width="100%",
border_radius="xl",
border=f"1px solid {BORDER_COLOR}",
box_shadow="lg",
)
```

File: reflex_dashboard/state/ai.py

State Management Module: Handles data logic and events.

```
import reflex as rx
import os
from google import genai
from typing import List, Dict
from .tables import TableState

from dotenv import load_dotenv
```

Reflex Dashboard Comprehensive Manual

```
load_dotenv() # Explicitly load .env file

# --- Configuration ---
GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY", "")

class AIState(TableState):
    # Chat State
    chat_history: List[Dict[str, str]] = []
    current_question: str = ""
    is_ai_thinking: bool = False

    def set_current_question(self, val: str):
        self.current_question = val

    async def ask_gemini(self):
        if not self.current_question:
            return

        question = self.current_question
        self.chat_history.append({"role": "user", "text": question})
        self.current_question = ""
        self.is_ai_thinking = True
        yield

        try:
            # Initialize Client
            api_key = GEMINI_API_KEY
            if not api_key:
                api_key = os.environ.get("GEMINI_API_KEY", "")

            if not api_key:
                response_text = "Please set your GEMINI_API_KEY in the environment or code to use this feature."
                self.chat_history.append({"role": "ai", "text": response_text})
                self.is_ai_thinking = False
                return

            client = genai.Client(api_key=api_key)

            context = "No data loaded yet."
            if not self.filtered_df.empty:
                # Create a summary context from valid filtered data
                df = self.filtered_df
                total_rev = df["Sales Amount"].sum()

                # Columns summary
                cols = ", ".join(df.columns)

                context = f"""
                You are a helpful data assistant. Analyze this sales data summary:
                - Total Revenue: ${total_rev / 100000000:.2f} Cr
                - Row Count: {len(df)}
                - Columns: {cols}

                User Question: {question}
                Answer concisely based on this context.
                """
            else:
                context = f"User Question: {question}"

            # Generate content using the new SDK
            response = client.models.generate_content(
```

Reflex Dashboard Comprehensive Manual

```
        model='gemini-3-pro-preview',
        contents=context
    )
    response_text = response.text

    self.chat_history.append({"role": "ai", "text": response_text})

except Exception as e:
    self.chat_history.append({"role": "ai", "text": f"Error: {str(e)}"})

self.is_ai_thinking = False
```

File: reflex_dashboard/state/base.py

State Management Module: Handles data logic and events.

```
import reflex as rx
import pandas as pd
from typing import List, Dict, Any

class BaseState(rx.State):
    """The base state with shared data."""
    # Raw Data
    # Raw data is now stored in StateStore to avoid serialization overhead

    # Debug / Deployment Status
    deployment_status: str = ""

    # Upload State
    is_upload_modal_open: bool = True
    sales_data_uploaded: bool = False
    courier_data_uploaded: bool = False

    # Sidebar State
    is_sidebar_open: bool = True

    # Dashboard Tab State (Controls active tab)
    current_tab: str = ""

    def toggle_sidebar(self):
        self.is_sidebar_open = not self.is_sidebar_open

    def close_upload_modal(self):
        self.is_upload_modal_open = False

    def get_token(self) -> str:
        """Get the current session ID token."""
        return self.router.session.session_id
```

File: reflex_dashboard/state/charts.py

State Management Module: Handles data logic and events.

```
import reflex as rx
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
```

Reflex Dashboard Comprehensive Manual

```
from .kpi import KPIState
from ..colors import CHART_COLORS, ACCENT_COLOR, TEXT_COLOR, BORDER_COLOR, CARD_BG

class ChartState(KPIState):

    @rx.var
    def monthly_sales_chart(self) -> go.Figure:
        if self.filtered_df.empty:
            return go.Figure()

        data = self.filtered_df.groupby("Month_Date")["Sales Amount"].sum().reset_index()
        data["Month"] = data["Month_Date"].dt.strftime('%b-%Y')
        data["Sales Amount"] = (data["Sales Amount"] / 10000000).round(2)

        # Create colors for each point
        marker_colors = [CHART_COLORS[i % len(CHART_COLORS)] for i in range(len(data))]

        fig = go.Figure()

        fig.add_trace(go.Scatter(
            x=data["Month"],
            y=data["Sales Amount"],
            mode='lines+markers+text',
            text=data["Sales Amount"],
            textposition='middle center',
            textfont=dict(
                size=10,
                color='white',
                weight='bold'
            ),
            marker=dict(
                size=35,
                color=marker_colors,
                symbol='circle',
                line=dict(width=2, color='white'),
                opacity=1
            ),
            line=dict(
                color=ACCENT_COLOR,
                width=3
            ),
            hoverinfo='x+y',
        ))

        fig.update_layout(
            template="plotly_dark",
            height=400, # Explicit height override
            paper_bgcolor="rgba(0,0,0,0)", # Transparent to let card bg show
            plot_bgcolor="rgba(0,0,0,0)",
            font_color=TEXT_COLOR,
            margin=dict(l=20, r=20, t=20, b=20),
            hovermode="x unified",
            xaxis=dict(
                showgrid=True,
                gridwidth=1,
                gridcolor=BORDER_COLOR,
                showline=False
            ),
            yaxis=dict(
                showgrid=True,
                gridwidth=1,
```

Reflex Dashboard Comprehensive Manual

```
        gridcolor=BORDER_COLOR,
        showline=False,
        showticklabels=False # Hide y-axis labels as values are in bubbles
    )
)
return fig

@rx.var
def state_sales_chart(self) -> go.Figure:
    if self.filtered_df.empty:
        return go.Figure()

    data = self.filtered_df.groupby("CUSTOMER STATE")["Sales Amount"].sum().reset_index()
    data = data.sort_values("Sales Amount", ascending=False)

    if len(data) > 10:
        top_10 = data.head(10)
        others_val = data.iloc[10:]["Sales Amount"].sum()
        others = pd.DataFrame([{'CUSTOMER STATE': 'Other states', 'Sales Amount': others_val}])
        data = pd.concat([top_10, others], ignore_index=True)

    data["Sales Amount"] = (data["Sales Amount"] / 10000000).round(2)
    # Sort for display
    data = data.sort_values("Sales Amount", ascending=True)

    fig = px.treemap(
        data,
        path=["CUSTOMER STATE"],
        values="Sales Amount",
        color="CUSTOMER STATE",
        hover_data=['Sales Amount'],
        color_discrete_sequence=CHART_COLORS,
        color_discrete_map={"Other states": "#2D3748"}, # Dark Gray for "Other states"
        template="plotly_dark",
    )

    fig.update_traces(
        textinfo="label+value+percent entry",
        textfont=dict(size=14, color="white"),
        marker=dict(line=dict(width=1, color='white')),
        textposition="middle center"
    )

    fig.update_layout(
        height=580, # Explicit height override
        paper_bgcolor=CARD_BG,
        plot_bgcolor=CARD_BG,
        font_color=TEXT_COLOR,
        margin=dict(l=0, r=0, t=0, b=0), # Edge to Edge
    )
    fig.update_xaxes(showgrid=False)
    fig.update_yaxes(showgrid=True, gridwidth=1, gridcolor="#4A5568")
    return fig

@rx.var
def product_sales_chart(self) -> go.Figure:
    if self.filtered_df.empty:
        return go.Figure()

    data = self.filtered_df.groupby("Product")["Sales Amount"].sum().reset_index()
    data = data.sort_values("Sales Amount", ascending=False)
```

Reflex Dashboard Comprehensive Manual

```
if len(data) > 5:
    top_5 = data.head(5)
    others_val = data.iloc[5:]["Sales Amount"].sum()
    others = pd.DataFrame([{'Product': 'Other', 'Sales Amount': others_val}])
    data = pd.concat([top_5, others], ignore_index=True)

data["Sales Amount"] = (data["Sales Amount"] / 10000000).round(2)

# Calculate Total for Center Text
total_sales_val = data["Sales Amount"].sum()

fig = px.pie(
    data,
    names="Product",
    values="Sales Amount",
    template="plotly_dark",
    hole=0.5 # Increased hole size for text
)
fig.update_traces(
    textposition='inside',
    textinfo='percent+label',
    sort=True,
    direction='clockwise'
)
fig.update_layout(
    height=580, # Explicit height override
    paper_bgcolor="#000000",
    plot_bgcolor="#000000",
    font_color=TEXT_COLOR,
    margin=dict(l=20, r=20, t=20, b=20),
    showlegend=True,
    annotations=[dict(text=f"Total Sales<br>{total_sales_val:.2f} Cr", x=0.5, y=0.5, font_size=16, showarrow=False,
font_weight="bold", font_color="white")]
)
return fig

@rx.var
def supply_sales_chart(self) -> go.Figure:
    if self.filtered_df.empty or "TYPE OF SUPPLY" not in self.filtered_df.columns:
        return go.Figure()

    data = self.filtered_df.groupby("TYPE OF SUPPLY")["Sales Amount"].sum().reset_index()
    data["Sales Amount"] = (data["Sales Amount"] / 10000000).round(2)

    # Calculate Total for Center Text
    total_sales_val = data["Sales Amount"].sum()

    fig = px.pie(
        data,
        names="TYPE OF SUPPLY",
        values="Sales Amount",
        template="plotly_dark",
        hole=0.5 # Match Product chart
    )
    fig.update_traces(
        textposition='inside',
        textinfo='percent+label',
        texttemplate='%{percent}<br>(%{value:.2f} Cr)',
        sort=True
    )
```

Reflex Dashboard Comprehensive Manual

```
fig.update_layout(
    height=580, # Explicit height override
    paper_bgcolor="#000000",
    plot_bgcolor="#000000",
    font_color=TEXT_COLOR,
    margin=dict(l=20, r=20, t=20, b=20),
    showlegend=True,
    annotations=[dict(text=f"Total Sales<br>{total_sales_val:.2f} Cr", x=0.5, y=0.5, font_size=16, showarrow=False,
font_weight="bold", font_color="white")]
)
return fig

# --- Channel Chart Logic (moved from component/chart snippet if existed,
# but seems it was using state var `channel_sales_chart` which was missing in original file view?
# I saw `State.channel_sales_chart` in `channel_sales_card` text.
# I'll implement it here assuming it follows similar pattern.

@rx.var
def channel_sales_chart(self) -> go.Figure:
    if self.filtered_df.empty:
        return go.Figure()

    data = self.filtered_df.groupby("Channel")["Sales Amount"].sum().reset_index()
    data = data.sort_values("Sales Amount", ascending=False)

    data["Sales Amount"] = (data["Sales Amount"] / 10000000).round(2)
    total_val = data["Sales Amount"].sum()

    fig = px.pie(
        data,
        names="Channel",
        values="Sales Amount",
        template="plotly_dark",
        hole=0.7,
        color_discrete_sequence=CHART_COLORS
    )

    fig.update_traces(
        textinfo='none', # Hide labels on chart, use legend
        hoverinfo='label+value+percent'
    )

    fig.update_layout(
        height=580,
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        font_color=TEXT_COLOR,
        margin=dict(l=0, r=0, t=0, b=0),
        showlegend=False, # Custom legend used
        annotations=[dict(text=f"Total Sales<br>{total_val:.2f} Cr", x=0.5, y=0.5, font_size=20, showarrow=False,
font_weight="bold", font_color="white")]
    )
    return fig

@rx.var
def channel_sales_stats(self) -> list[dict]:
    if self.filtered_df.empty:
        return []

    data = self.filtered_df.groupby("Channel")["Sales Amount"].sum().reset_index()
    data = data.sort_values("Sales Amount", ascending=False)
```

Reflex Dashboard Comprehensive Manual

```
total = data["Sales Amount"].sum()

results = []
for i, row in enumerate(data.to_dict('records')):
    val = row["Sales Amount"]
    pct = val / total * 100 if total > 0 else 0
    results.append({
        "channel": row["Channel"],
        "value": f"?{val/10000000:.2f} Cr",
        "pct": f"{pct:.1f}%",
        "color": CHART_COLORS[i % len(CHART_COLORS)]
    })
return results
```

File: `reflex_dashboard/state/customer.py`

State Management Module: Handles data logic and events.

```
import reflex as rx
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
from .base import BaseState
from ..colors import CHART_COLORS, TEXT_COLOR, CARD_BG, BORDER_COLOR

class CustomerState(BaseState):
    """State for the Customer Science tab."""

    @rx.var
    def rfm_radar_chart(self) -> go.Figure:
        """Mock data for RFM Behavioral Radar."""
        categories = ['Recency', 'Frequency', 'Monetary', 'Consistency', 'Bundling']
        values = [80, 65, 40, 70, 55] # Example shape

        fig = go.Figure()

        fig.add_trace(go.Scatterpolar(
            r=values + [values[0]], # Close the loop
            theta=categories + [categories[0]],
            fill='toself',
            fillcolor='rgba(0, 255, 255, 0.2)', # Neon Cyan tint
            line=dict(color='#00FFFF', width=3),
            marker=dict(size=5, color='#00FFFF'),
            name='Activity'
        ))

        # Add a secondary "benchmark" or "avg" shape for visual complexity (reddish)
        values_2 = [50, 50, 50, 50, 50]
        fig.add_trace(go.Scatterpolar(
            r=values_2 + [values_2[0]],
            theta=categories + [categories[0]],
            fill='toself',
            fillcolor='rgba(255, 0, 255, 0.1)', # Neon Pink tint
            line=dict(color='#FF00FF', width=2, dash='dot'),
            marker=dict(size=0),
            name='Benchmark'
        ))

        fig.update_layout(
```


Reflex Dashboard Comprehensive Manual

```
polar=dict(
    radialaxis=dict(
        visible=True,
        range=[0, 100],
        showticklabels=False, # Clean look
        gridcolor='rgba(255,255,255,0.1)'
    ),
    angularaxis=dict(
        tickfont=dict(size=16, color='gray', family="Inter", weight="bold"),
        gridcolor='rgba(255,255,255,0.1)'
    ),
    bgcolor='rgba(0,0,0,0)'
),
paper_bgcolor='rgba(0,0,0,0)',
plot_bgcolor='rgba(0,0,0,0)',
margin=dict(l=40, r=40, t=80, b=20), # Increased top margin for Recency label
showlegend=False,
height=660,
hoverlabel=dict(bgcolor="white", font_size=16, font_family="Inter", font_color="black"),
font=dict(size=14),
)
return fig
```

@rx.var

```
def market_basket_data(self) -> list[dict]:
    """Mock data for Market Basket Associations."""
    return [
        {"pairs": "RG 100g + Pan", "lift": 4.2, "confidence": 82, "support": 15},
        {"pairs": "RG 117g + RG 4g", "lift": 3.8, "confidence": 75, "support": 12},
        {"pairs": "Elaichi + Twin", "lift": 2.5, "confidence": 45, "support": 8},
        {"pairs": "Saffron + 100g", "lift": 2.1, "confidence": 38, "support": 5},
        {"pairs": "Dates + Silver", "lift": 1.9, "confidence": 32, "support": 4},
        {"pairs": "Supari + Mix", "lift": 1.5, "confidence": 28, "support": 3},
    ]
```

@rx.var

```
def revenue_trends_chart(self) -> go.Figure:
    """Mock data for New vs Returning Revenue."""
    months = ["Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec", "Jan", "Feb"]
    new_rev = [15, 14, 13, 17, 18, 20, 24, 23, 20, 18, 19] # Smoothed random
    ret_rev = [20, 22, 28, 30, 35, 38, 48, 55, 35, 38, 40]

    # Area chart needs stacked values usually, or just direct values with fill

    fig = go.Figure()

    # New Revenue (Bottom)
    fig.add_trace(go.Scatter(
        x=months, y=new_rev,
        mode='lines',
        line=dict(width=3, color='#00BFFF'), # Neon Blue
        stackgroup='one',
        fillcolor='rgba(0, 191, 255, 0.4)',
        name='New Revenue'
    ))

    # Returning Revenue (Top)
    fig.add_trace(go.Scatter(
        x=months, y=ret_rev,
        mode='lines',
        line=dict(width=3, color='#9D00FF'), # Neon Purple
```

Reflex Dashboard Comprehensive Manual

```
        stackgroup='one',
        fillcolor='rgba(157, 0, 255, 0.2)',
        name='Returning Revenue'
    ))

    fig.update_layout(
        template="plotly_dark",
        paper_bgcolor='rgba(0,0,0,0)',
        plot_bgcolor='rgba(0,0,0,0)',
        font_color=TEXT_COLOR,
        margin=dict(l=10, r=10, t=20, b=20),
        height=660,
        showlegend=True,
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1, font=dict(color="white")),
        hovermode="x unified",
        xaxis=dict(showgrid=False, showline=False, tickfont=dict(size=10, color='gray')),
        yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)', gridwidth=1, showline=False, zeroline=False,
        tickfont=dict(size=10, color='gray')),
        clickmode="event+select", # Trying to avoid interactions blocking view
        hoverlabel=dict(bgcolor="white", font_size=16, font_family="Inter", font_color="black"),
        font=dict(size=14),
    )
    return fig

@rx.var
def wallet_share_chart(self) -> go.Figure:
    """Mock data for Customer Wallet Share."""
    dates = ["Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec", "Jan", "Feb"]
    share = [18, 17.5, 17.5, 18, 19, 18.5, 19.5, 20, 20.5, 21, 21.5] # Slow growth

    fig = go.Figure()

    # Step line equivalent
    fig.add_trace(go.Scatter(
        x=dates, y=share,
        mode='lines',
        line=dict(color='#39FF14', width=4, shape='hv'), # Neon Lime
        name='Wallet Share'
    ))

    fig.update_layout(
        template="plotly_dark",
        paper_bgcolor='rgba(0,0,0,0)',
        plot_bgcolor='rgba(0,0,0,0)',
        font_color=TEXT_COLOR,
        margin=dict(l=10, r=10, t=20, b=20),
        height=660,
        showlegend=False,
        xaxis=dict(showgrid=False, showline=True, linecolor='gray', tickfont=dict(size=10, color='gray')),
        yaxis=dict(
            showgrid=True, gridcolor='rgba(255,255,255,0.1)', gridwidth=1,
            showline=False,
            zeroline=False,
            tickfont=dict(size=10, color='gray'),
            range=[0, 24]
        ),
        hoverlabel=dict(bgcolor="white", font_size=16, font_family="Inter", font_color="black"),
        font=dict(size=14),
    )
    return fig
```

Reflex Dashboard Comprehensive Manual

```
@rx.var
def ltv_segments_chart(self) -> go.Figure:
    """Mock data for Predictive LTV Segments (Donut)."""
    labels = ['Champions', 'Loyal', 'Potential', 'Churning']
    values = [55, 25, 15, 5]
    colors = ['#2C3E50', '#4A5568', '#718096', '#F56565'] # Dark Blue/Grays + Red for Churn
    # Adjust colors to match screenshot: Dark blue dominant, some gray, some red
    colors = ['#39FF14', '#A0AEC0', '#718096', '#FF0000'] # Very Green, Gray, Gray, Very Red

    fig = go.Figure(data=[go.Pie(
        labels=labels,
        values=values,
        hole=0.6,
        marker=dict(colors=colors, line=dict(color='#000000', width=2)),
        sort=False
    )])

    fig.update_traces(
        textinfo='none',
        hoverinfo='label+percent'
    )

    fig.update_layout(
        template="plotly_dark",
        paper_bgcolor='rgba(0,0,0,0)',
        plot_bgcolor='rgba(0,0,0,0)',
        font_color=TEXT_COLOR,
        margin=dict(l=10, r=10, t=10, b=10),
        height=660,
        showlegend=False, # Custom legend in UI
        hoverlabel=dict(bgcolor="white", font_size=16, font_family="Inter", font_color="black"),
        font=dict(size=14),
    )
    return fig

class CustomerBehaviorState(BaseState):
    """State for the Behavioral Metrics & AI section."""

    # --- Interactive AOV Calculator ---
    aov_revenue: float = 0.0
    aov_orders: int = 0
    calculated_aov: float = 0.0

    def calculate_aov(self):
        if self.aov_orders > 0:
            self.calculated_aov = round(self.aov_revenue / self.aov_orders, 2)
        else:
            self.calculated_aov = 0.0

    def set_aov_revenue(self, val):
        try:
            self.aov_revenue = float(val)
        except ValueError:
            self.aov_revenue = 0.0

    def set_aov_orders(self, val):
        try:
            self.aov_orders = int(val)
        except ValueError:
            self.aov_orders = 0
```

Reflex Dashboard Comprehensive Manual

```
# --- Feature Importance Chart ---
@rx.var
def feature_importance_chart(self) -> go.Figure:
    try:
        df = pd.read_csv("data/feature_importance.csv")
        df = df.sort_values(by="Importance", ascending=True) # Ascending for horizontal bar
    except Exception:
        # Fallback if file not found
        data = {'Feature': ['Recency', 'Frequency', 'Monetary'], 'Importance': [0.5, 0.3, 0.2]}
        df = pd.DataFrame(data)

    fig = go.Figure(go.Bar(
        x=df['Importance'],
        y=df['Feature'],
        orientation='h',
        marker=dict(
            color=df['Importance'],
            colorscale=[[0, '#00BFFF'], [1, '#48BB78']], # Blue to Green
            line=dict(width=0)
        ),
        text=df['Importance'],
        textposition='auto',
    ))

    fig.update_layout(
        template="plotly_dark",
        paper_bgcolor='rgba(0,0,0,0)',
        plot_bgcolor='rgba(0,0,0,0)',
        font_color=TEXT_COLOR,
        margin=dict(l=20, r=20, t=10, b=10),
        height=300,
        xaxis=dict(visible=False),
        yaxis=dict(tickfont=dict(size=12, color='white')),
        bargap=0.2
    )
    return fig

# --- Cohort Retention Heatmap ---
@rx.var
def cohort_heatmap_chart(self) -> go.Figure:
    try:
        df = pd.read_csv("data/cohort_retention.csv")
        cohorts = df['Cohort'].tolist()
        # Extract only the M columns for z-values
        z_values = df[['M0', 'M1', 'M2', 'M3', 'M4']].values.tolist()
        x_labels = ['M0', 'M1', 'M2', 'M3', 'M4']
    except Exception:
        cohorts = ['Jan', 'Feb']
        z_values = [[100, 50], [100, 60]]
        x_labels = ['M0', 'M1']

    fig = go.Figure(data=go.Heatmap(
        z=z_values,
        x=x_labels,
        y=cohorts,
        colorscale=[[0, '#0E1117'], [1, '#48BB78']], # Dark to Green
        text=z_values,
        texttemplate="%{text}%",
        showscale=False
    ))
```

Reflex Dashboard Comprehensive Manual

```
fig.update_layout(
    template="plotly_dark",
    paper_bgcolor='rgba(0,0,0,0)',
    plot_bgcolor='rgba(0,0,0,0)',
    font_color=TEXT_COLOR,
    margin=dict(l=40, r=20, t=20, b=20),
    height=350,
    xaxis=dict(side="top", tickfont=dict(color='gray')),
    yaxis=dict(autorange="reversed", tickfont=dict(color='white'))
)
return fig
```

File: `reflex_dashboard/state/data.py`

State Management Module: Handles data logic and events.

```
import reflex as rx
import pandas as pd
import io
from typing import List
from .filters import FilterState
from .store import DataStore

class DataState(FilterState):
    # Load data
    async def handle_sales_upload(self, files: List[rx.UploadFile]):
        """Handle sales data upload."""
        if not files:
            return

        for file in files:
            try:
                upload_data = await file.read()

                # Save to a temporary file or read directly if pandas supports bytes (it does for read_excel with engine
                # openpyxl usually, or BytesIO)
                # For simplicity and robustness with read_excel, let's wrap in BytesIO
                df = pd.read_excel(io.BytesIO(upload_data))

                if self.process_sales_data(df):
                    self.sales_data_uploaded = True

                if self.courier_data_uploaded:
                    self.deployment_status = "All Files Uploaded Successfully"
                else:
                    self.deployment_status = "Sales Data Uploaded Successfully"

            except Exception as e:
                print(f"Error processing sales upload: {e}")
                self.deployment_status = f"Error: {str(e)}"

    async def handle_courier_upload(self, files: List[rx.UploadFile]):
        """Handle courier data upload."""
        if not files:
            return

        for file in files:
            try:
                upload_data = await file.read()
```

Reflex Dashboard Comprehensive Manual

```
df = pd.read_excel(io.BytesIO(upload_data))
if self.process_courier_data(df):
    self.courier_data_uploaded = True

    if self.sales_data_uploaded:
        self.deployment_status = "All Files Uploaded Successfully"
    else:
        self.deployment_status = "Courier Data Uploaded Successfully"

except Exception as e:
    print(f"Error processing courier upload: {e}")
    self.deployment_status = f"Error: {str(e)}"

def download_sample_sales(self):
    """Download sample sales data."""
    return rx.download(url="/Uoload_ecom_sale.xlsx", filename="Sales_Data_Sample.xlsx")

def download_sample_courier(self):
    """Download sample courier data."""
    return rx.download(url="/Courier_details.xlsx", filename="Courier_Data_Sample.xlsx")

def process_sales_data(self, df: pd.DataFrame):
    """Process loaded sales dataframe."""
    try:
        # Usage of local temp_df prevents reactive state triggers during intermediate steps
        temp_df = df.copy()
        print(f"Data processed. Rows: {len(temp_df)}")

        # Column Mapping for Sales_Data.csv
        # Force all columns to uppercase and strip whitespace for consistent matching
        temp_df.columns = temp_df.columns.str.upper().str.strip()

        column_mapping = {
            "BILLING DATE": "Billing_Date", # Renamed from Month to avoid confusion
            "TOTAL VALUE": "Sales Amount",
            "BRAND DESCRIPTION": "Brand",
            "SKU QTY": "Quantity",
            "CHANNEL": "Channel",
            "PRODUCT": "Product",
            "CUSTOMER STATE": "CUSTOMER STATE", # Ensure this matches code usage
            "TYPE OF SUPPLY": "TYPE OF SUPPLY",
            "DOCUMENT DESCRIPTION": "DOCUMENT DESCRIPTION",
            "BILLING DOCUMENT": "BILLING DOCUMENT",
        }

        # VALIDATION: Check if this looks like a sales file
        # 1. Check if it's actually a Courier file (contains 'waybill_num')
        if "waybill_num" in temp_df.columns.str.lower():
            print("Warning: Courier file detected in Sales upload.")
            self.deployment_status = "?? Warning: You uploaded the Courier file to the Sales slot. Please upload the
Sales file."
            return False

        # 2. We expect at least TOTAL VALUE or Sales Amount
        if "TOTAL VALUE" not in temp_df.columns and "Sales Amount" not in temp_df.columns:
            raise ValueError("Invalid Sales File. Missing 'TOTAL VALUE' column. Did you upload the Courier file by
mistake?")

        temp_df = temp_df.rename(columns=column_mapping)
        # Remove duplicate columns to prevent "Grouper not 1-dimensional" errors
        temp_df = temp_df.loc[:, ~temp_df.columns.duplicated()]
```

Reflex Dashboard Comprehensive Manual

```
# Clean and Convert Data

# 1. Force categorical columns to string to avoid mixed types and comparison errors
for col in ["Brand", "Channel", "CUSTOMER STATE", "TYPE OF SUPPLY", "Product"]:
    if col in temp_df.columns:
        temp_df[col] = temp_df[col].astype(str).fillna("").str.strip()
        if col == "TYPE OF SUPPLY":
            # Standardize B2B/B2C
            temp_df[col] = temp_df[col].str.upper().replace({
                "BUSINESS TO BUSINESS": "B2B",
                "BUSINESS TO CONSUMER": "B2C",
                "DIRECT": "B2C", # Assumption if Direct exists
                "DEALER": "B2B", # Assumption
            })
        if col == "Channel":
            # Standardize Channel Names
            def map_channel(val):
                v_upper = val.upper()
                if "AMAZON" in v_upper: return "Amazon"
                if "FLIPKART" in v_upper: return "Flipkart"
                if "RCLUB" in v_upper: return "Rclub.in"
                if "BUSINESS CLUB" in v_upper: return "Rajnigandha Business Club"
                if "RAJNIGANDHA" in v_upper: return "Rajnigandha.com"
                if "B2B" in v_upper or "DEALER" in v_upper: return "B2B"
                return val

            temp_df[col] = temp_df[col].apply(map_channel)

        # Convert to category to reduce serialization size
        temp_df[col] = temp_df[col].astype("category")

# Handle Sales Amount
if temp_df["Sales Amount"].dtype == "object":
    temp_df["Sales Amount"] = temp_df["Sales Amount"].astype(str).str.replace(",", "", regex=False)
temp_df["Sales Amount"] = pd.to_numeric(temp_df["Sales Amount"], errors="coerce").fillna(0)

# Handle Quantity
if temp_df["Quantity"].dtype == "object":
    temp_df["Quantity"] = temp_df["Quantity"].astype(str).str.replace(",", "", regex=False)
temp_df["Quantity"] = pd.to_numeric(temp_df["Quantity"], errors="coerce").fillna(0)

# Handle Discount Amount
if "DISCOUNT AMOUNT" in temp_df.columns:
    if temp_df["DISCOUNT AMOUNT"].dtype == "object":
        temp_df["DISCOUNT AMOUNT"] = temp_df["DISCOUNT AMOUNT"].astype(str).str.replace(",", "", regex=False)
    temp_df["DISCOUNT AMOUNT"] = pd.to_numeric(temp_df["DISCOUNT AMOUNT"], errors="coerce").fillna(0)

# Handle Date
temp_df["Billing_Date"] = pd.to_datetime(temp_df["Billing_Date"], format="%d-%m-%Y", errors="coerce")
temp_df["Month_Date"] = temp_df["Billing_Date"].dt.to_period("M").dt.to_timestamp()
temp_df["Month_Label"] = temp_df["Month_Date"].dt.strftime('%B-%Y').astype("category")

# Populate Filter Options
# NOTE: We assume self.months etc are available via mixin inheritance in the final State class
self.months = sorted([m for m in temp_df['Month_Label'].unique().tolist() if m is not None], key=lambda x:
pd.to_datetime(x, format='%B-%Y', errors='coerce'))
self.states = sorted(temp_df['CUSTOMER STATE'].unique().tolist())
self.brands = sorted(temp_df['Brand'].unique().tolist())
self.channels = sorted(temp_df['Channel'].unique().tolist())
if 'TYPE OF SUPPLY' in temp_df.columns:
```

Reflex Dashboard Comprehensive Manual

```
self.supply_types = sorted(temp_df['TYPE OF SUPPLY'].unique().tolist())

# Default Selections (All)
self.selected_months = self.months
self.selected_states = self.states
self.selected_brands = self.brands
self.selected_channels = self.channels
self.selected_supply_types = self.supply_types

# Date Range Initialization
if not temp_df["Billing_Date"].isnull().all():
    min_d = temp_df["Billing_Date"].min()
    max_d = temp_df["Billing_Date"].max()
    self.start_date = min_d.strftime("%Y-%m-%d")
    self.end_date = max_d.strftime("%Y-%m-%d")
    self.min_date = self.start_date
    self.max_date = self.end_date
else:
    self.start_date = ""
    self.end_date = ""
    self.min_date = ""
    self.max_date = ""

# ATOMIC UPDATE using DataStore
try:
    # Use router.session.session_id to get the unique session ID
    token = self.router.session.session_id
    DataStore.set_sales_data(token, temp_df)
    print(f"Data successfully stored for session: {token}")
except Exception as e:
    print(f"DEBUG: Failed to store data (missing token?): {e}")
    print(f"DEBUG: Failed to store data (missing token?): {e}")
    # If we can't store the data, we can't really proceed with the dashboard
    return False

return True

except Exception as e:
    print(f"Error processing sales data: {e}")
    self.deployment_status += f" [Data Processing Error: {str(e)}] "
    return False

def process_courier_data(self, df: pd.DataFrame):
    try:
        temp_df = df.copy()
        # Normalize columns to lowercase to ensure 'gross_amount' etc match
        temp_df.columns = temp_df.columns.str.lower().str.strip()

        print(f"DEBUG: Original Courier Columns: {temp_df.columns.tolist()}")

        # --- COLUMN MAPPING LOGIC ---
        # Map various possible column names to our standard internal names
        column_map = {
            # Standard Target : [Possible Variations]
            "gross_amount": ["total amount", "cod amount", "bill value", "invoice value", "declared value"],
            "waybill_num": ["waybill no", "awb", "awb no", "docket no", "tracking number"],
            "status": ["current status", "delivery status", "shipment status"],
            "pickup_date": ["pickup date", "manifest date", "booking date"],
            "fpd": ["edd", "promised delivery date", "expected date"],
            "brand": ["client", "client name", "sender", "shipper"]
        }
    }
```


Reflex Dashboard Comprehensive Manual

```
rename_dict = {}
for standard, variations in column_map.items():
    # If standard name already exists, skip
    if standard in temp_df.columns:
        continue
    # Check variations
    for var in variations:
        if var in temp_df.columns:
            rename_dict[var] = standard
            break # Found a match, stop looking for this standard col

if rename_dict:
    print(f"DEBUG: Renaming columns: {rename_dict}")
    temp_df = temp_df.rename(columns=rename_dict)

# VALIDATION: Check for expected courier columns

# 1. Check if it's actually a Sales file
if "TOTAL VALUE" in [c.upper() for c in temp_df.columns] or "SALES AMOUNT" in [c.upper() for c in
temp_df.columns]:
    print("Warning: Sales file detected in Courier upload.")
    self.deployment_status = "?? Warning: You uploaded the Sales file to the Courier slot. Please upload the
Courier file."
    return False

# Based on courier_metrics usage: gross_amount, waybill_num, fpd, pickup_date, status
# We relax the strictness slightly but warn
required_cols = ["gross_amount", "waybill_num", "status"]
missing = [col for col in required_cols if col not in temp_df.columns]

if missing:
    print(f"ERROR: Missing Courier Columns: {missing}")
    # Try to provide a more helpful error
    raise ValueError(f"Invalid Courier File. Missing columns: {missing}. Found: {temp_df.columns.tolist()}")

# If brand/client column exists, filter for Rajnigandha.com if requested
# The user asked for "Coureir Performance(Only for Rajnigandha.com)"
# We check if there's a column that might indicate this.
if "brand" in temp_df.columns:
    # normalize content
    counts_before = len(temp_df)
    temp_df["brand"] = temp_df["brand"].astype(str).str.lower()
    # flexible matching
    mask = temp_df["brand"].str.contains("rajnigandha", na=False)
    if mask.any():
        temp_df = temp_df[mask]
        print(f"DEBUG: Filtered for Rajnigandha. Rows {counts_before} -> {len(temp_df)}")
    else:
        print("DEBUG: Brand column found but 'rajnigandha' not found. using all data.")

# Ensure numeric types
if "gross_amount" in temp_df.columns:
    temp_df["gross_amount"] = pd.to_numeric(temp_df["gross_amount"], errors='coerce').fillna(0)

try:
    # Use router.session.session_id
    token = self.router.session.session_id
    DataStore.set_courier_data(token, temp_df)
    print(f"Courier Data successfully stored for session: {token}. Columns: {temp_df.columns.tolist()}")
except Exception as e:
```

Reflex Dashboard Comprehensive Manual

```
        print(f"DEBUG: Failed to store courier data (missing token?): {e}")
        return False
    print(f"DEBUG: Courier Data Processed. Shape: {temp_df.shape}")
    return True
except Exception as e:
    print(f"DEBUG: Error processing courier data: {str(e)}")
    self.deployment_status = f"Error: {str(e)}"
    return False

# --- New Tabbed Workflow State ---
show_dashboard: bool = False

def start_analysis(self):
    """Switch to dashboard view."""
    if not self.sales_data_uploaded:
        self.deployment_status = "Please upload Sales Data before launching."
        return
    self.show_dashboard = True

# Intro Page State
show_intro: bool = True

def dismiss_intro(self):
    """Dismiss the intro page and show the main dashboard."""
    self.show_intro = False

@property
def filtered_df(self) -> pd.DataFrame:
    try:
        token = self.get_token()
        df_source = DataStore.get_sales_data(token)
    except AttributeError:
        # During initial compilation/static analysis, get_token might not be available
        return pd.DataFrame()

    if df_source is None or df_source.empty:
        return pd.DataFrame()

    # Apply Filters
    # Ensure Billing_Date is datetime for comparison
    # We rely on Billing_Date being correctly set in process_sales_data

    mask = (
        (df_source['CUSTOMER STATE'].isin(self.selected_states)) &
        (df_source['Brand'].isin(self.selected_brands)) &
        (df_source['Channel'].isin(self.selected_channels))
    )

    if self.start_date and self.end_date:
        mask = mask & (df_source['Billing_Date'] >= self.start_date) & (df_source['Billing_Date'] <= self.end_date)

    df = df_source[mask]

    if self.selected_supply_types and 'TYPE OF SUPPLY' in df.columns:
        df = df[df['TYPE OF SUPPLY'].isin(self.selected_supply_types)]

    return df

@property
def courier_df(self) -> pd.DataFrame:
    try:
```

Reflex Dashboard Comprehensive Manual

```
        token = self.get_token()
        df = DataStore.get_courier_data(token)
        if df is None: return pd.DataFrame()
        return df
    except AttributeError:
        return pd.DataFrame()

def load_data(self):
    """Check if data exists in memory on load, otherwise reset state to force re-upload."""
    token = self.get_token()
    dfs = DataStore.get_sales_data(token)

    if dfs is None or dfs.empty:
        if self.sales_data_uploaded:
            print(f"DEBUG: Data missing for session {token} but State thinks uploaded. Resetting.")
            self.sales_data_uploaded = False
            self.courier_data_uploaded = False
            self.show_dashboard = False
            self.deployment_status = "Session expired or server restarted. Please re-upload data."
```

File: reflex_dashboard/state/filters.py

State Management Module: Handles data logic and events.

```
import reflex as rx
import datetime
import calendar
from .base import BaseState

class FilterState(BaseState):
    # Filter Options
    months: list[str] = []
    states: list[str] = []
    brands: list[str] = []
    channels: list[str] = []
    supply_types: list[str] = []

    # Selected Filters
    selected_months: list[str] = []
    selected_states: list[str] = []
    selected_brands: list[str] = []
    selected_channels: list[str] = []
    selected_supply_types: list[str] = []

    # Date Range Filter
    start_date: str = ""
    end_date: str = ""
    min_date: str = ""
    max_date: str = ""

    def reset_filters(self):
        """Reset all filters to default state (Select All)."""
        self.selected_months = self.months
        self.selected_states = self.states
        self.selected_brands = self.brands
        self.selected_channels = self.channels
        self.selected_supply_types = self.supply_types
        # Reset dates to full range
        self.start_date = self.min_date
```

Reflex Dashboard Comprehensive Manual

```
self.end_date = self.max_date

def set_start_date(self, val: str):
    self.start_date = val

def set_end_date(self, val: str):
    self.end_date = val

# Helper properties for Select All/None logic
def toggle_all_months(self, select: bool):
    self.selected_months = self.months if select else []

def toggle_all_states(self, select: bool):
    self.selected_states = self.states if select else []

def toggle_all_brands(self, select: bool):
    self.selected_brands = self.brands if select else []

def toggle_all_channels(self, select: bool):
    self.selected_channels = self.channels if select else []

def toggle_all_supply(self, select: bool):
    self.selected_supply_types = self.selected_supply_types if select else []

# Toggle Individual Filters
def toggle_month(self, month: str, checked: bool):
    if checked:
        self.selected_months = self.selected_months + [month]
    else:
        self.selected_months = [m for m in self.selected_months if m != month]

def toggle_state(self, state: str, checked: bool):
    if checked:
        self.selected_states = self.selected_states + [state]
    else:
        self.selected_states = [s for s in self.selected_states if s != state]

def toggle_brand(self, brand: str, checked: bool):
    if checked:
        self.selected_brands = self.selected_brands + [brand]
    else:
        self.selected_brands = [b for b in self.selected_brands if b != brand]

def toggle_channel(self, channel: str, checked: bool):
    if checked:
        self.selected_channels = self.selected_channels + [channel]
    else:
        self.selected_channels = [c for c in self.selected_channels if c != channel]

def toggle_supply(self, supply: str, checked: bool):
    if checked:
        self.selected_supply_types = self.selected_supply_types + [supply]
    else:
        self.selected_supply_types = [s for s in self.selected_supply_types if s != supply]

# --- Custom Date Picker State ---
show_picker: bool = False
picker_target: str = "start" # "start" or "end"
picker_view_date: str = "" # The month we are looking at
picker_temp_date: str = "" # The date currently selected in the picker (before OK)
```

Reflex Dashboard Comprehensive Manual

```
def open_picker(self, target: str):
    self.picker_target = target
    current_val = self.start_date if target == "start" else self.end_date

    today_str = datetime.date.today().strftime("%Y-%m-%d")

    if current_val:
        self.picker_view_date = current_val
        self.picker_temp_date = current_val
    else:
        self.picker_view_date = today_str
        self.picker_temp_date = today_str

    self.show_picker = True

def close_picker(self):
    self.show_picker = False

def picker_prev_month(self):
    try:
        dt = datetime.datetime.strptime(self.picker_view_date, "%Y-%m-%d")
        first = dt.replace(day=1)
        prev = first - datetime.timedelta(days=1)
        self.picker_view_date = prev.strftime("%Y-%m-%d")
    except:
        pass

def picker_next_month(self):
    try:
        dt = datetime.datetime.strptime(self.picker_view_date, "%Y-%m-%d")
        days_in_month = calendar.monthrange(dt.year, dt.month)[1]
        next_month = dt + datetime.timedelta(days=days_in_month)
        self.picker_view_date = next_month.replace(day=1).strftime("%Y-%m-%d")
    except:
        pass

def picker_select_date(self, date_str: str):
    self.picker_temp_date = date_str

def picker_confirm(self):
    if self.picker_target == "start":
        self.set_start_date(self.picker_temp_date)
    else:
        self.set_end_date(self.picker_temp_date)
    self.show_picker = False

@rx.var
def picker_calendar_grid(self) -> list[dict]:
    """Returns the grid for the current view month."""
    if not self.picker_view_date: return []
    try:
        dt = datetime.datetime.strptime(self.picker_view_date, "%Y-%m-%d")
        year = dt.year
        month = dt.month

        c = calendar.Calendar(firstweekday=6) # Sunday start

        grid = []
        for date_obj in c.itermonthdates(year, month):
            is_current_month = (date_obj.month == month)
            date_str = date_obj.strftime("%Y-%m-%d")
```

Reflex Dashboard Comprehensive Manual

```
# Check strict equality for today and selected
# We return simple types
grid.append({
    "day": date_obj.day,
    "date_str": date_str,
    "is_current_month": is_current_month,
    "is_selected": (date_str == self.picker_temp_date),
    # "is_today": (date_str == datetime.date.today().strftime("%Y-%m-%d")) # Optional
})

return grid
except:
    return []

@rx.var
def picker_display_year(self) -> str:
    if not self.picker_temp_date: return ""
    try:
        return datetime.datetime.strptime(self.picker_temp_date, "%Y-%m-%d").strftime("%Y")
    except:
        return ""

@rx.var
def picker_display_date_formatted(self) -> str:
    if not self.picker_temp_date: return "Select date"
    try:
        # Keep this for compatibility if used elsewhere, but we primarily use the split ones below
        return datetime.datetime.strptime(self.picker_temp_date, "%Y-%m-%d").strftime("%a, %b %d")
    except:
        return "Select date"

@rx.var
def picker_display_weekday(self) -> str:
    if not self.picker_temp_date: return ""
    try:
        return datetime.datetime.strptime(self.picker_temp_date, "%Y-%m-%d").strftime("%A")
    except:
        return ""

@rx.var
def picker_display_day(self) -> str:
    if not self.picker_temp_date: return ""
    try:
        return datetime.datetime.strptime(self.picker_temp_date, "%Y-%m-%d").strftime("%d")
    except:
        return ""

@rx.var
def picker_display_month(self) -> str:
    if not self.picker_temp_date: return ""
    try:
        return datetime.datetime.strptime(self.picker_temp_date, "%Y-%m-%d").strftime("%B")
    except:
        return ""

@rx.var
def picker_month_year_title(self) -> str:
    if not self.picker_view_date: return ""
    try:
        return datetime.datetime.strptime(self.picker_view_date, "%Y-%m-%d").strftime("%B %Y")
```

Reflex Dashboard Comprehensive Manual

```
except:
    return ""

@rx.var
def dashboard_title(self) -> str:
    """Dynamic dashboard title based on date range."""
    base_title = "Interactive Sales Dashboard"
    if self.start_date and self.end_date:
        try:
            # Parse YYYY-MM-DD
            s_date = datetime.datetime.strptime(self.start_date, "%Y-%m-%d")
            e_date = datetime.datetime.strptime(self.end_date, "%Y-%m-%d")

            # Format as Apr'25 - Nov'25
            # Using %b for Month abbr, %y for 2-digit year
            s_str = s_date.strftime("%b'%y")
            e_str = e_date.strftime("%b'%y")

            return f"{base_title} ({s_str} - {e_str})"
        except Exception:
            return base_title
    return base_title
```

File: reflex_dashboard/state/kpi.py

State Management Module: Handles data logic and events.

```
import reflex as rx
import pandas as pd
from typing import Dict, List, Any
from .data import DataState

class KPIState(DataState):
    @rx.var
    def total_sales(self) -> str:
        if self.filtered_df.empty:
            return "?0.00 Cr"
        val = self.filtered_df["Sales Amount"].sum() / 10000000
        return f"?{val:,.2f} Cr"

    @rx.var
    def average_monthly_sale(self) -> str:
        if self.filtered_df.empty:
            return "?0.00 Cr"
        total_sales = self.filtered_df["Sales Amount"].sum() / 10000000
        unique_months = self.filtered_df["Month_Label"].nunique()
        if unique_months == 0:
            return "?0.00 Cr"
        avg = total_sales / unique_months
        return f"?{avg:,.2f} Cr"

    @rx.var
    def total_invoices(self) -> str:
        if self.filtered_df.empty:
            return "0"
        # Count only standard Sales Invoices (exclude Returns)
        invoices = self.filtered_df[self.filtered_df["DOCUMENT DESCRIPTION"] != "Sales Return"]
        return f"{invoices['BILLING DOCUMENT'].nunique():,}"
```

Reflex Dashboard Comprehensive Manual

```
@rx.var
def total_sales_return(self) -> str:
    if self.filtered_df.empty:
        return "0"

    # Count unique Return Invoices
    returns = self.filtered_df[self.filtered_df["DOCUMENT DESCRIPTION"] == "Sales Return"]
    return f"{returns['BILLING DOCUMENT'].nunique():,}"

@rx.var
def total_b2b_sales(self) -> str:
    if self.filtered_df.empty or "TYPE OF SUPPLY" not in self.filtered_df.columns:
        return "B2B: ?0.00 Cr"

    val = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2B"]["Sales Amount"].sum() / 10000000
    return f"B2B: ?{val:,.2f} Cr"

@rx.var
def total_b2c_sales(self) -> str:
    if self.filtered_df.empty or "TYPE OF SUPPLY" not in self.filtered_df.columns:
        return "B2C: ?0.00 Cr"

    val = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2C"]["Sales Amount"].sum() / 10000000
    return f"B2C: ?{val:,.2f} Cr"

@rx.var
def avg_monthly_b2b(self) -> str:
    if self.filtered_df.empty:
        return "?0.00 Cr"

    b2b_df = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2B"]
    total_sales = b2b_df["Sales Amount"].sum() / 10000000
    unique_months = self.filtered_df["Month_Label"].nunique()
    if unique_months == 0:
        return "?0.00 Cr"

    avg = total_sales / unique_months
    return f"?{avg:,.2f} Cr"

@rx.var
def avg_monthly_b2c(self) -> str:
    if self.filtered_df.empty:
        return "?0.00 Cr"

    b2c_df = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2C"]
    total_sales = b2c_df["Sales Amount"].sum() / 10000000
    unique_months = self.filtered_df["Month_Label"].nunique()
    if unique_months == 0:
        return "?0.00 Cr"

    avg = total_sales / unique_months
    return f"?{avg:,.2f} Cr"

# Trend Indicators
def calculate_mom(self, df) -> str:
    """Calculates Month-over-Month growth for the given dataframe."""
    if df.empty or "Month_Date" not in df.columns:
        return ""

    # Get monthly sums
    monthly_sales = df.groupby("Month_Date")["Sales Amount"].sum().sort_index()

    if len(monthly_sales) < 2:
        return ""

    # Get last two months
    current_month = monthly_sales.iloc[-1]
    previous_month = monthly_sales.iloc[-2]
```


Reflex Dashboard Comprehensive Manual

```
if previous_month == 0:
    return "? 100% MoM" # Avoid division by zero

growth = ((current_month - previous_month) / previous_month) * 100
direction = "?" if growth >= 0 else "?"
return f"{direction} {abs(growth):.1f}% MoM"

@rx.var
def total_sales_trend(self) -> str:
    return self.calculate_mom(self.filtered_df)

@rx.var
def b2b_trend(self) -> str:
    if self.filtered_df.empty or "TYPE OF SUPPLY" not in self.filtered_df.columns: return ""
    df = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2B"]
    return self.calculate_mom(df)

@rx.var
def b2c_trend(self) -> str:
    if self.filtered_df.empty or "TYPE OF SUPPLY" not in self.filtered_df.columns: return ""
    df = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2C"]
    return self.calculate_mom(df)

@rx.var
def average_monthly_sale_trend(self) -> str:
    # For average monthly sale, MoM of the average itself might be noisy.
    # Often it tracks the same trend as total sales if the # of months is constant in the window.
    # Let's use total sales trend for simplicity or implement specific logic if requested.
    return self.total_sales_trend

@rx.var
def total_invoices_trend(self) -> str:
    # Similar logic for invoices
    if self.filtered_df.empty or "Month_Date" not in self.filtered_df.columns:
        return ""

    monthly_inv = self.filtered_df[self.filtered_df["DOCUMENT DESCRIPTION"] != "Sales Return"] \
        .groupby("Month_Date")["BILLING DOCUMENT"].nunique().sort_index()

    if len(monthly_inv) < 2:
        return ""

    curr = monthly_inv.iloc[-1]
    prev = monthly_inv.iloc[-2]

    if prev == 0: return ""
    growth = ((curr - prev) / prev) * 100
    direction = "?" if growth >= 0 else "?"
    return f"{direction} {abs(growth):.1f}% MoM"

@rx.var
def total_sales_return_trend(self) -> str:
    return "" # Keep simple or implement logic

@rx.var
def courier_metrics(self) -> dict:
    """Calculates performance metrics for Courier Service."""
    # TRIGGER: Determine dependency on upload state to force re-calculation
    if not self.courier_data_uploaded:
        return {
```

Reflex Dashboard Comprehensive Manual

```
        "avg_cost": "?0.0",
        "avg_time": "0.0 days",
        "success_rate": "0.0%",
        "return_rate": "0.0%"
    }

# Use property access
df = self.courier_df
print(f"DEBUG: courier_metrics called. DF Shape: {df.shape}")

if not df.empty:
    print(f"DEBUG: DF Columns: {df.columns.tolist()}")

if df.empty:
    return {
        "avg_cost": "?0.0",
        "avg_time": "0.0 days",
        "success_rate": "0.0%",
        "return_rate": "0.0%"
    }

# Already copied in property access if using DataStore, but df is a reference.
# Copy to be safe from modifications if we do any below (we don't mutate, but safe practice)
df = df.copy()

# 1. Avg Delivery Cost = sum of gross_amount divided by count of waybill_num
df["gross_amount"] = pd.to_numeric(df["gross_amount"], errors='coerce').fillna(0)
total_gross = df["gross_amount"].sum()
total_waybills = df["waybill_num"].nunique() # Using nunique to be safe, or just len(df) if unique
avg_cost = total_gross / total_waybills if total_waybills > 0 else 0

# 2. Avg Delivery Time = fpd - pickup_date
# Only where fpd is present (implies delivered or attempted)
time_df = df.dropna(subset=["fpd", "pickup_date"]).copy()
avg_time = 0
if not time_df.empty:
    # Ensure datetime
    time_df["pickup_date"] = pd.to_datetime(time_df["pickup_date"])
    time_df["fpd"] = pd.to_datetime(time_df["fpd"])

    # Calculate duration in days
    # User formula: fpd - pickup_date
    duration = (time_df["fpd"] - time_df["pickup_date"]).dt.total_seconds() / (24 * 3600)
    avg_time = duration.mean()

# 3. Successful Delivery Rate = (Number of Successful Deliveries / Total Number of Deliveries) × 100%
# Successful = status 'Delivered'
# Total = Total rows
total_orders = len(df)

success_count = 0
bfs_count = 0

if "status" in df.columns:
    # Case insensitive match
    df["status_norm"] = df["status"].astype(str).str.lower().str.strip()

    # Check unique statuses for debugging
    unique_statuses = df["status"].unique().tolist()
    if len(unique_statuses) < 20:
        print(f"DEBUG: Unique Statuses found: {unique_statuses}")
```

Reflex Dashboard Comprehensive Manual

```
else:
    print(f"DEBUG: Unique Statuses found (first 20): {unique_statuses[:20]}")

    success_count = len(df[df["status_norm"] == "delivered"])

    # 4. Return Rate = (Number of courier Returned / Total Number of courier) x 100
    # Returned = status 'RTO'
    bfs_count = len(df[df["status_norm"] == "rto"])
else:
    print("DEBUG: 'status' column missing in courier metrics calculation")

success_rate = (success_count / total_orders * 100) if total_orders > 0 else 0
return_rate = (bfs_count / total_orders * 100) if total_orders > 0 else 0

return {
    "avg_cost": f"{avg_cost:.1f}",
    "avg_time": f"{avg_time:.1f} days",
    "success_rate": f"{success_rate:.1f}%",
    "return_rate": f"{return_rate:.1f}%"
}

# New KPI Calculations
@rx.var
def daily_sales_velocity_data(self) -> Dict[str, str]:
    if self.filtered_df.empty:
        return {"total": "?0 L", "b2b": "?0 L", "b2c": "?0 L"}

    total_sales = self.filtered_df["Sales Amount"].sum()

    # User defined: "total sales divided by unique number of billing date"
    # "Billing_Date" column holds the billing date as datetime
    days = self.filtered_df["Billing_Date"].nunique()

    daily_velocity = total_sales / days if days > 0 else 0

    # B2B/B2C Breakdown
    if "TYPE OF SUPPLY" in self.filtered_df.columns:
        b2b_sales = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2B"]["Sales Amount"].sum()
        b2c_sales = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2C"]["Sales Amount"].sum()
    else:
        b2b_sales = 0
        b2c_sales = 0

    b2b_daily = b2b_sales / days if days > 0 else 0
    b2c_daily = b2c_sales / days if days > 0 else 0

    # Format as Lacs (L)
    return {
        "total": f"{daily_velocity/100000:,.2f} L",
        "b2b": f"{b2b_daily/100000:,.2f}L",
        "b2c": f"{b2c_daily/100000:,.2f}L"
    }

@rx.var
def invoice_stats(self) -> Dict[str, str]:
    if self.filtered_df.empty:
        return {"count": "0", "daily_avg": "0"}

    invoices = self.filtered_df[self.filtered_df["DOCUMENT DESCRIPTION"] != "Sales Return"]
    unique_invoices = invoices["BILLING DOCUMENT"].nunique()
```

Reflex Dashboard Comprehensive Manual

```
# User defined: "Unique BILLING DOCUMENT ... divided by Count of unique BILLING DATE"
days = self.filtered_df["Billing_Date"].nunique()

daily_avg = unique_invoices / days if days > 0 else 0

return {
    "count": f"{unique_invoices:}",
    "daily_avg": f"~{int(daily_avg)}"
}

@rx.var
def return_metrics(self) -> Dict[str, str]:
    if self.filtered_df.empty:
        return {"count": "0", "rate_pct": "0.00%", "val_cr": "?0.00 Cr"}

    # Returns
    returns_df = self.filtered_df[self.filtered_df["DOCUMENT DESCRIPTION"] == "Sales Return"]
    unique_returns = returns_df["BILLING DOCUMENT"].nunique()

    # Total Invoices (for rate)
    invoices_df = self.filtered_df[self.filtered_df["DOCUMENT DESCRIPTION"] != "Sales Return"]
    total_invoices_count = invoices_df["BILLING DOCUMENT"].nunique()

    rate = (unique_returns / total_invoices_count * 100) if total_invoices_count > 0 else 0.0

    return_sum = returns_df["Sales Amount"].sum()
    return_val = return_sum / 10000000 # Crores

    return {
        "count": f"{unique_returns:}",
        "rate_pct": f"{rate:.2f}%",
        "val_cr": f"?{return_val:,.2f} Cr"
    }

@rx.var
def return_breakdown(self) -> List[Dict[str, str]]:
    if self.filtered_df.empty:
        return []

    # Group by Channel
    # Calculate Return Rate per Channel: (Unique Sales Return / Unique Invoices) * 100

    breakdown = []
    channels = self.filtered_df["Channel"].unique()

    for ch in channels:
        ch_df = self.filtered_df[self.filtered_df["Channel"] == ch]

        # Returns
        returns_df = ch_df[ch_df["DOCUMENT DESCRIPTION"] == "Sales Return"]
        unique_returns = returns_df["BILLING DOCUMENT"].nunique()

        # Invoices
        invoices_df = ch_df[ch_df["DOCUMENT DESCRIPTION"] != "Sales Return"]
        total_invoices_count = invoices_df["BILLING DOCUMENT"].nunique()

        rate = (unique_returns / total_invoices_count * 100) if total_invoices_count > 0 else 0.0

        breakdown.append({
            "channel": ch,
            "formatted_rate": f"{rate:.1f}%",
```

Reflex Dashboard Comprehensive Manual

```
        "rate": rate
    })

    # Sort by rate desc
    breakdown.sort(key=lambda x: x["rate"], reverse=True)

    return breakdown

@rx.var
def pareto_stats(self) -> Dict[str, Any]:
    if self.filtered_df.empty:
        return {"count_80": 0, "pct_catalog": "0.0%"}

    sales_df = self.filtered_df[self.filtered_df["Sales Amount"] > 0]

    product_sales = sales_df.groupby("Product")["Sales Amount"].sum().sort_values(ascending=False).reset_index()
    total_sales = product_sales["Sales Amount"].sum()

    if total_sales <= 0:
        return {"count_80": 0, "pct_catalog": "0.0%"}

    product_sales["cumulative_sales"] = product_sales["Sales Amount"].cumsum()
    product_sales["cumulative_pct"] = product_sales["cumulative_sales"] / total_sales

    cutoff_mask = product_sales["cumulative_pct"] <= 0.80
    count_80 = cutoff_mask.sum()

    if count_80 == 0 and not product_sales.empty:
        count_80 = 1
    elif count_80 < len(product_sales):
        count_80 += 1

    total_products = len(product_sales)
    pct_catalog = (count_80 / total_products * 100) if total_products > 0 else 0

    return {
        "count_80": str(count_80),
        "pct_catalog": f"({pct_catalog:.1f}% of catalog)"
    }

@rx.var
def pareto_top_products(self) -> List[Dict[str, str]]:
    if self.filtered_df.empty:
        return []

    sales_df = self.filtered_df[self.filtered_df["Sales Amount"] > 0]
    product_sales = sales_df.groupby("Product")["Sales Amount"].sum().sort_values(ascending=False).reset_index()
    total_sales = product_sales["Sales Amount"].sum()

    if total_sales <= 0:
        return []

    product_sales["pct"] = product_sales["Sales Amount"] / total_sales * 100

    # Take top 5 for display
    top_5 = product_sales.head(5)

    result = []
    for i, row in top_5.iterrows():
        val_cr = row["Sales Amount"] / 10000000
        result.append({
```

Reflex Dashboard Comprehensive Manual

```
        "rank": str(i + 1),
        "name": row["Product"],
        "value_cr": f"?{val_cr:.2f} Cr",
        "pct": f"{row['pct']:.1f}%"
    })

    return result
```

File: reflex_dashboard/state/predictive.py

State Management Module: Handles data logic and events.

```
import reflex as rx
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import numpy as np
from .ai import AIState
from ..colors import CHART_COLORS, TEXT_COLOR, CARD_BG

class PredictiveState(AIState):
    """State for Predictive Engine."""

    revenue_data: list[dict] = []
    regression_data: list[dict] = []
    survival_data: list[dict] = []
    propensity_data: list[dict] = []
    bcg_data: list[dict] = []
    toxic_data: list[dict] = []

    # Phase 3 Data
    sku_forecast_data: list[dict] = []
    demand_data: list[dict] = []
    seasonality_data: list[dict] = []
    risk_data: list[dict] = []

    def load_predictive_data(self):
        """Load data from Excel file."""
        try:
            file_path = "assets/predictive_analytics_data.xlsx"

            # Load Revenue Growth
            df_rev = pd.read_excel(file_path, sheet_name='RevenueGrowth')
            self.revenue_data = df_rev.to_dict('records')

            # Load Regression
            df_reg = pd.read_excel(file_path, sheet_name='Regression')
            self.regression_data = df_reg.to_dict('records')

            # Load Survival
            df_surv = pd.read_excel(file_path, sheet_name='Survival')
            self.survival_data = df_surv.to_dict('records')

            # Load Propensity
            df_prop = pd.read_excel(file_path, sheet_name='Propensity')
            self.propensity_data = df_prop.to_dict('records')

            # Load BCG
            df_bcg = pd.read_excel(file_path, sheet_name='BCG')
```

Reflex Dashboard Comprehensive Manual

```
self.bcg_data = df_bcg.to_dict('records')

# Load Toxic SKU
df_toxic = pd.read_excel(file_path, sheet_name='ToxicSKU')
self.toxic_data = df_toxic.to_dict('records')

# --- Phase 3 Loading ---
# SKU Forecast
df_sku = pd.read_excel(file_path, sheet_name='SKUForecast')
self.sku_forecast_data = df_sku.to_dict('records')

# Demand Planning
df_demand = pd.read_excel(file_path, sheet_name='DemandPlanning')
self.demand_data = df_demand.to_dict('records')

# Seasonality
df_seas = pd.read_excel(file_path, sheet_name='Seasonality')
self.seasonality_data = df_seas.to_dict('records')

# Risk Alerts
df_risk = pd.read_excel(file_path, sheet_name='StockRisk')
self.risk_data = df_risk.to_dict('records')

except Exception as e:
    print(f"Error loading predictive data: {e}")

@rx.var
def revenue_growth_chart(self) -> go.Figure:
    if not self.revenue_data:
        return go.Figure()

    df = pd.DataFrame(self.revenue_data)

    fig = go.Figure()

    # Realized Sales Line
    fig.add_trace(go.Scatter(
        x=df['Month'],
        y=df['Realized Sales'],
        mode='lines+markers',
        name='Realized Sales',
        line=dict(color='#3B82F6', width=4), # Blue
        marker=dict(size=8)
    ))

    # Forecast Line
    fig.add_trace(go.Scatter(
        x=df['Month'],
        y=df['Forecast (AI)'],
        mode='lines+markers',
        name='Forecast (AI)',
        line=dict(color='#8B5CF6', width=4, dash='dot'), # Purple Dotted
        marker=dict(size=8, symbol='circle-open')
    ))

    fig.update_layout(
        template="plotly_dark",
        height=300,
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        font_color=TEXT_COLOR,
```

Reflex Dashboard Comprehensive Manual

```
        margin=dict(l=20, r=20, t=30, b=20),
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
        xaxis=dict(showgrid=False),
        yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)')
    )
    return fig

@rx.var
def regression_chart(self) -> go.Figure:
    if not self.regression_data:
        return go.Figure()

    df = pd.DataFrame(self.regression_data)

    # Sort values by Discount for clean line plotting
    df = df.sort_values(by="Discount")

    fig = go.Figure()

    # Scatter points (Actual Data)
    fig.add_trace(go.Scatter(
        x=df['Discount'],
        y=df['Sales Impact'],
        mode='markers',
        name='Actual Data',
        marker=dict(size=8, color='rgba(59, 130, 246, 0.6)', line=dict(width=1, color='#3B82F6'))
    ))

    # Trendline (Linear Regression Fit)
    # Using numpy polyfit for a cleaner trendline than connecting raw dots
    x = df['Discount']
    y = df['Sales Impact']
    slope, intercept = np.polyfit(x, y, 1)
    trend_line = slope * x + intercept

    fig.add_trace(go.Scatter(
        x=x,
        y=trend_line,
        mode='lines',
        name='Trend (Linear Fit)',
        line=dict(color='#F59E0B', width=3) # Amber for visibility
    ))

    fig.update_layout(
        template="plotly_dark",
        height=300,
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        font_color=TEXT_COLOR,
        margin=dict(l=20, r=20, t=30, b=20),
        showlegend=True,
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
        xaxis=dict(showgrid=False, title='Discount (%)'),
        yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)', title='Sales Impact ($)')
    )
    return fig

@rx.var
def survival_chart(self) -> go.Figure:
    if not self.survival_data:
        return go.Figure()
```


Reflex Dashboard Comprehensive Manual

```
df = pd.DataFrame(self.survival_data)

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=df['Time'],
    y=df['Survival Probability'],
    mode='lines+markers',
    name='Survival %',
    line=dict(color='#8B5CF6', width=3),
    marker=dict(size=8, color='#C4B5FD', line=dict(width=2, color='#8B5CF6'))
))

fig.update_layout(
    template="plotly_dark",
    height=300,
    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",
    font_color=TEXT_COLOR,
    margin=dict(l=20, r=20, t=30, b=20),
    xaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)'),
    yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)', range=[0.6, 1.05]),
    showlegend=True,
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
)
return fig

@rx.var
def propensity_chart(self) -> go.Figure:
    if not self.propensity_data:
        return go.Figure()

    df = pd.DataFrame(self.propensity_data)

    fig = go.Figure()
    fig.add_trace(go.Bar(
        x=df['Category'],
        y=df['Score'],
        name='Score',
        marker_color='#3B82F6'
    ))

    fig.update_layout(
        template="plotly_dark",
        height=300,
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        font_color=TEXT_COLOR,
        margin=dict(l=20, r=20, t=30, b=20),
        xaxis=dict(showgrid=False),
        yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)'),
        showlegend=True,
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
    )
    return fig

@rx.var
def bcg_matrix_chart(self) -> go.Figure:
    if not self.bcg_data:
        return go.Figure()
```

Reflex Dashboard Comprehensive Manual

```
df = pd.DataFrame(self.bcg_data)

# Define specific colors for each entity
color_map = {
    'Zepto': '#8B5CF6', # Violet
    'Blinkit': '#FACC15', # Yellow
    'Amazon': '#F97316', # Orange
    'Flipkart': '#3B82F6' # Blue
}

fig = px.scatter(
    df,
    x="Market Share",
    y="Growth Rate",
    size="Size",
    color="Entity",
    hover_name="Entity",
    size_max=50, # Slightly larger bubbles
    template="plotly_dark",
    color_discrete_map=color_map
)

fig.update_layout(
    height=300,
    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)", # Fully transparent
    font_color=TEXT_COLOR,
    margin=dict(l=20, r=20, t=20, b=20),
    xaxis=dict(title='Share', showgrid=True, gridcolor='rgba(255,255,255,0.1)'),
    yaxis=dict(title='Growth', showgrid=True, gridcolor='rgba(255,255,255,0.1)'),
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
)

return fig

@rx.var
def toxic_sku_chart(self) -> go.Figure:
    if not self.toxic_data:
        return go.Figure()

    df = pd.DataFrame(self.toxic_data)

    fig = px.scatter(
        df,
        x="Metric_X",
        y="Metric_Y",
        hover_name="SKU_ID",
        template="plotly_dark",
        color_discrete_sequence=['#EF4444'] # Red
    )

    fig.update_traces(marker=dict(size=8))

    fig.update_layout(
        height=300,
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)", # Fully transparent
        font_color=TEXT_COLOR,
        margin=dict(l=20, r=20, t=20, b=20),
        xaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)'),
        yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)'),
        showlegend=True,
```

Reflex Dashboard Comprehensive Manual

```
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
    )
    return fig

# --- Phase 3 Logic ---

@rx.var
def sku_forecast_chart(self) -> go.Figure:
    if not self.sku_forecast_data:
        return go.Figure()

    df = pd.DataFrame(self.sku_forecast_data)

    fig = go.Figure()

    # Actual Demand
    fig.add_trace(go.Scatter(
        x=df['Date'],
        y=df['Actual Demand'],
        mode='lines+markers',
        name='Actual Demand',
        line=dict(color='#10B981', width=3), # Emerald Green
        marker=dict(size=6)
    ))

    # AI Forecast
    fig.add_trace(go.Scatter(
        x=df['Date'],
        y=df['AI Forecast'],
        mode='lines+markers',
        name='AI Forecast',
        line=dict(color='#3B82F6', width=3, dash='dot'), # Blue Dotted
        marker=dict(size=6, symbol='diamond')
    ))

    fig.update_layout(
        template="plotly_dark",
        height=300,
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        font_color=TEXT_COLOR,
        margin=dict(l=20, r=20, t=30, b=20),
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
        xaxis=dict(showgrid=False),
        yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)')
    )
    return fig

@rx.var
def seasonality_chart(self) -> go.Figure:
    if not self.seasonality_data:
        return go.Figure()

    df = pd.DataFrame(self.seasonality_data)

    fig = go.Figure()

    fig.add_trace(go.Bar(
        x=df['Month'],
        y=df['Baseline Sales'],
        name='Baseline',
```

Reflex Dashboard Comprehensive Manual

```
        marker_color='#64748B' # Slate
    ))

    fig.add_trace(go.Bar(
        x=df['Month'],
        y=df['Promo Uplift'],
        name='Promo Uplift',
        marker_color='#F59E0B' # Amber
    ))

    fig.update_layout(
        barmode='stack',
        template="plotly_dark",
        height=300,
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        font_color=TEXT_COLOR,
        margin=dict(l=20, r=20, t=30, b=20),
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
        xaxis=dict(showgrid=False),
        yaxis=dict(showgrid=True, gridcolor='rgba(255,255,255,0.1)')
    )
    return fig

@rx.var
def demand_planning_display(self) -> list[dict]:
    if not self.demand_data:
        return []

    data = self.demand_data
    enhanced_data = []
    for row in data:
        status_color = "gray"
        if row['Status'] == 'Critical Low': status_color = "red"
        elif row['Status'] == 'Reorder Soon': status_color = "yellow"
        elif row['Status'] == 'Overstock': status_color = "orange"
        elif row['Status'] == 'Healthy': status_color = "green"

        row['status_color'] = status_color
        enhanced_data.append(row)

    return enhanced_data

@rx.var
def risk_alerts(self) -> list[dict]:
    if not self.risk_data:
        return []
    return self.risk_data
```

File: reflex_dashboard/state/store.py

State Management Module: Handles data logic and events.

```
import pandas as pd
from typing import Dict, Optional

class DataStore:
    """
    Global in-memory store for large datasets to avoid Reflex State serialization issues.
```

Reflex Dashboard Comprehensive Manual

```
Data is keyed by session token to ensure multi-user isolation.
"""
_sales_data: Dict[str, pd.DataFrame] = {}
_courier_data: Dict[str, pd.DataFrame] = {}

@classmethod
def set_sales_data(cls, token: str, df: pd.DataFrame):
    cls._sales_data[token] = df

@classmethod
def get_sales_data(cls, token: str) -> Optional[pd.DataFrame]:
    return cls._sales_data.get(token)

@classmethod
def set_courier_data(cls, token: str, df: pd.DataFrame):
    cls._courier_data[token] = df

@classmethod
def get_courier_data(cls, token: str) -> Optional[pd.DataFrame]:
    return cls._courier_data.get(token)

@classmethod
def clear_session(cls, token: str):
    if token in cls._sales_data:
        del cls._sales_data[token]
    if token in cls._courier_data:
        del cls._courier_data[token]
```

File: reflex_dashboard/state/tables.py

State Management Module: Handles data logic and events.

```
import reflex as rx
import pandas as pd
from typing import List, Dict
from .charts import ChartState
from ..colors import CHART_COLORS, TEXT_COLOR
from ..models import MonthlyValue, ChannelSummary, StateSummary, InvRetChannel, InvRetRow

class TableState(ChartState):

    @rx.var
    def product_data(self) -> list[dict]:
        """Top 10 Products + Others"""
        if self.filtered_df.empty:
            return []

        df = self.filtered_df.copy()
        if "Product" not in df.columns:
            return []

        total_revenue = df["Sales Amount"].sum()
        prod_grp = df.groupby("Product")["Sales Amount"].sum().reset_index().sort_values("Sales Amount", ascending=False)

        top_10 = prod_grp.head(10).to_dict("records")
        others_val = prod_grp.iloc[10:]["Sales Amount"].sum() if len(prod_grp) > 10 else 0

        results = []
        for i, r in enumerate(top_10):
```

Reflex Dashboard Comprehensive Manual

```
val = r["Sales Amount"]
pct = (val / total_revenue * 100) if total_revenue else 0

# Styling logic
rank_color = CHART_COLORS[i % len(CHART_COLORS)]
is_even = i % 2 == 0
row_bg = "rgba(255,255,255,0.03)" if is_even else "transparent"

results.append({
    "rank": str(i+1),
    "label": str(r["Product"]),
    "formatted_value": f"?{val / 10000000:.2f} Cr",
    "formatted_pct": f"{pct:.1f}%",
    "rank_bg": rank_color,
    "row_bg": row_bg,
})

if others_val > 0:
    pct_other = (others_val / total_revenue * 100) if total_revenue else 0
    results.append({
        "rank": "-",
        "label": "Other Products",
        "formatted_value": f"?{others_val / 10000000:.2f} Cr",
        "formatted_pct": f"{pct_other:.1f}%",
        "rank_bg": "#718096", # Gray for others
        "row_bg": "transparent",
    })

return results

@rx.var
def b2c_data(self) -> list[dict]:
    """B2C Average Order Value by Channel"""
    if self.filtered_df.empty:
        return []

    df = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2C"]
    if df.empty:
        return []

    stats = df.groupby("Channel").agg(
        sales=("Sales Amount", "sum"),
        orders=("BILLING DOCUMENT", "nunique")
    ).reset_index()

    # Filter out Blinkit and Zepto as requested
    stats = stats[~stats["Channel"].isin(["Blinkit", "Zepto"])]

    stats["aov"] = stats.apply(lambda x: x["sales"] / x["orders"] if x["orders"] > 0 else 0, axis=1)
    stats = stats.sort_values("aov", ascending=False)

    results = []
    for i, r in enumerate(stats.to_dict("records")):
        results.append({
            "channel": str(r["Channel"]),
            "formatted_sales": f"?{r['sales'] / 10000000:.2f} Cr",
            "orders": f"{r['orders']:, .0f}",
            "formatted_aov": f"?{r['aov']:, .0f}",
            "channel_color": CHART_COLORS[i % len(CHART_COLORS)],
        })
    return results
```

Reflex Dashboard Comprehensive Manual

```
@rx.var
def state_data(self) -> list[dict]:
    """Top-selling B2C Product per State"""
    if self.filtered_df.empty:
        return []

    df = self.filtered_df[self.filtered_df["TYPE OF SUPPLY"] == "B2C"]
    if df.empty:
        return []

    # Group by State and Product
    state_prod = df.groupby(["CUSTOMER STATE", "Product"])["Sales Amount"].sum().reset_index()
    # Find product with max sales for each state
    # Using sort_values + drop_duplicates is safer if idxmax is tricky
    top_per_state = state_prod.sort_values("Sales Amount", ascending=False).drop_duplicates(["CUSTOMER STATE"])

    # Calculate total B2C sales per state for contribution %
    state_total_sales = df.groupby("CUSTOMER STATE")["Sales Amount"].sum().to_dict()

    results = []
    for r in top_per_state.to_dict("records"):
        state_name = r["CUSTOMER STATE"]
        prod_sales = r["Sales Amount"]
        total_state_sales = state_total_sales.get(state_name, 0)
        pct_contribution = (prod_sales / total_state_sales * 100) if total_state_sales > 0 else 0

        results.append({
            "state": str(state_name),
            "product": str(r["Product"]),
            "formatted_sales": f"?{prod_sales / 100000:.2f} L",
            "formatted_pct": f"{pct_contribution:.1f}%",
            "formatted_total_sales": f"?{total_state_sales / 100000:.2f} L"
        })
    return results

@rx.var
def table_date_range_title(self) -> str:
    """Dynamic title suffix with date range."""
    if not self.start_date or not self.end_date:
        return ""

    try:
        s = pd.to_datetime(self.start_date).strftime("%B - %Y")
        e = pd.to_datetime(self.end_date).strftime("%B - %Y")
        return f"{{s}} To {{e}}"
    except:
        return ""

@rx.var
def summary_table_columns(self) -> list[str]:
    """Dynamic columns for the monthly summary table."""
    if self.filtered_df.empty:
        return []

    months = sorted(self.filtered_df["Month_Date"].unique())
    return [pd.Timestamp(m).strftime('%b') for m in months]

@rx.var
def monthly_summary_data(self) -> list[ChannelSummary]:
    """
    Data for Monthly Summary of Gross Sale Value (Channel wise) Table.
    """
```

Reflex Dashboard Comprehensive Manual

```
if self.filtered_df.empty:
    return []

df = self.filtered_df.copy()

# Ensure Month_Date is present
if "Month_Date" not in df.columns:
    return []

# Get all unique months sorted for columns
months = sorted(df["Month_Date"].unique())

# Helper to format value
def fmt(val):
    return f"?{val/100000000:.2f} Cr"

# 1. Group by Channel
channels = df["Channel"].unique()
summary_data = []

# Pre-initialize grand total monthly sums
gt_monthly_sums = {m: 0.0 for m in months}

for channel in channels:
    # Filter for this channel
    ch_df = df[df["Channel"] == channel]
    ch_total = ch_df["Sales Amount"].sum()

    # Channel Monthly Totals
    ch_monthly_grp = ch_df.groupby("Month_Date")["Sales Amount"].sum()
    ch_monthly_values = []
    for i, m in enumerate(months):
        val = ch_monthly_grp.get(m, 0.0)
        is_shade_col = (i % 2 == 0) # Shade even indices (Apr, Jun, etc.)
        ch_monthly_values.append(
            MonthlyValue(
                month=pd.Timestamp(m).strftime('%b'),
                value=fmt(val),
                is_shaded=is_shade_col
            )
        )
        gt_monthly_sums[m] += val

    # 2. Process States for this Channel
    state_grp = ch_df.groupby("CUSTOMER STATE")["Sales Amount"].sum().sort_values(ascending=False)

    top_5_states = state_grp.head(5)
    other_states_val = state_grp.iloc[5:].sum() if len(state_grp) > 5 else 0

    children = []

    # Add Top 5
    for state_name, state_total in top_5_states.items():
        st_df = ch_df[ch_df["CUSTOMER STATE"] == state_name]
        st_monthly_grp = st_df.groupby("Month_Date")["Sales Amount"].sum()
        st_monthly_vals = []
        for i, m in enumerate(months):
            val = st_monthly_grp.get(m, 0.0)
            is_shade_col = (i % 2 == 0)
            st_monthly_vals.append(
                MonthlyValue(
```


Reflex Dashboard Comprehensive Manual

```
        month=pd.Timestamp(m).strftime('%b'),
        value=fmt(val),
        is_shaded=is_shade_col
    )
)

children.append(
    StateSummary(
        state=str(state_name),
        monthly_values=st_monthly_vals,
        total_value=fmt(state_total)
    )
)

# Add "Other States" if exists
if other_states_val > 0:
    top_5_names = top_5_states.index.tolist()
    others_df = ch_df[~ch_df["CUSTOMER STATE"].isin(top_5_names)]

    oth_monthly_grp = others_df.groupby("Month_Date")["Sales Amount"].sum()
    oth_monthly_vals = []
    for i, m in enumerate(months):
        val = oth_monthly_grp.get(m, 0.0)
        is_shade_col = (i % 2 == 0)
        oth_monthly_vals.append(
            MonthlyValue(
                month=pd.Timestamp(m).strftime('%b'),
                value=fmt(val),
                is_shaded=is_shade_col
            )
        )

    children.append(
        StateSummary(
            state="Other States",
            monthly_values=oth_monthly_vals,
            total_value=fmt(other_states_val)
        )
    )

summary_data.append(
    ChannelSummary(
        channel=str(channel),
        monthly_values=ch_monthly_values,
        total_value=fmt(ch_total),
        children=children,
        is_total=False
    )
)

# Finalize Grand Total Row
gt_vals = []
gt_total_all = 0
for i, m in enumerate(months):
    val = gt_monthly_sums[m]
    is_shade_col = (i % 2 == 0)
    gt_vals.append(
        MonthlyValue(
            month=pd.Timestamp(m).strftime('%b'),
            value=fmt(val),
            is_shaded=is_shade_col
```

Reflex Dashboard Comprehensive Manual

```
        )
    )
    gt_total_all += val

summary_data.append(
    ChannelSummary(
        channel="Grand Total",
        monthly_values=gt_vals,
        total_value=fmt(gt_total_all),
        children=[],
        is_total=True
    )
)

return summary_data

@rx.var
def monthly_sales_return_data(self) -> list[ChannelSummary]:
    if self.filtered_df.empty:
        return []

    df = self.filtered_df[self.filtered_df["DOCUMENT DESCRIPTION"] == "Sales Return"].copy()

    # Ensure Month_Date is present
    if "Month_Date" not in self.filtered_df.columns:
        return []

    # Get all unique months from the main filtered_df to ensure we show all months
    months = sorted(self.filtered_df["Month_Date"].unique())

    # Helper to format value
    def fmt(val):
        # Format as absolute value with Indian numbering
        # val is negative for returns, so abs(val)
        s = str(int(abs(val)))
        if len(s) <= 3:
            res = s
        else:
            res = s[-3:]
            s = s[:-3]
            while len(s) > 2:
                res = s[-2:] + "," + res
                s = s[:-2]
            res = s + "," + res
        return f"?{res}"

    channels = df["Channel"].unique() if not df.empty else []

    summary_data = []
    gt_monthly_sums = {m: 0.0 for m in months}

    for channel in channels:
        ch_df = df[df["Channel"] == channel]
        ch_total = ch_df["Sales Amount"].sum()

        ch_monthly_grp = ch_df.groupby("Month_Date")["Sales Amount"].sum()
        ch_monthly_values = []

        for i, m in enumerate(months):
            val = ch_monthly_grp.get(m, 0.0)
            is_shade_col = (i % 2 == 0)
```

Reflex Dashboard Comprehensive Manual

```
        ch_monthly_values.append(
            MonthlyValue(
                month=pd.Timestamp(m).strftime('%b'),
                value=fmt(val),
                is_shaded=is_shade_col
            )
        )
        gt_monthly_sums[m] += val

    summary_data.append(
        ChannelSummary(
            channel=str(channel),
            monthly_values=ch_monthly_values,
            total_value=fmt(ch_total),
            children=[],
            is_total=False
        )
    )
```

```
# Finalize Grand Total Row
gt_vals = []
gt_total_all = 0
for i, m in enumerate(months):
    val = gt_monthly_sums[m]
    is_shade_col = (i % 2 == 0)
    gt_vals.append(
        MonthlyValue(
            month=pd.Timestamp(m).strftime('%b'),
            value=fmt(val),
            is_shaded=is_shade_col
        )
    )
    gt_total_all += val
```

```
summary_data.append(
    ChannelSummary(
        channel="Total Returns",
        monthly_values=gt_vals,
        total_value=fmt(gt_total_all),
        children=[],
        is_total=True
    )
)
```

```
return summary_data
```

```
@rx.var
```

```
def invoice_return_columns(self) -> list[str]:
    if self.filtered_df.empty:
        return []
    # Get unique channels + Grand Total
    channels = sorted(self.filtered_df["Channel"].unique().tolist())
    return channels
```

```
@rx.var
```

```
def invoice_vs_return_data(self) -> list[InvRetRow]:
    if self.filtered_df.empty:
        return []
```

```
df = self.filtered_df.copy()
months = sorted(df["Month_Date"].unique())
```

Reflex Dashboard Comprehensive Manual

```
channels = sorted(df["Channel"].unique().tolist())

# Prepare Data
rows = []

# Helper for totals
col_totals_inv = {c: 0 for c in channels}
col_totals_ret = {c: 0 for c in channels}
grand_total_inv_sum = 0
grand_total_ret_sum = 0

for m in months:
    m_df = df[df["Month_Date"] == m]
    month_label = pd.Timestamp(m).strftime('%b')

    channel_data = []

    row_total_inv = 0
    row_total_ret = 0

    for ch in channels:
        ch_data = m_df[m_df["Channel"] == ch]

        inv_count = ch_data[ch_data["DOCUMENT DESCRIPTION"] != "Sales Return"]["BILLING DOCUMENT"].nunique()
        ret_count = ch_data[ch_data["DOCUMENT DESCRIPTION"] == "Sales Return"]["BILLING DOCUMENT"].nunique()

        # Update Column Totals
        col_totals_inv[ch] += inv_count
        col_totals_ret[ch] += ret_count

        # Update Row Totals
        row_total_inv += inv_count
        row_total_ret += ret_count

        ret_color = "red" if ret_count > 0 else "black"

        channel_data.append(
            InvRetChannel(
                channel=ch,
                inv=str(inv_count) if inv_count > 0 else "-",
                ret=str(ret_count) if ret_count > 0 else "-",
                ret_color=ret_color
            )
        )

    grand_total_inv_sum += row_total_inv
    grand_total_ret_sum += row_total_ret

# Append Grand Total to channels list
channel_data.append(
    InvRetChannel(
        channel="Grand Total",
        inv=str(row_total_inv),
        ret=str(row_total_ret) if row_total_ret > 0 else "-",
        ret_color="red" if row_total_ret > 0 else "black"
    )
)

rows.append(
    InvRetRow(
        month=month_label,
```

Reflex Dashboard Comprehensive Manual

```
        channels=channel_data,
        grand_total_inv=str(row_total_inv),
        grand_total_ret=str(row_total_ret) if row_total_ret > 0 else "-"
    )
)

# Totals Row
total_channels = []
for ch in channels:
    total_channels.append(
        InvRetChannel(
            channel=ch,
            inv=str(col_totals_inv[ch]),
            ret=str(col_totals_ret[ch]),
            ret_color="red"
        )
    )

# Append Grand Total for Totals Row
total_channels.append(
    InvRetChannel(
        channel="Grand Total",
        inv=str(grand_total_inv_sum),
        ret=str(grand_total_ret_sum),
        ret_color="red"
    )
)

rows.append(
    InvRetRow(
        month="Total Count",
        channels=total_channels,
        grand_total_inv=str(grand_total_inv_sum),
        grand_total_ret=str(grand_total_ret_sum),
        is_total=True
    )
)

# Percentage Row
pct_channels = []
for ch in channels:
    inv = col_totals_inv[ch]
    ret = col_totals_ret[ch]
    pct = (ret / inv * 100) if inv > 0 else 0
    pct_channels.append(
        InvRetChannel(
            channel=ch,
            val=f"{pct:.2f}%",
            color="green"
        )
    )

gt_pct = (grand_total_ret_sum / grand_total_inv_sum * 100) if grand_total_inv_sum > 0 else 0
pct_channels.append(
    InvRetChannel(
        channel="Grand Total",
        val=f"{gt_pct:.2f}%",
        color=TEXT_COLOR # Changed from "green"
    )
)
)
```

Reflex Dashboard Comprehensive Manual

```
rows.append(  
    InvRetRow(  
        month="Return % (Ret/Inv)",  
        channels=pct_channels,  
        grand_total_val=f"{gt_pct:.2f}%",  
        is_pct=True  
    )  
)  
  
return rows
```